



Sunbeam Institute of Information Technology

Pune and Karad

Module – Data Structures and Algorithms

Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

- organizing data inside memory for efficient processing along with operations like add, delete, search etc.
e.g. stack - push/pop/peek.

used to achieve:—

- 1) Abstraction
 - Abstract Data Types
- 2) Reusability
- 3) Efficiency
 - time (time required to execute)
 - space (space inside memory)

Types

Linear

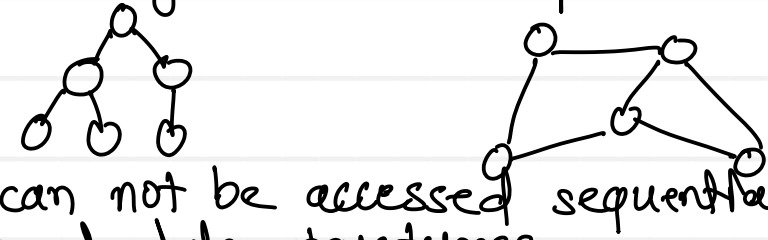
- data is organized linearly/sequentially.



- data can be accessed sequentially.
- Basic data structures
e.g. Array, structure/class, stack, Queue, Linked List

Non Linear

- data is organized in multiple levels (hierarchy)



- data can not be accessed sequentially.
- Advanced data structures
e.g. Tree, Graph

Algorithm

Program - set of instructions to machine (CPU)

Algorithm - set of instructions to human (developer)

- step by step solution of given problem statement

- written in human understandable languages.
- programming language independent
- templates/blue print.

Algorithm $\longrightarrow$ Programs

e.g. searching, sorting

- Find sum of array elements.

- 1) Create sum & initialize to 0
- 2) traverse array from start to end
- 3) add every element of array in sum
- 4) print/return sum variable

Linear search

- it works on unsorted data

- 1) decide / take key from user
- 2) traverse collection (array) from one end to another.
- 3) compare key with each element of the array.
- 4) if key is matching, return true/index.
- 5) if key is not found, return false/-1.

Binary Search

- it works on sorted data

- 1) Find middle element of array.
- 2) compare key with middle element.
- 3) if key is matching, return true/index.
- 4) if key is less than middle element, then search it in left side
- 5) if key is greater than middle element, then search it in right side
- 6) if key is not found, return false/-1

Linear search

arr	88	33	66	99	11	77	22	55	14	SIZE = 9
	0	1	2	3	4	5	6	7	8	
5	key == arr[i]					77	key is found return true / i			
i						key				
9						100	key is not found return false / -1			
i						key				

Binary search

88
key

arr

11	22	33	44	55	66	77	88	99
0	1	2	3	4	5	6	7	8
left				mid				right

mid = (left + right)/2

11	22	33	44
0	1	2	3
left			

66	77	88	99
5	6	7	8
left	mid		right

66		88	99
5		7	8
left		left mid	right

key is found

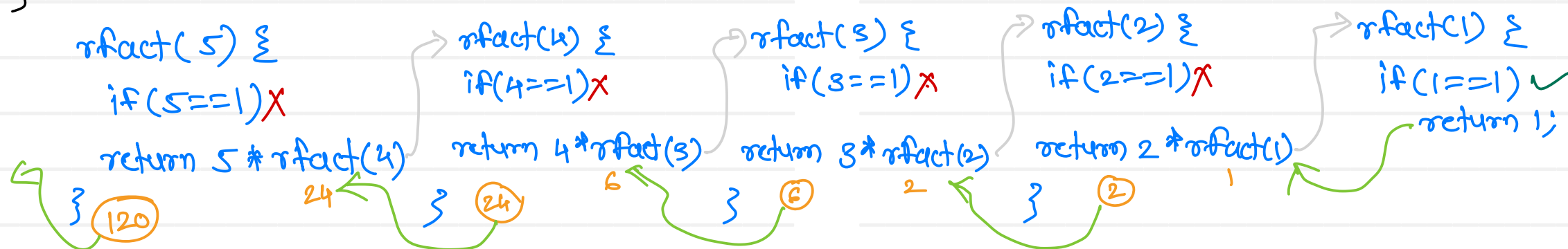
left partition --> left = left, right = mid-1
right partition --> left = mid+1, right = right



- calling function within itself
- we can use recursion
 - when we can define process/formula in terms of itself
 - when we know the terminating condition

```
int rfact(int num) {
    if (num == 1)
        return 1;
    return num * rfact(num - 1);
}
```

e.g. $n! = n * (n-1)!$
 $0! = 1! = 1$



Algorithm implementation approaches

Iterative

- loops are used

```
int fact(int num) {  
    int f=1;  
    for(int i=1; i<=num; i++)  
        f *= i;  
    return f;  
}
```

Recursive

- recursion is used

```
int rfact(int num) {  
    if(num==1)  
        return 1;  
    return num * rfact(num-1);  
}
```



Thank you!!!

Devendra Dhande

devendra.dhande@sunbeaminfo.com



Sunbeam Institute of Information Technology

Pune and Karad

Module – Data Structures and Algorithms

Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

Algorithm analysis

- it is done for efficiency measurement and also known as time/space complexity
- It is done to finding time and space requirements of the algorithm
 1. Time - time required to execute the algorithm (ns, us, ms, ...)
 2. Space - space required to execute the algorithm inside memory (bytes, kb, mb, ...)
- finding exact time and space of the algorithm is not possible because it depends on few external factors like
 - time is dependent on type of machine (CPU), number of processes running at that time
 - space is dependent on type of machine (architecture), data types
- Approximate time and space analysis of the algorithm is always done
- Mathematical approach is used to find time and space requirements of the algorithm and it is known as “Asymptotic analysis”
- Asymptotic analysis also tells about behaviour of the algorithm for different input or for change in sequence of input
- This behaviour of the algorithm is observed in three different cases
 1. Best case
 2. Average case
 3. Worst case

'Big O' notation is used to denote space & time complexity.

Time complexity

- time is directly proportional to number of iterations of the loops used in an algorithm
- To find time complexity/requirement of the algorithm count number of iterations of the loops

1. Print 1D array on console

```
void print1DArray(int arr[]) {  
    for(int i=0; i<n; i++)  
        sysout(arr[i]);  
}
```

No. of loop iterations = n

Time $\propto n$

$$T(n) = O(n)$$

2. Print 2D array on console

```
void print2DArray(int arr[][]){  
    for(int i=0; i<m; i++) {  
        for(int j=0; j<n; j++)  
            sysout(arr[i][j]);  
    }  
}
```

Iterations of outer loop = m

Iterations of inner loop = n

Total iteration = $m * n$

Time $\propto m * n$

$$T(m, n) = O(m * n)$$

$\because m = n$
Time $\propto n^2$
 $T(n) = O(n^2)$

3. Add two numbers

```
int sum(int n1, int n2) {  
    int s = n1 + n2;  
    return s;  
}
```

- time will be constant irrespective of values of $n1$ & $n2$
- constant time requirement

$$T(n) = O(1)$$

4. Print table of given number

```
void printTable(int num) {  
    for(int i=1; i<=10; i++)  
        sysout(num * i);  
}
```

- No. of iterations of loop = 10
- irrespective of value of num, loop will iterate constant number of times
 - constant time requirement.

$$T(n) = O(1)$$

5. Print binary of decimal number

2	9
	4
	2
	1

$(9)_{10} = (1001)_2$

n	n > 0	rem
9	T	1
4	T	0
2	T	0
1	T	1
0	F	

```

void printBinary (int n) {
    while (n > 0) {
        sysout (n % 2);
        n = n / 2;
    }
}

```

$$n = n, n/2, n/4, n/8, \dots$$

$$n = \frac{n}{2^0}, \frac{n}{2^1}, \frac{n}{2^2}, \dots, \frac{n}{2^{\text{itr}}}$$

$\therefore$ For $n=1$, last time loop will be executed.

$$\frac{n}{2^{\text{itr}}} = 1$$

$$n = 2^{\text{itr}}$$

$$\log 2^{\text{itr}} = \log n$$

$$\text{itr} \log 2 = \log n$$

$$\text{itr} = \frac{\log n}{\log 2}$$

$$\text{Time} \propto \text{itr}$$

$$\text{Time} \propto \frac{\log n}{\log 2}$$

$$T(n) = O(\log n)$$

Time complexity

Time complexities : $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(n^3)$, ..., $O(2^n)$,

Modification : + or - : time complexity is in terms of n

Modification : * or / : time complexity is in terms of $\log n$

for($i=0$; $i < n$; $i++$) $\rightarrow O(n)$

for($i=n$; $i > 0$; $i--$) $\rightarrow O(n)$

for($i=0$; $i < n$; $i+=20$) $\rightarrow O(n)$

for($i=1$; $i \leq 10$; $i++$) $\rightarrow O(1)$

for($i=n$; $i > 0$; $i/=2$) $\rightarrow O(\log n)$

for($i=1$; $i < n$; $i*=2$) $\rightarrow O(\log n)$

$n=9$

$i/=2 \rightarrow 9, 4, 2, 1$

$i*=2 \rightarrow 1, 2, 4, 8$

for($i=0$; $i < n$; $i++$)
for($j=0$; $j < n$; $j++$) $\rightarrow O(n^2)$

for($i=0$; $i < n$; $i++$) ; $\rightarrow \frac{n}{+} = 2n$ $T \propto 2n$
for($j=0$; $j < n$; $j++$) ; $\rightarrow \frac{n}{+} = 2n$ $O(n)$

for($i=0$; $i < n$; $i++$) $\rightarrow \frac{n}{*}$ $T \propto n \log n$
for($j=n$; $j > 0$; $j/=2$) $\rightarrow \log n$ $O(n \log n)$

Space complexity

- Finding approximate space requirement of the algorithm to execute inside memory

Total space

=

Input space

+

Auxiliary space

Find sum of array element

```
int sumArray(int arr[], int n)
{

```

```
    sum = 0;
```

```
    for(i=0; i<n; i++)
```

```
        sum += arr[i];
```

```
    return sum;
}
```

Auxiliary space $\rightarrow$ processing variables $\rightarrow n, i, \text{sum}$

Auxiliary space $\propto 3$

$AS(n) = O(1)$

space required
to store input

space required to
process the input

Input space $\rightarrow$ Input variables $\rightarrow \text{arr}$

Auxiliary space $\rightarrow$ Processing variables $\rightarrow n, i, \text{sum}$

Input space $= n$

Auxiliary space $= 3$

Total space $= n + 3$

space $\propto n + 3$

space $\propto n$

$\therefore n \gg 3$

$S(n) = O(n)$

Iterative

- loops are used

```
int fact(int num) {  
    int f=1;  
    for(int i=1; i<=num; i++)  
        f *= i;  
    return f;  
}
```

Time $\propto$ No. of iterations

Time $\propto n$

$T(n) = O(n)$

$AS(n) = O(1)$

Recursive

- recursion is used

```
int rfact(int num) {  
    if(num == 1)  
        return 1;  
    return num * rfact(num-1);  
}
```

Time $\propto$ No. of recursive calls

Time $\propto n$

$T(n) = O(n)$

$AS(n) = O(n)$

Searching algorithms analysis

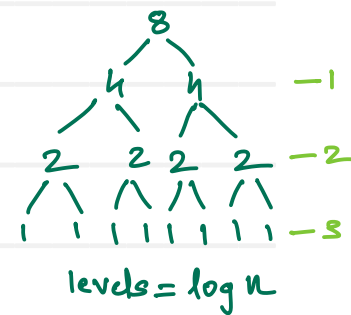
- Time is directly proportional to number of comparisons
- For searching and sorting algorithms, count number of comparisons done

1. Linear search

- Best case - Key is found at first few locations $\rightarrow O(1)$
- Average case - Key is found at middle few locations $\rightarrow O(n)$
- Worst case - Key is not found or found at last few locations $\rightarrow O(n)$

2. Binary search

- Best case - Key is found at first few levels $\rightarrow O(1)$
- Average case - Key is found at middle few levels $\rightarrow O(\log n)$
- Worst case - Key is found at last level or not found $\rightarrow O(\log n)$



Selection sort

1. Select one position of the array
2. Find smallest element out of remaining elements
3. Swap selected position element and smallest element
4. Repeat above steps until array is sorted $(N-1)$

No. of elements = n

$n = 6$

No. of passes = $n-1$

pass	comp	
1	5	$n-1$
2	4	$n-2$
3	3	$\vdots$
4	2	$\vdots$
5	1	1

$$\begin{aligned} \text{Total comps} &= 1 + 2 + \dots + (n-1) \\ &= \frac{n(n+1)}{2} \end{aligned}$$

$$\text{Time} \propto \frac{1}{2}(n^2 + n)$$

Mathematical Polynomial

Highest degree – degree of polynomial
- while writing time complexity consider only highest degree term because it is highest growing term in the polynomial

n	n^2
1	1
10	100
100	10000
1000	1000000

Best }
Avg }
Worst }

$$T(n) = O(n^2)$$

Selection sort

arr

44	11	55	22	66	33
0	1	2	3	4	5

Pass 1

44	11	55	22	66	33
0					
11	44	55	22	66	33

Pass 2

11	44	55	22	66	33
	1				j
11	22	55	44	66	33

Pass 3

11	22	55	44	66	33
		2			
11	22	33	44	66	55

Pass 4

11	22	33	44	66	55
			3		
11	22	33	44	66	55

Pass 5

11	22	33	44	66	55
				4	
11	22	33	44	55	66

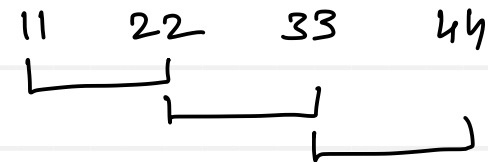
Bubble sort

1. Compare all pairs of consecutive elements of the array one by one
2. If right element is less than left element, swap both
3. Repeat above steps until array is sorted

$N=6$

	$j < N-1$	$j < N-i$
i	j	j
1	0-4	0-4
2	0-4	0-3
3	0-4	0-2
4	0-4	0-1
5	0-4	0-0

Best case



if array is sorted only $n-1$ comps will enough to check it.

$$T(n) = O(n)$$

No. of elements = n
No. of passes = $n-1$

$n=6$

Pass	Coms	
1	5	$n-1$
2	4	$n-2$
3	3	$\vdots$
4	2	$\vdots$
5	1	$\vdots$

$$\text{Total comps} = 1+2+3+\dots+(n-1)$$

$$= \frac{n(n+1)}{2}$$

$$\text{Time} \propto \frac{1}{2}(n^2+n)$$

Avg } $T(n) = O(n^2)$
Worst }

Bubble sort

33	22	66	55	44	11
0	1	2	3	4	5

33	22	66	55	44	11
----	----	----	----	----	----

22	33	66	55	44	11
----	----	----	----	----	----

22	33	66	55	44	11
----	----	----	----	----	----

22	33	55	66	44	11
----	----	----	----	----	----

22	33	55	44	66	11
----	----	----	----	----	----

22	33	55	44	11	66
----	----	----	----	----	----

22	33	55	44	11	66
----	----	----	----	----	----

22	33	55	44	11	66
----	----	----	----	----	----

22	33	55	44	11	66
----	----	----	----	----	----

22	33	44	55	11	66
----	----	----	----	----	----

22	33	44	11	55	66
----	----	----	----	----	----

22	33	44	11	55	66
----	----	----	----	----	----

22	33	44	11	55	66
----	----	----	----	----	----

22	33	44	11	55	66
----	----	----	----	----	----

22	33	11	44	55	66
----	----	----	----	----	----

22	33	11	44	55	66
----	----	----	----	----	----

22	33	11	44	55	66
----	----	----	----	----	----

22	11	33	44	55	66
----	----	----	----	----	----

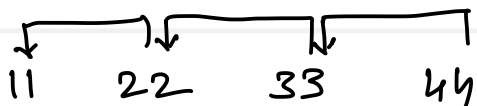
22	11	33	44	55	66
----	----	----	----	----	----

11	22	33	44	55	66
----	----	----	----	----	----

Insertion sort

1. Pick one element of the array (start from 2nd index)
2. Compare picked element with all its left neighbours one by one
3. If left neighbour is greater, move it one position ahead
4. Insert picked element at its appropriate position
5. Repeat above steps until array is sorted

Best case



if array is sorted only $n-1$ comps will enough to check it.

$$T(n) = O(n)$$

No. of elements = n

No. of passes = $n-1$

$n=6$

pass	comps
1	1
2	2
3	3
4	4
5	5

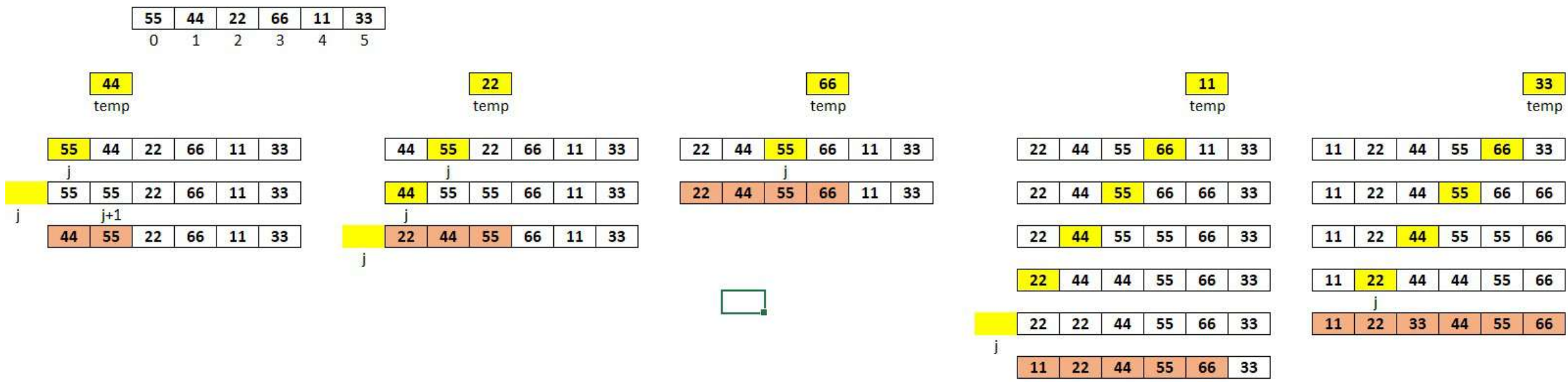
1
2
3
:
 $n-1$

$$\begin{aligned} \text{Total comps} &= 1 + 2 + 3 + \dots + (n-1) \\ &= \frac{n(n+1)}{2} \end{aligned}$$

$$\text{Time} \propto \frac{1}{2}(n^2 + n)$$

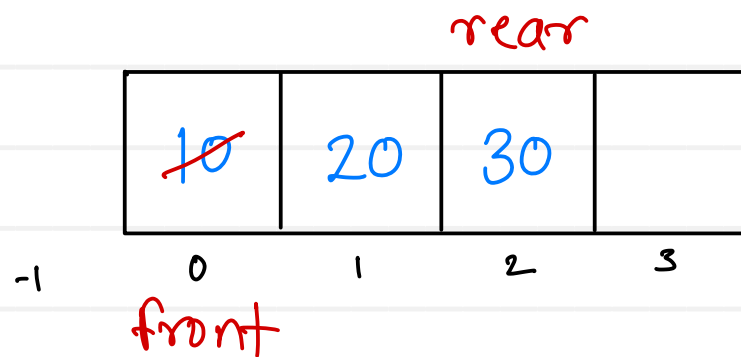
Avg } $T(n) = O(n^2)$
worst }

Insertion sort



Linear queue

- linear data structure which stores similar type of data.
- data is inserted from one end (rear)
- data is removed from another end (front)
- works on principle of "FIFO"/
"First In First Out"



Operations:

1) Push/add/insert/Enqueue:

- reposition rear (inc)
- add value at rear index

2) Pop/delete/remove/Dequeue:

- reposition front (inc)

3) Peek:

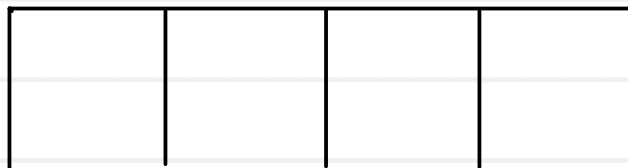
- read/return data of front+1 index

All operations of queue are performed in $O(1)$ time complexity.

Linear queue - Conditions

Empty

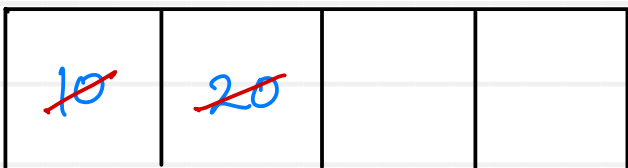
rear



-1 0 1 2 3

front

rear



-1 0 1 2 3

front

$front == rear$

Full

rear



-1 0 1 2 3

front

$rear == size - 1$

- if rear reaches last index of array and few initial elements are deleted, then still will not be able to use those empty locations. This will lead to poor memory utilization
- solution for this is circular queue



Thank you!!!

Devendra Dhande

devendra.dhande@sunbeaminfo.com



Sunbeam Institute of Information Technology

Pune and Karad

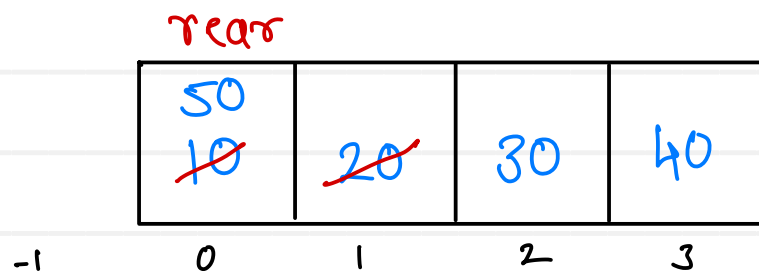
Module – Data Structures and Algorithms

Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

Circular queue

size=4



front

$$\text{front} = (\text{front} + 1) \% \text{size}$$

$$\text{rear} = (\text{rear} + 1) \% \text{size}$$

$$\text{front} = \text{rear} = -1$$

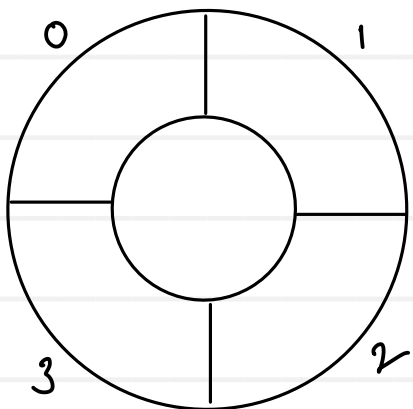
$$= (-1 + 1) \% 4 = 0$$

$$= (0 + 1) \% 4 = 1$$

$$= (1 + 1) \% 4 = 2$$

$$= (2 + 1) \% 4 = 3$$

$$= (3 + 1) \% 4 = 0$$



Operations:

1) Push/Enqueue :

- reposition rear (inc)
- add value at rear index

2) Pop/Dequeue :

- reposition front (dec)

3) Peek :

- read value of front+1 index

All operations are performed in $O(1)$ time complexity.

Circular queue - Conditions

Empty

r

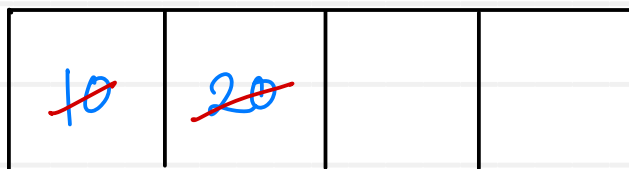


-1 0 1 2 3

f

$front == rear$ && $rear == -1$

r



-1 0 1 2 3

f

pop() {

$front = (front + 1) \% size;$

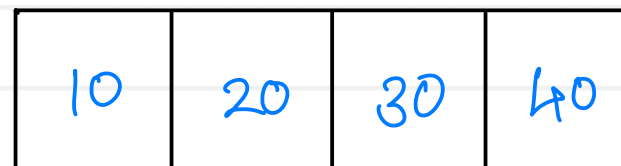
if ($front == rear$)

$front = rear = -1$

}

Full

r

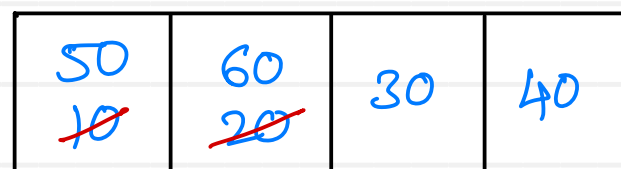


-1 0 1 2 3

f

$front == -1$ && $rear == size - 1$

r

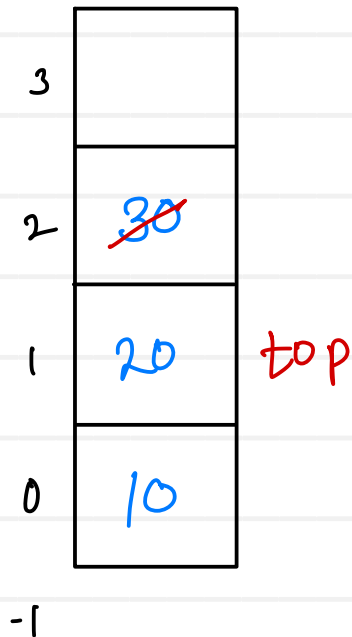


-1 0 1 2 3

f

$front == rear$ && $rear != -1$

- stack is linear data structure which stores similar type of data.
- stack has only one end (top)
- insert & delete both are allowed only from top end



- stack works on principle of "LIFO"/"Last In First Out".
- top always points to last inserted data.

Operations :

1) Add / push :

- 1) reposition top (inc)
- 2) add value at top index

2) Delete / pop :

- 1) reposition top (dec)

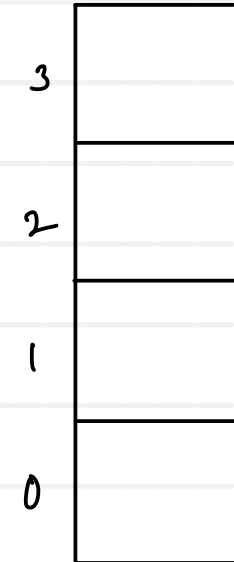
3) Peek :

- 1) read / return data of top index

All operations are performed in $O(1)$ time complexity.

Stack - Conditions

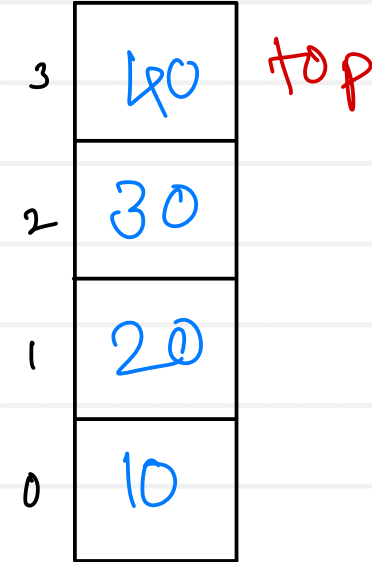
Empty



$top = -1$

$top == -1$

Full



-1

$top == size - 1$

Stack

- Parenthesis balancing
- Expression conversion and evaluation
- Function calls
- Used in advanced data structures for traversing
- **Expression conversion and evaluation:**
 - Infix to postfix
 - Infix to prefix
 - Postfix evaluation
 - Prefix evaluation

Expressions :

1) Infix : $a + b$ (human)

2) Postfix : $a b +$

3) Prefix : $+ a b$

} (machine)

Queue

- Jobs submitted to printer
- In Network setups – file access of file server machine is given to First come First serve basis
- Calls are placed on a queue when all operators are busy
- Used in advanced data structures to give efficiency.
- Process waiting queues in OS

Postfix Evaluation

- Process each element of postfix expression from left to right
- If element is operand
 - Push it on a stack
- If element is operator
 - Pop two elements (Operands) from stack, in such a way that
 - Op2 – first popped element
 - Op1 – second popped element
 - Perform current element (Operator) operation between Op1 and Op2
 - Again push back result onto the stack
- When single value will remain on stack, it is final result
- e.g. 4 5 6 * 3 / + 9 + 7 -

Postfix evaluation

Postfix expression : 4 5 6 * 3 / + 9 + 7 -

l $\longrightarrow$ r

Result : 16

- ⑤ $23 - 7 = 16$
- ④ $14 + 9 = 23$
- ③ $4 + 10 = 14$
- ② $30 / 3 = 10$
- ① $5 * 6 = 30$

16
7
23
9
14
10
3
30
6
5
4

Prefix Evaluation

- Process each element of prefix expression from right to left
- If element is operand
 - Push it on a stack
- If element is operator
 - Pop two elements (Operands) from stack, in such a way that
 - Op1 – first popped element
 - Op2 – second popped element
 - Perform current element (Operator) operation between Op1 and Op2
 - Again push back result onto the stack
- When single value will remain on stack, it is final result
- e.g. - + + 4 / * 5 6 3 9 7

Prefix evaluation

Prefix expression : - + + 4 / * 5 6 3 9 7

l ← ————— r

Result : 16

$$\begin{aligned}
 & \textcircled{5} \quad 23 - 7 \\
 & \quad \quad = 16 \\
 & \textcircled{4} \quad 14 + 9 \\
 & \quad \quad = 23 \\
 & \textcircled{3} \quad 4 + 10 \\
 & \quad \quad = 14 \\
 & \textcircled{2} \quad 30 / 3 \\
 & \quad \quad = 10 \\
 & \textcircled{1} \quad 5 * 6 \\
 & \quad \quad = 30
 \end{aligned}$$

16
23
14
4
10
30
5
6
3
5
7

- Process each element of infix expression from left to right
- If element is Operand
 - Append it to the postfix expression
- If element is Operator
 - If priority of topmost element (Operator) of stack is greater or equal to current element (Operator), pop topmost element from stack and append it to postfix expression
 - Repeat above step if required
 - Push element on stack
- Pop all remaining elements (Operators) from stack one by one and append them into the postfix expression
- e.g. $a * b / c * d + e - f * h + i$

Infix to Postfix conversion

Infix expression : $1 \$ 9 + 3 * 4 - (6 + 8 / 2) + 7$

$\hookrightarrow$ $\rightarrow$

Postfix expression : $19\$34*+682/+ -7+$

$\rightarrow$

+
x
+
(
/
*
+
\$

Priority :
()
\$
* / %
+ -
↑

Infix to Prefix Conversion

- Process each element of infix expression from right to left
- If element is Operand
 - Append it to the prefix expression
- If element is Operator
 - If priority of topmost element of stack is greater than current element (Operator), pop topmost element from stack and append it to prefix expression
 - Repeat above step if required
 - Push element on stack
- Pop all remaining elements (Operators) from stack one by one and append them into the prefix expression
- Reverse prefix expression
- e.g. $a * b / c * d + e - f * h + i$

Infix to Prefix conversion

Infix expression : $1 \$ 9 + 3 * 4 - (6 + 8 / 2) + 7$

$\hookleftarrow$

Expression : $7 2 8 / 6 + 4 3 * 9 1 \$ + - +$

Prefix expression : $+ - + \$ 1 9 * 3 4 + 6 / 8 2 7$

$($

$\$$
$+$
$*$
$/$
$+$
$+$
$)$
$+$

Prefix to Postfix

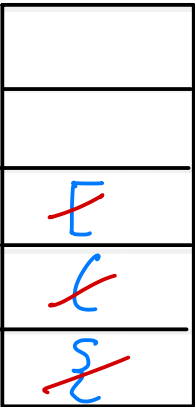
- Process each element of prefix expression from right to left
- If element is an Operand
 - Push it on to the stack
- If element is an Operator
 - Pop two elements (Operands) from stack, in such a way that
 - Op1 – first popped element
 - Op2 – second popped element
 - Form a string by concatenating Op1, Op2 and Opr (element)
 - String = “Op1+Op2+Opr”, push back on to the stack
- Repeat above two steps until end of prefix expression.
- Last remaining on the stack is postfix expression
- e.g. $* + a b - c d$

Postfix to Infix

- Process each element of postfix expression from left to right
- If element is an Operand
 - Push it on to the stack
- If element is an Operator
 - Pop two elements (Operands) from stack, in such a way that
 - Op2 – first popped element
 - Op1 – second popped element
 - Form a string by concatenating Op1, Opr (element) and Op2
 - String = “Op1+Opr+Op2”, push back on to the stack
- Repeat above two steps until end of postfix expression.
- Last remaining on the stack is infix expression
- E.g. a b c - + d e - f g - h + / *

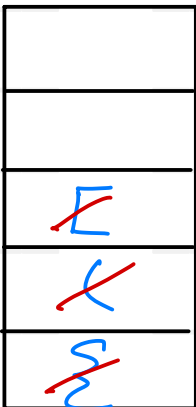
Parenthesis balancing using stack

{ ([]) }



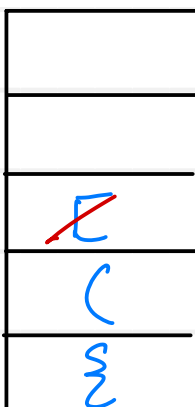
] == [
) == (
 } == {

{ ([]) })



] == [
) == (
 } == {
)

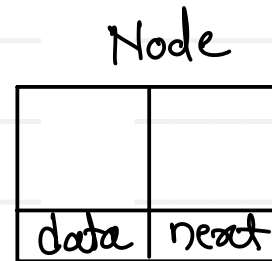
{ ([)] }



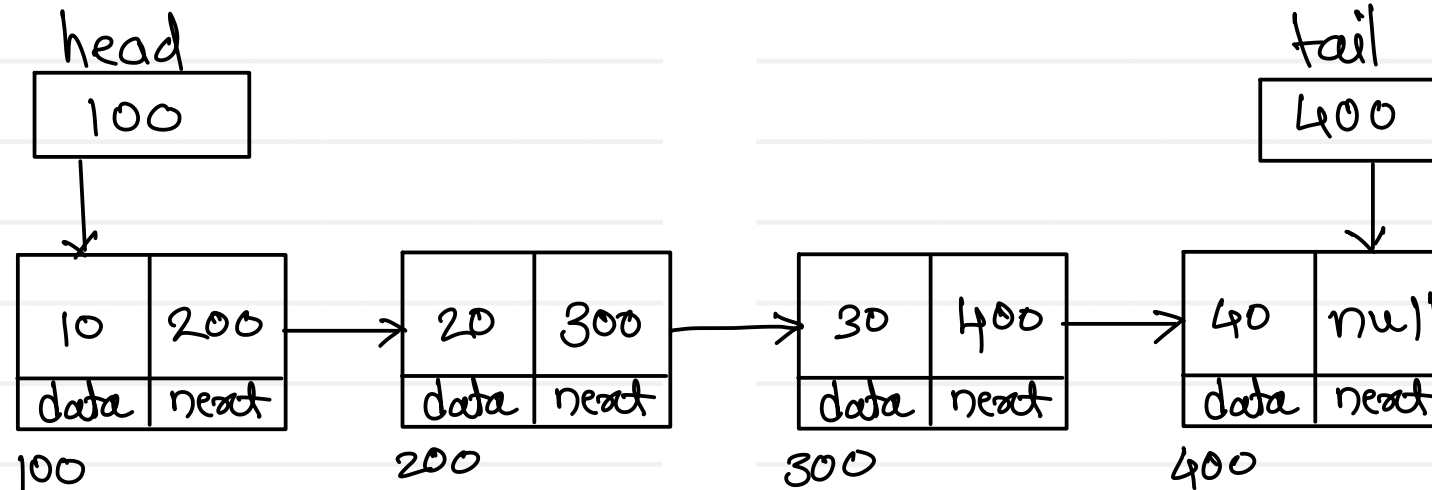
) != [

Linked List

- linear data structure
- Address/link of next data is kept with current data
- every element is known as "Node"
- every node has two parts:
 - 1) data : actual value
 - 2) link/next : add^r/reference of next data



- address/reference of first node is kept into head reference
- address/reference of last node is kept into tail reference (optional)



Operations

1. Add first
2. Add last
3. Add position (insert)

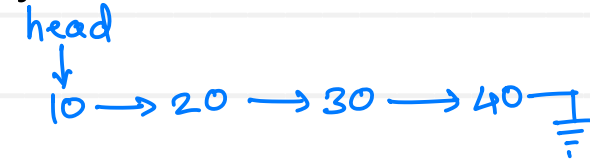
1. Delete first
2. Delete last
3. Delete position

1. Display (traverse) (forward/ backward)

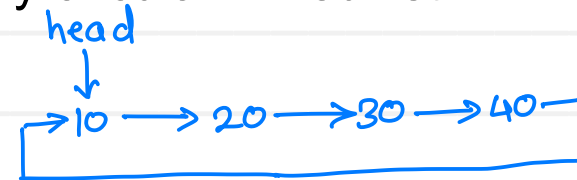
1. Search
2. Sort
3. Reverse

Types

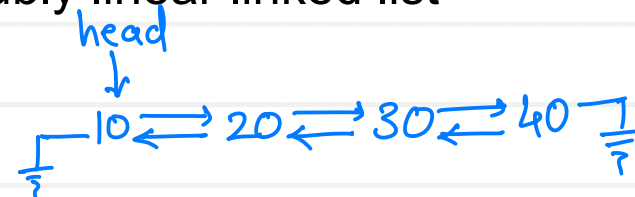
1. Singly linear linked list



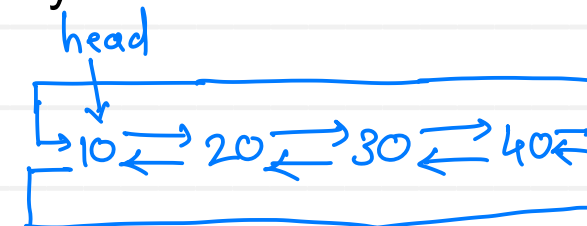
2. Singly circular linked list



3. Doubly linear linked list



4. Doubly circular linked list



Node has two parts

data : int, char, float, double, enum, class, string

next : reference

class List {
 static class Node {

class Node {
 int data;
 Node next;

}

why static? —————>
① no dependency of
List class to create
object of Node class

② private non static
fields of List class
should not be accessed
into Node class

static class Node {
 int data;
 Node next;

}
Node head, tail;

int size;

List() { --- }

add() { --- }

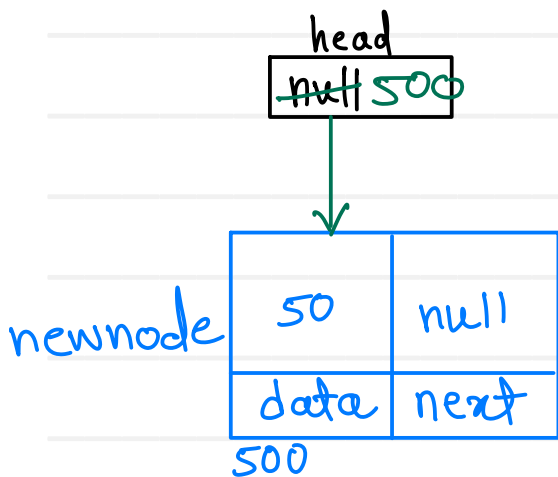
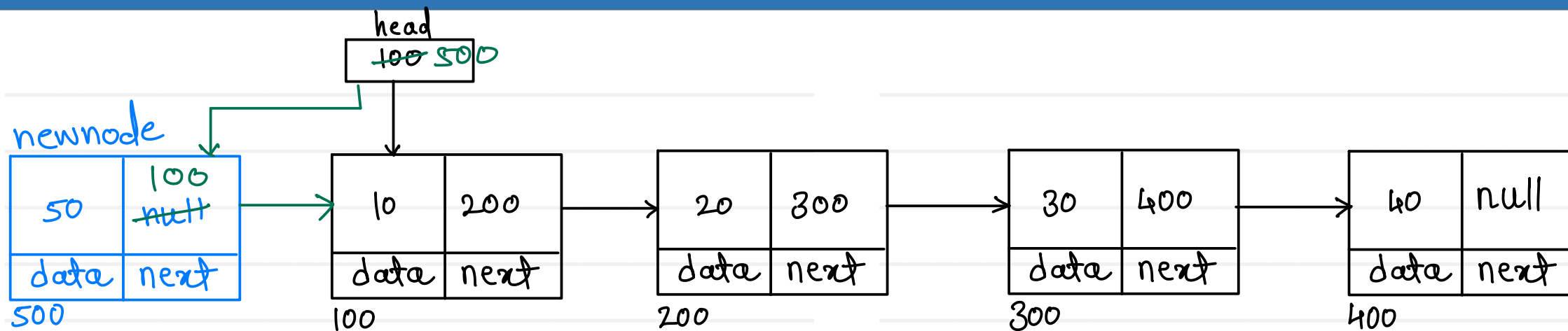
delete() { --- }

display() { --- }

deleteAll() { --- }

}

Singly linear Linked List - Add first

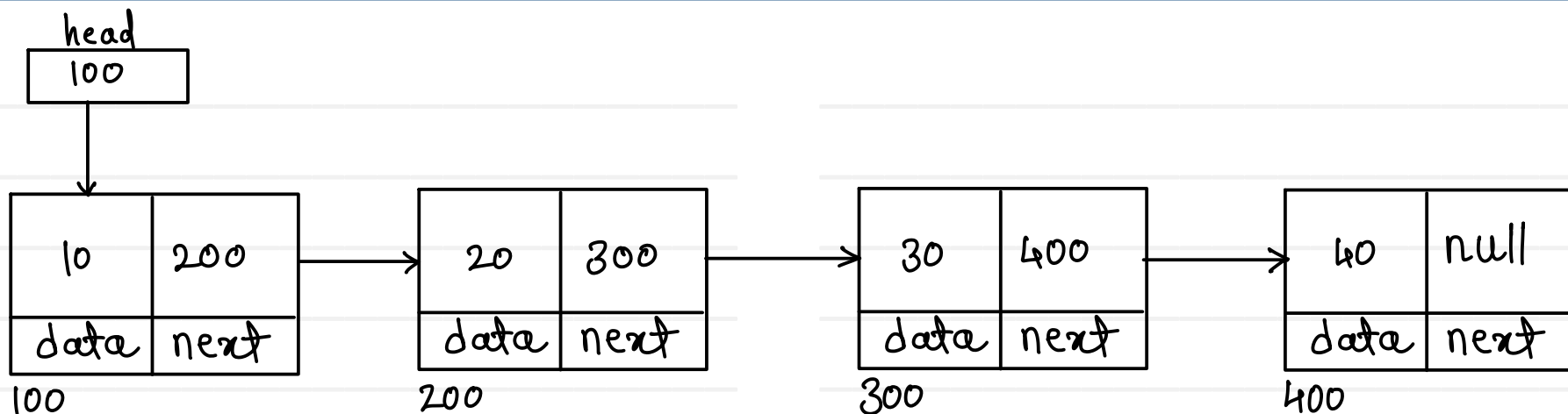


1. Create Node with given data/value.
2. add first node into next of newnode
3. Move head on newnode

newnode.next = head
head = newnode

$$T(n) = O(1)$$

Singly linear Linked List - Display

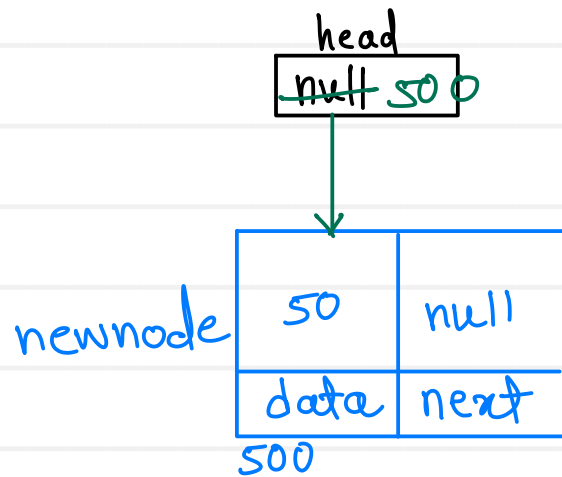
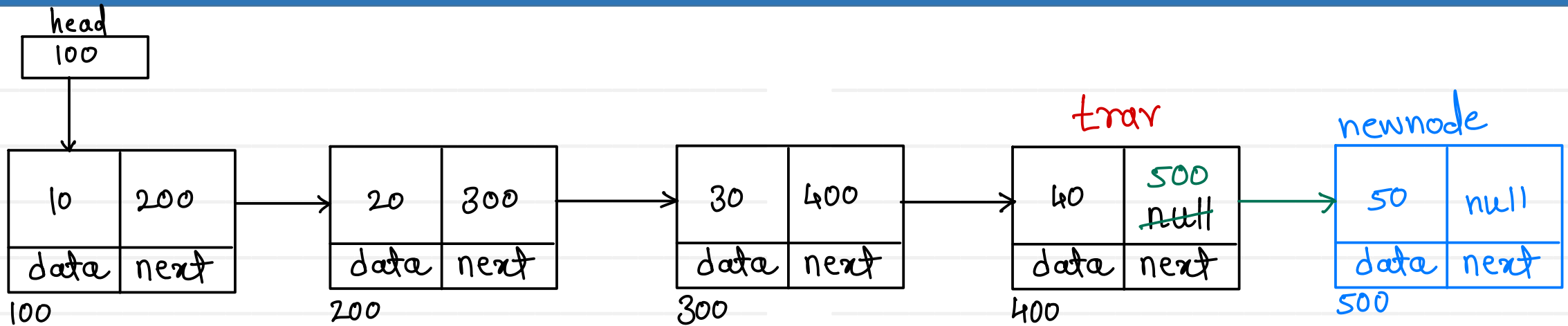


trav	trav.data	trav.next
100	10	200
200	20	300
300	30	400
400	40	null
null		

1. Create trav & start at head.
2. print/visit current node (trav.data)
3. go on next node (trav.next)
4. repeat above two steps for each node

$$T(n) = O(n)$$

Singly linear Linked List - Add last

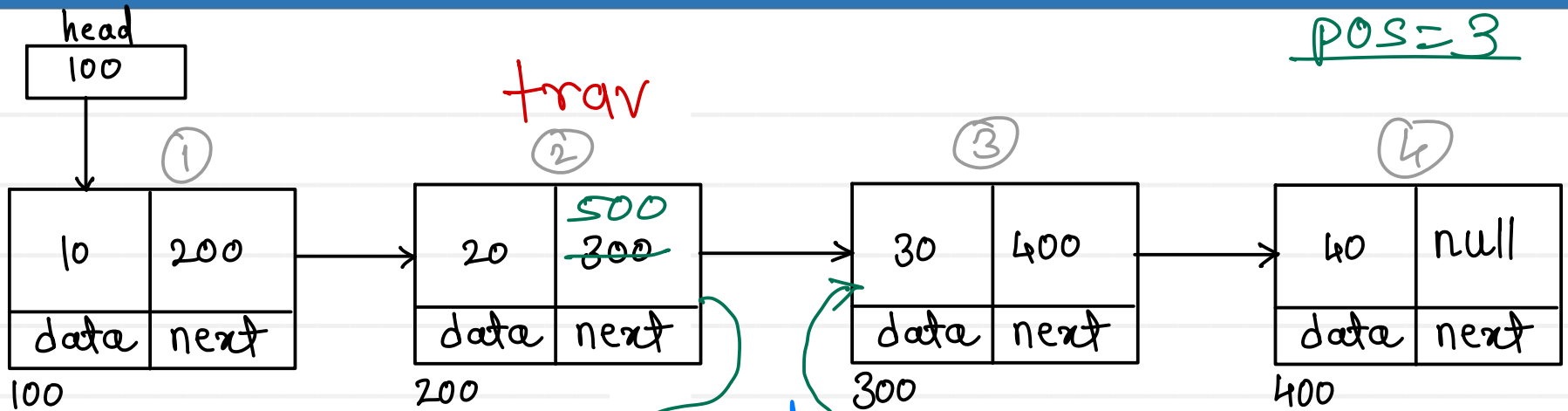


while (trav.next != null)
trav = trav.next;

1. Create Node with value
2. if list is empty
 - a. add newnode into head
3. if list is not empty
 - a. traverse till last node
 - b. add newnode into next of lastnode

$$T(n) = O(n)$$

Singly linear Linked List - Add position



1. Create node with data
2. if list is empty
head = newnode;
3. if list is not empty
 - a. traverse till pos-1 node
 - b. add pos node into next of newnode
 - c. add newnode into next of pos-1 node

$$T(n) = O(n)$$

Node trav = head;
for(int i=1; i < pos-1; i++)
trav = trav.next;

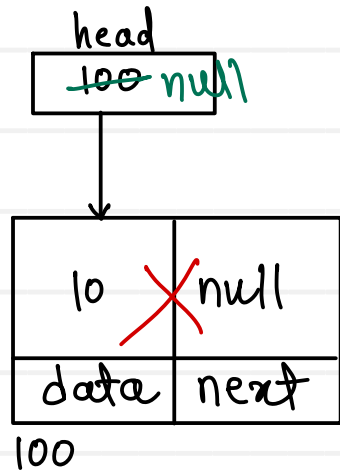
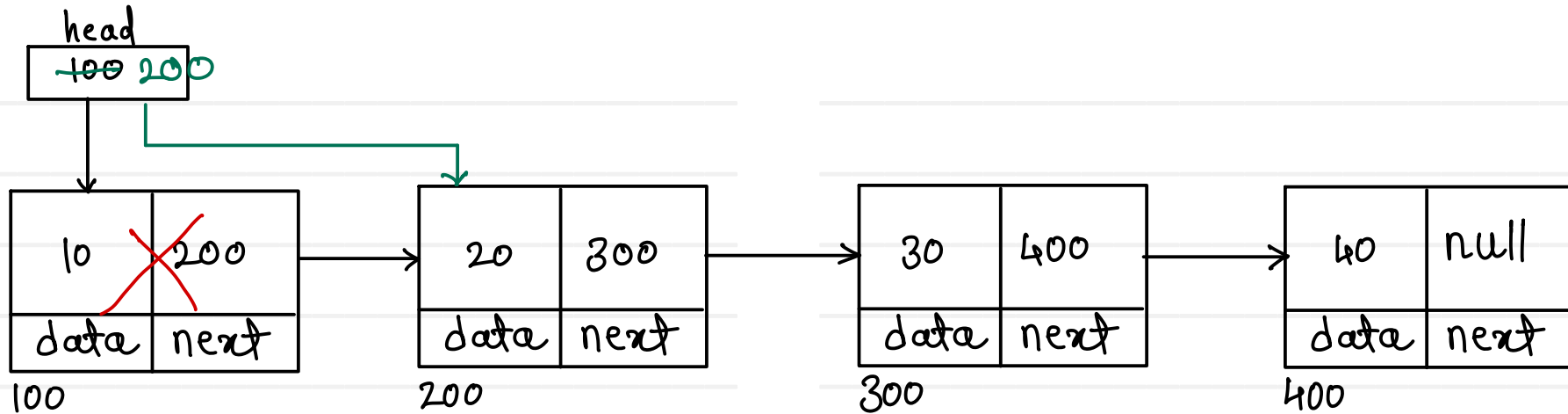
pos=3

trav	i	i < 2
100	1	T
200	2	F

pos=5

trav	i	i < 4
100	1	T
200	2	T
300	3	T
400	4	F

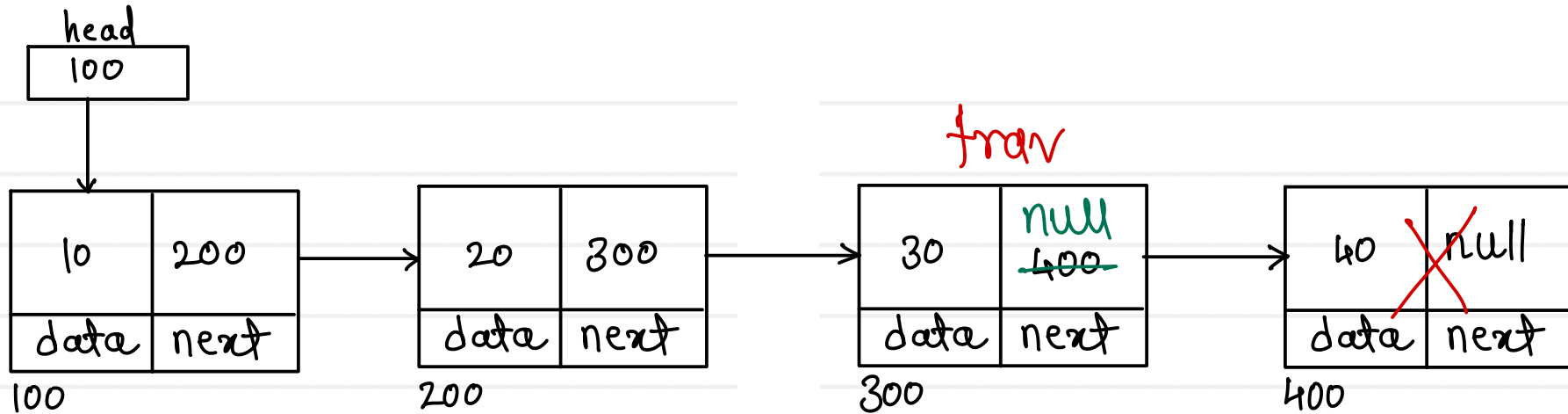
Singly linear Linked List - Delete first



1. if list is empty
return:
2. if list is not empty
move head on second node

$$T(n) = O(1)$$

Singly linear Linked List - Delete last



1. if list is empty
return;

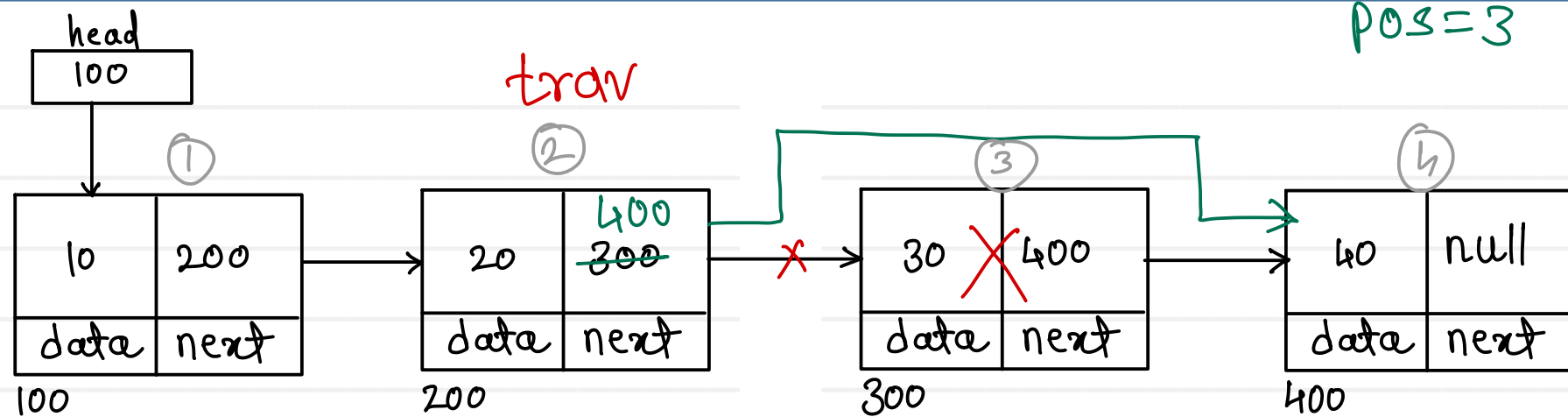
2. if list has single node
head = null;

3. if list has multiple nodes
a. traverse till second last node
b. add null into next of second last node

while (trav.next.next != null)
trav = trav.next;

$$T(n) = O(n)$$

Singly linear Linked List - Delete position



1. if list is empty
return;
2. if list has single node
head = null;
3. if list has multiple nodes
 - a. traverse till pos-1 node
 - b. add pos+1 node into next of pos-1 node

$$T(n) = O(n)$$



Thank you!!!

Devendra Dhande

devendra.dhande@sunbeaminfo.com



Sunbeam Institute of Information Technology

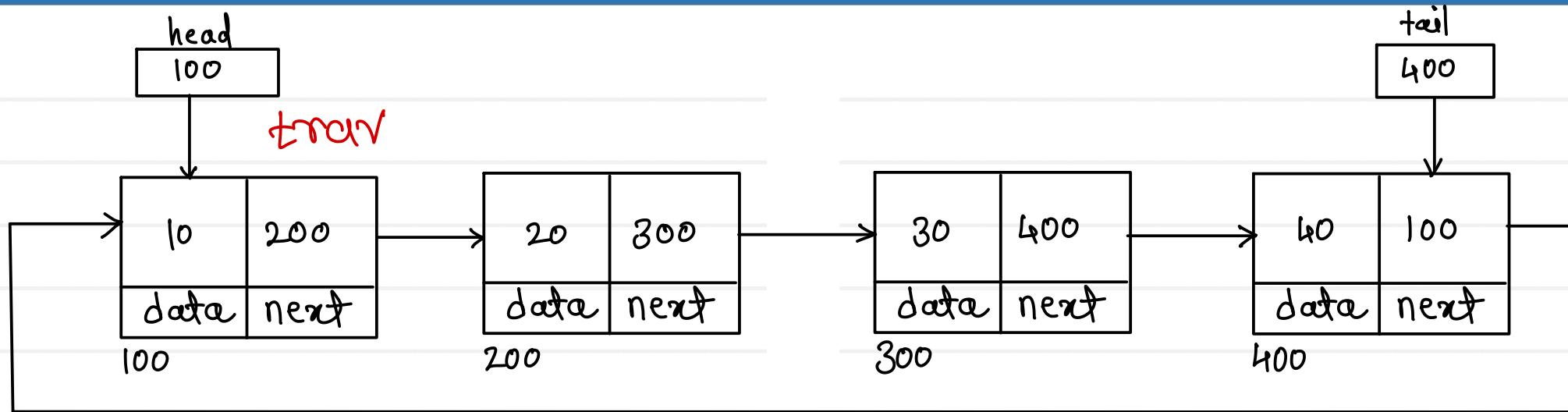
Pune and Karad

Module – Data Structures and Algorithms

Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

Singly Circular Linked List - Display

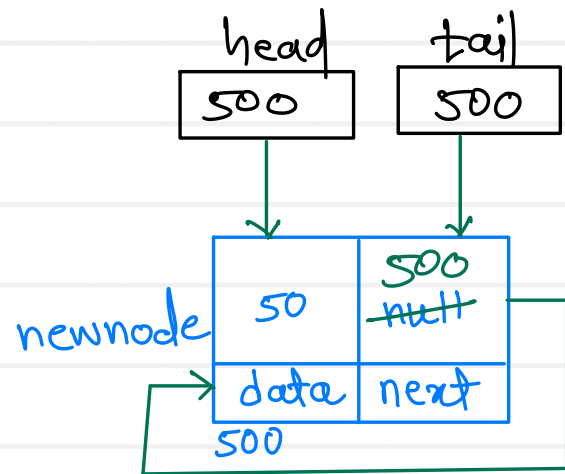
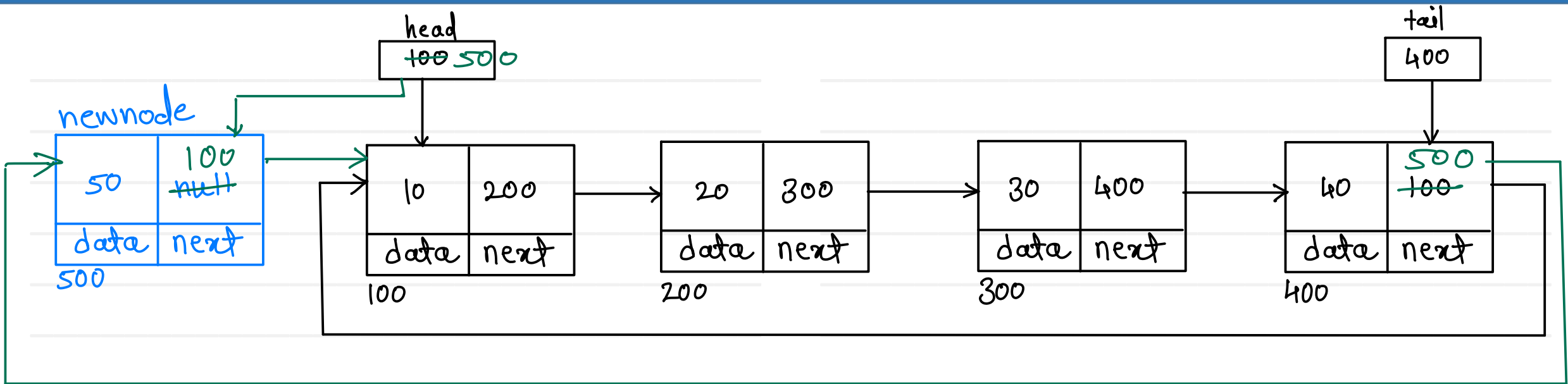


1. create trav & start at head.
2. print current node (trav.data)
3. go on next node (trav.next)
4. repeat above two step till last node

$$T(n) = O(n)$$

```
Node trav = head;  
do {  
    sysout(trav.data);  
    trav = trav.next;  
} while (trav != head);
```

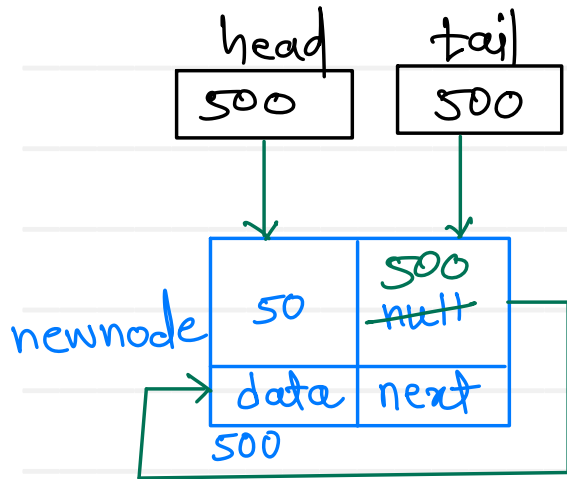
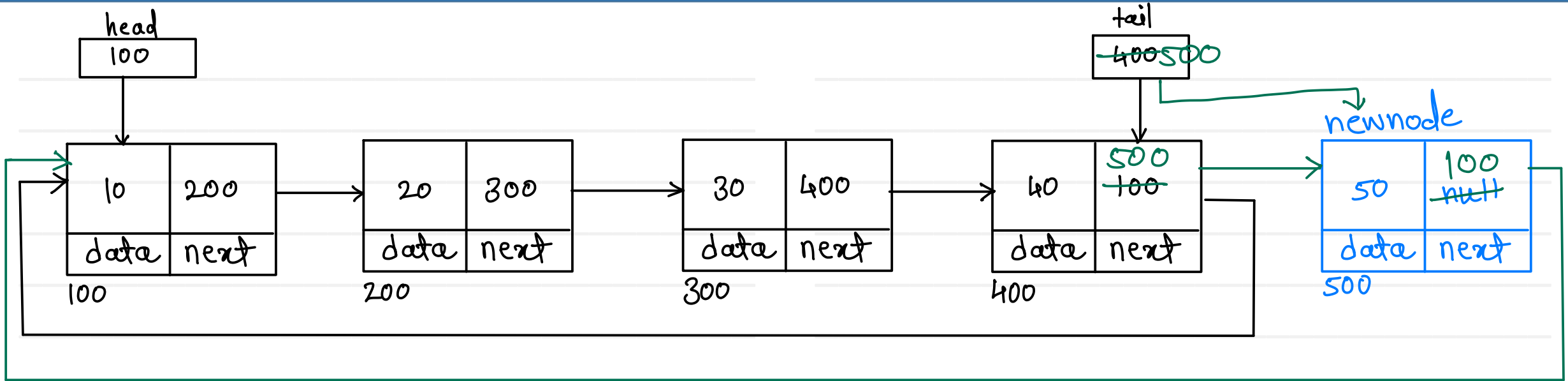
Singly Circular Linked List - Add first



1. Create node
2. if list is empty
 - a. add newnode into head & tail
 - b. make list circular
3. if list is not empty
 - a. add first node into next of newnode
 - b. add newnode into next of last node
 - c. move head on newnode

$$T(n) = O(1)$$

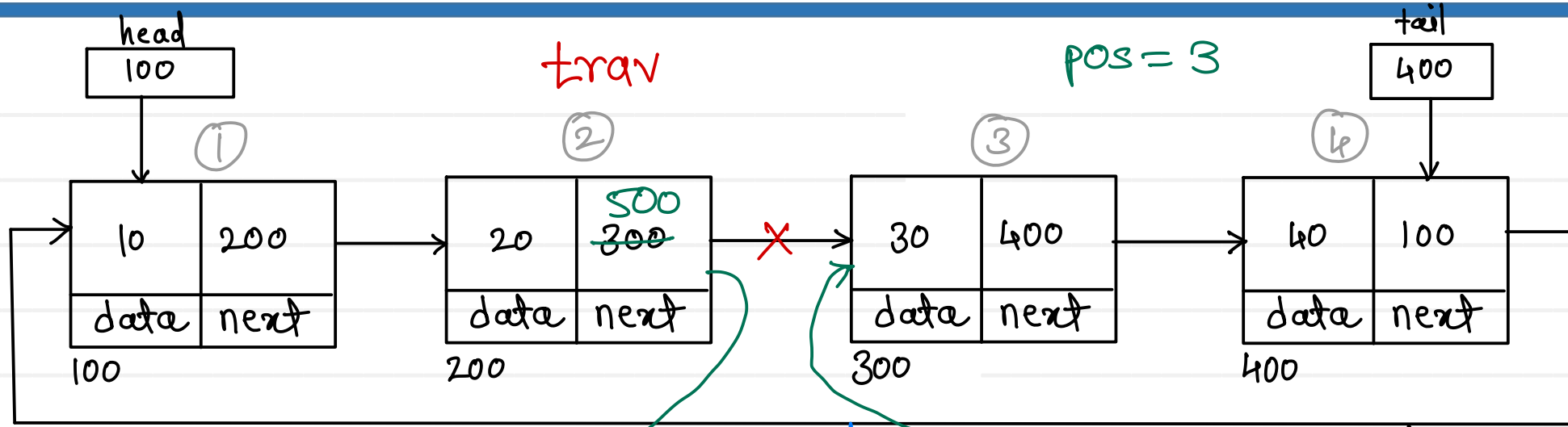
Singly Circular Linked List - Add last



$$T(n) = O(1)$$

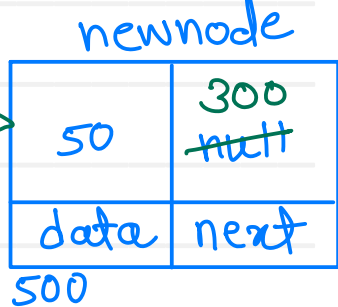
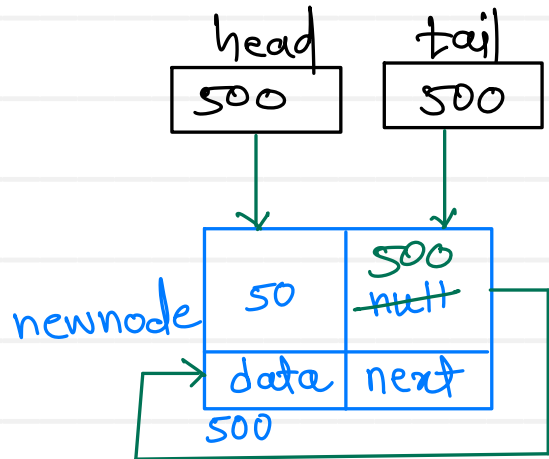
1. create node
2. if list is empty
 - a. add newnode into head & tail
 - b. make list circular.
3. if list is not empty.
 - a. add first node into next of newnode
 - b. add newnode into next of last node
 - c. move tail on last node.

Singly Circular Linked List - Add position



valid positions:
1 to size+1
Invalid positions:
 $pos < 1$ or $pos > size+1$

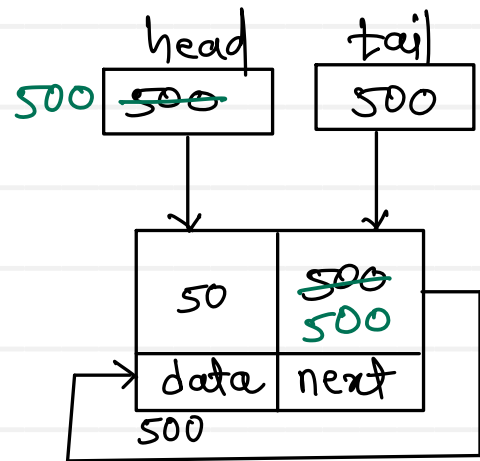
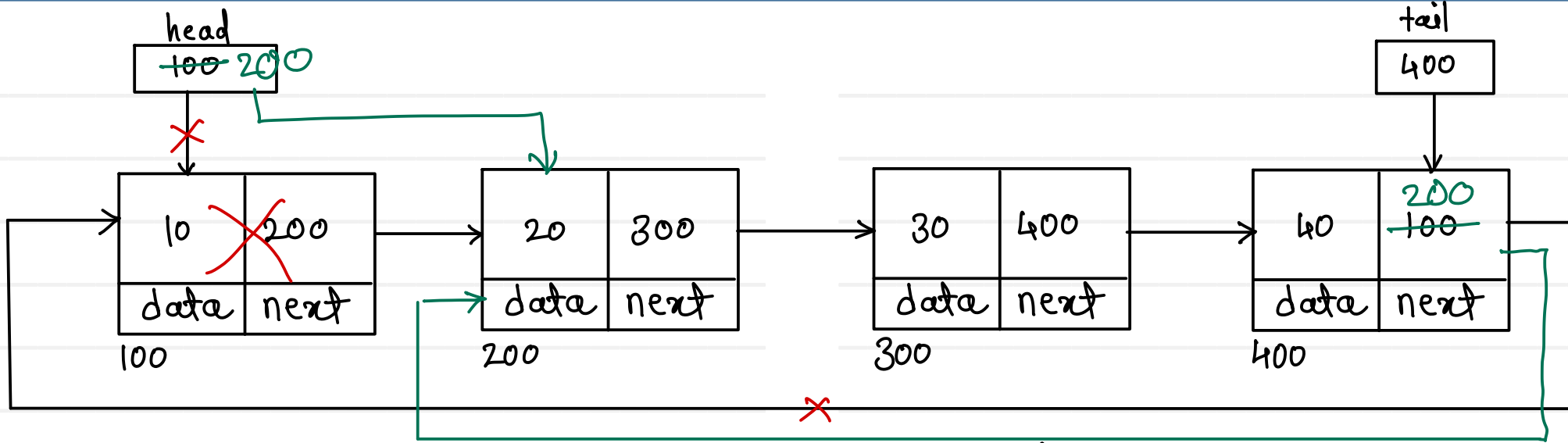
make before break



$$T(n) = O(n)$$

1. create node
2. if list is empty
 - a. add newnode into head & tail
 - b. make list circular
3. if list is not empty
 - a. traverse till $pos-1$ node
 - b. add pos node into next of new
 - c. add newnode into next of $pos-1$ node

Singly Circular Linked List - Delete first

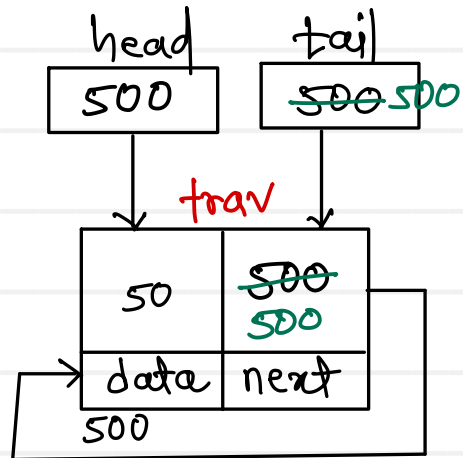
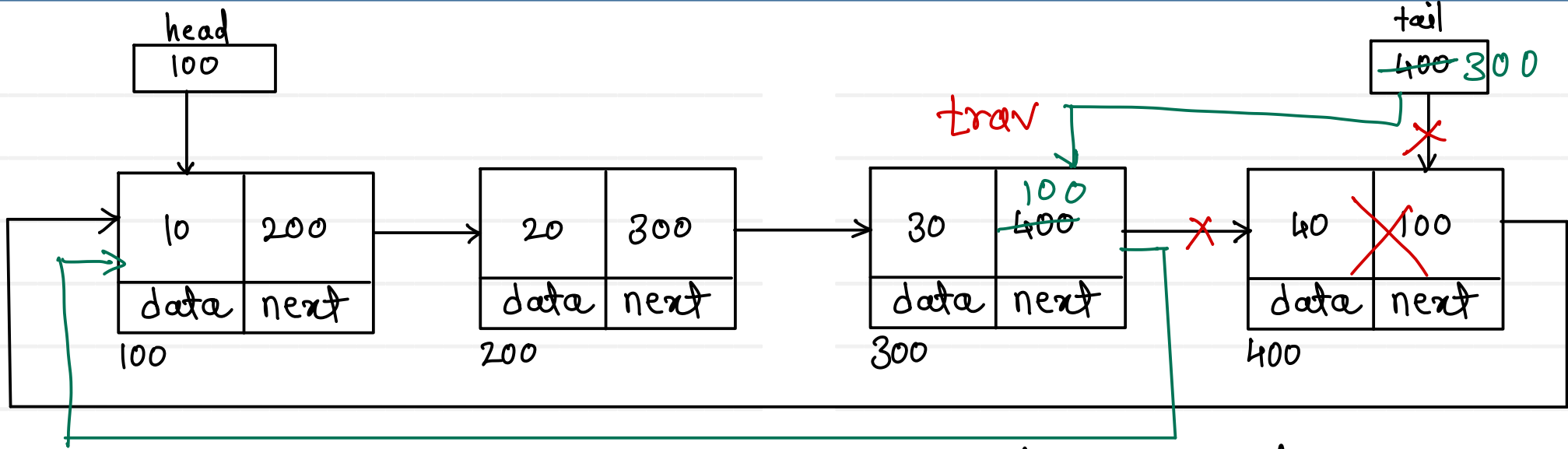


tail.next = head.next;
head = head.next

1. if list is empty
return;
2. if list has single node
head = tail = null;
3. if list has multiple nodes
 - a. add second node into next of last node
 - b. move head on second node

$$T(n) = O(1)$$

Singly Circular Linked List - Delete last

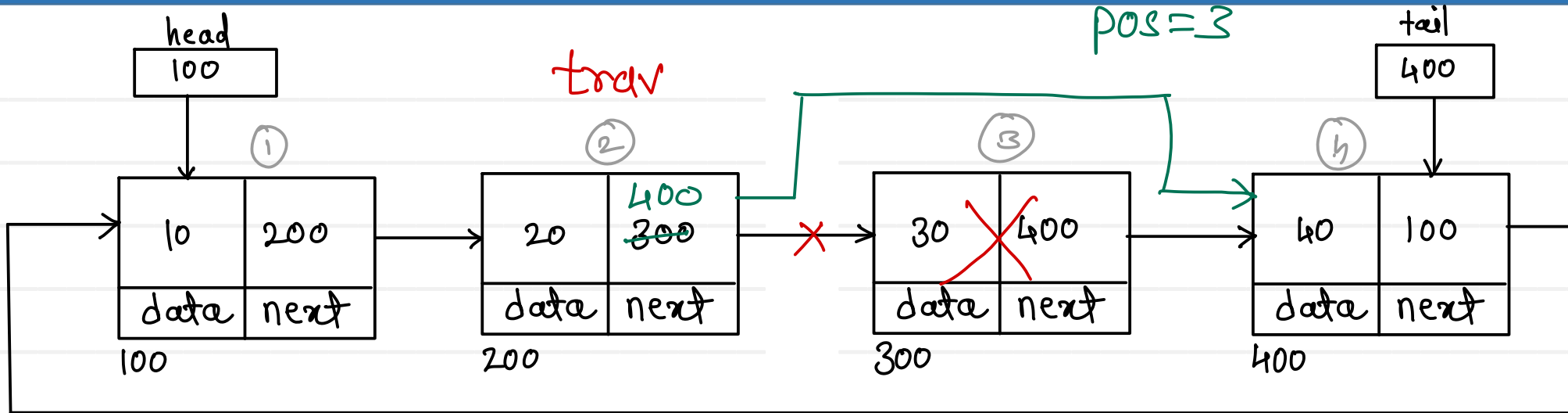


Node trav = head
 while (trav.next.next != head)
 trav = trav.next;
 trav.next = head;
 tail = trav.

$$T(n) = O(n)$$

1. if list is empty
 return;
2. if list has single node
 head = tail = null;
3. if list has multiple nodes
 - a. traverse till second last node
 - b. add first node into next of second last node
 - c. move tail on second last node

Singly Circular Linked List - Delete position



valid positions:

1 to size

Invalid positions:

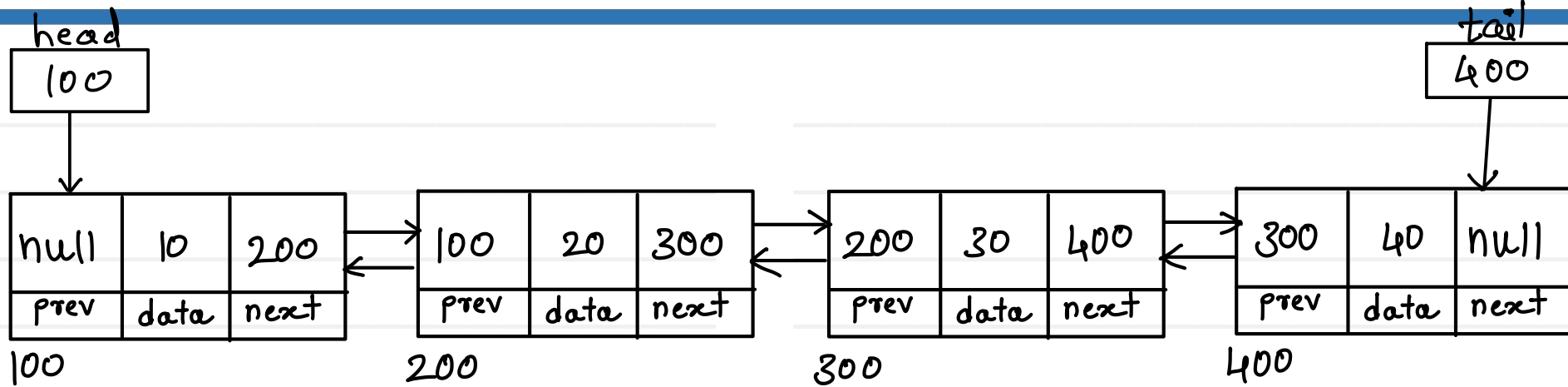
$pos < 1$ or $pos > size$

1. if list is empty
return;
2. if list has single node
head = tail = null;
3. if list has multiple nodes
 - a. traverse till $pos-1$ node
 - b. add $pos+1$ node into next of $pos-1$ node

pos = 3

trav	i	i < pos-1
100	1	T
200	2	X

Doubly Linear Linked List - Display

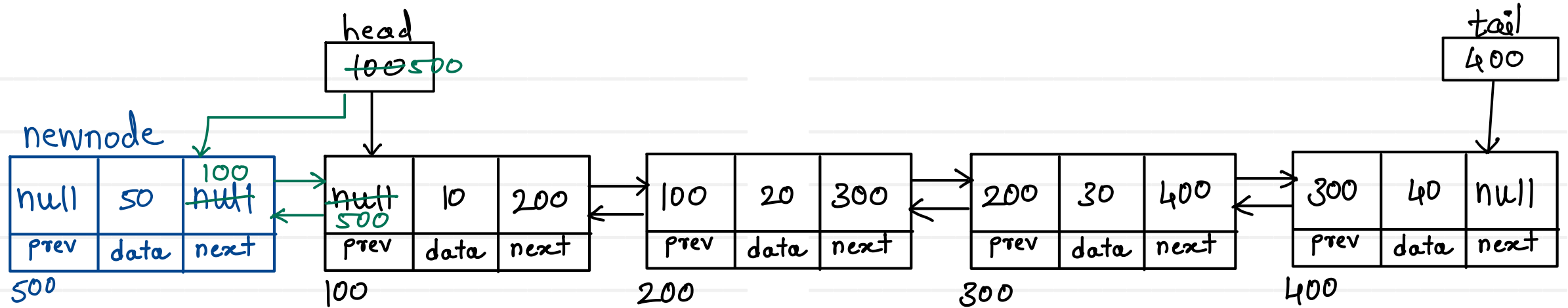


- 1) create trav & start at head
- 2) print current node
- 3) go on next node
- 4) repeat step 2 & 3 till last node

- 1) Create trav & start at tail
- 2) print current node
- 3) go on prev node
- 4) repeat step 2 & 3 till first node

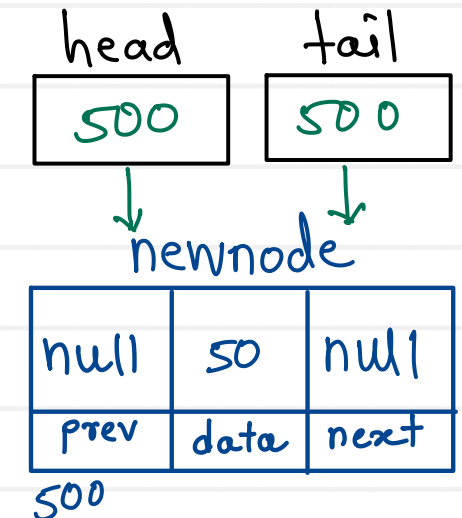
$$T(n) = O(n)$$

Doubly Linear Linked List - Add first

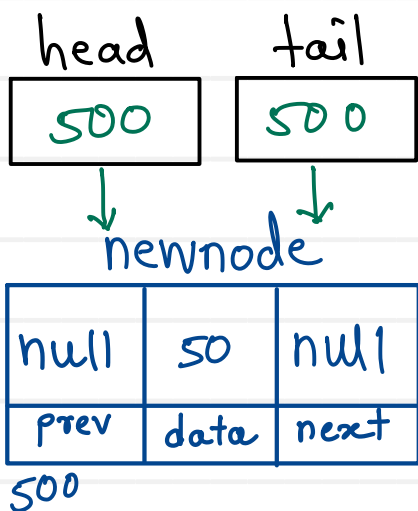
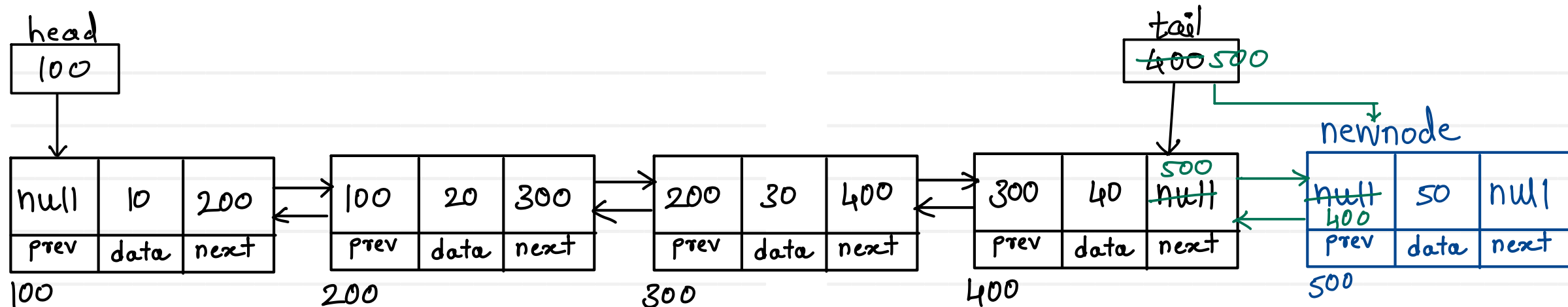


1. create node
2. if list is empty
head = tail = newnode;
3. if list is not empty
 - a. add first node into next of newnode
 - b. add newnode into prev of firstnode
 - c. move head on newnode.

$$T(n) = O(1)$$



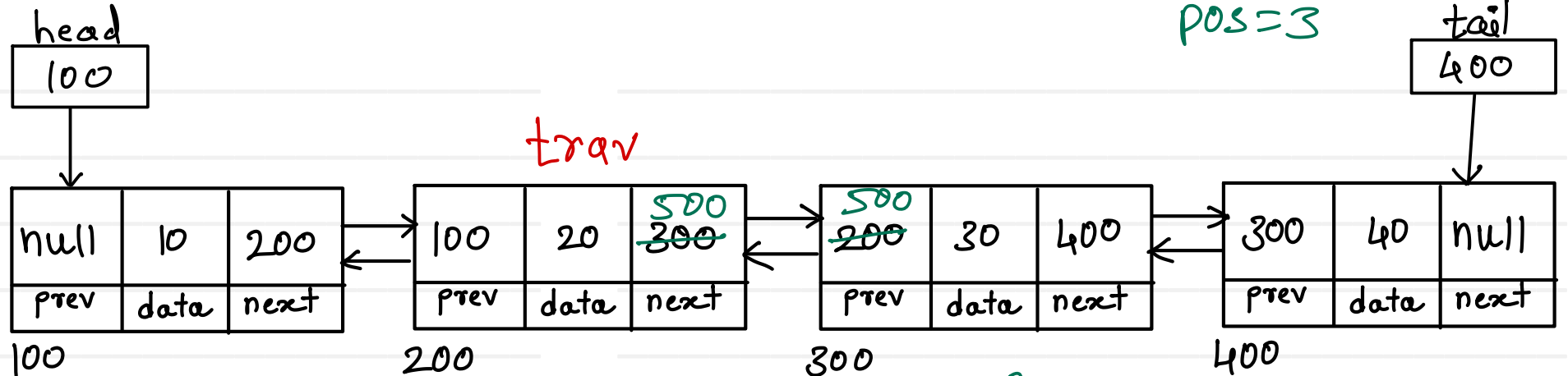
Doubly Linear Linked List - Add last



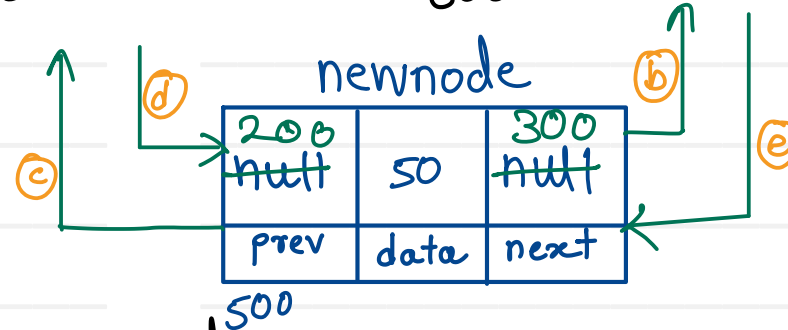
1. Create node
2. if list is empty
head = tail = newnode;
3. if list is not empty
 - a. add last node into prev of newnode
 - b. add newnode into next of last node
 - c. move tail on newnode

$$T(n) = O(1)$$

Doubly Linear Linked List - Add position

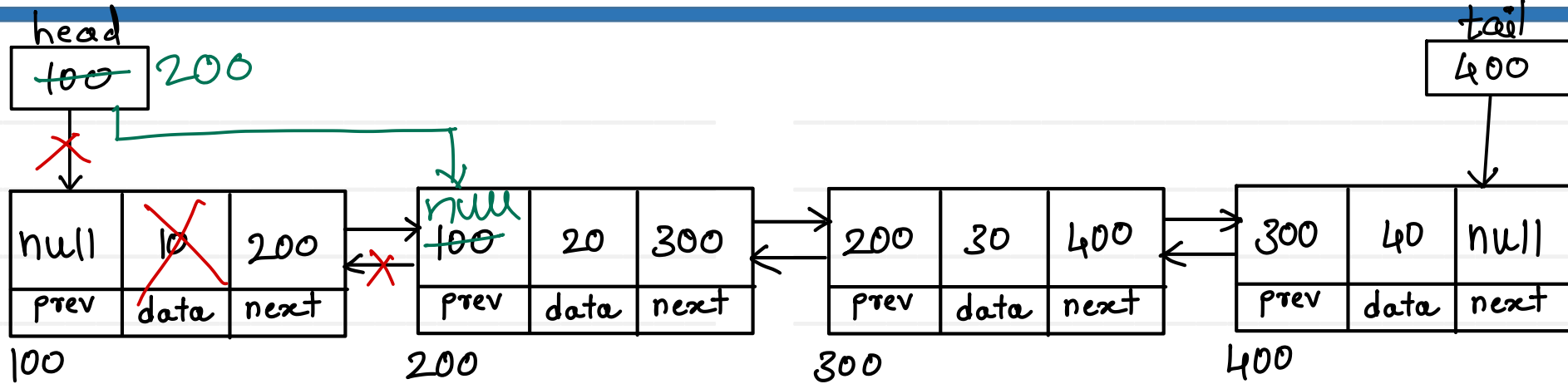


- 1) Create node
- 2) if list is empty
head = tail = newnode;
- 3) if list is not empty
 - a. traverse till pos-1
 - b. add pos node into next of newnode
 - c. add pos-1 node into prev of newnode
 - d. add newnode into next of pos-1 node
 - e. add newnode into prev of pos node.



$$T(n) = O(n)$$

Doubly Linear Linked List - Delete first



head = head.next
head.prev = null

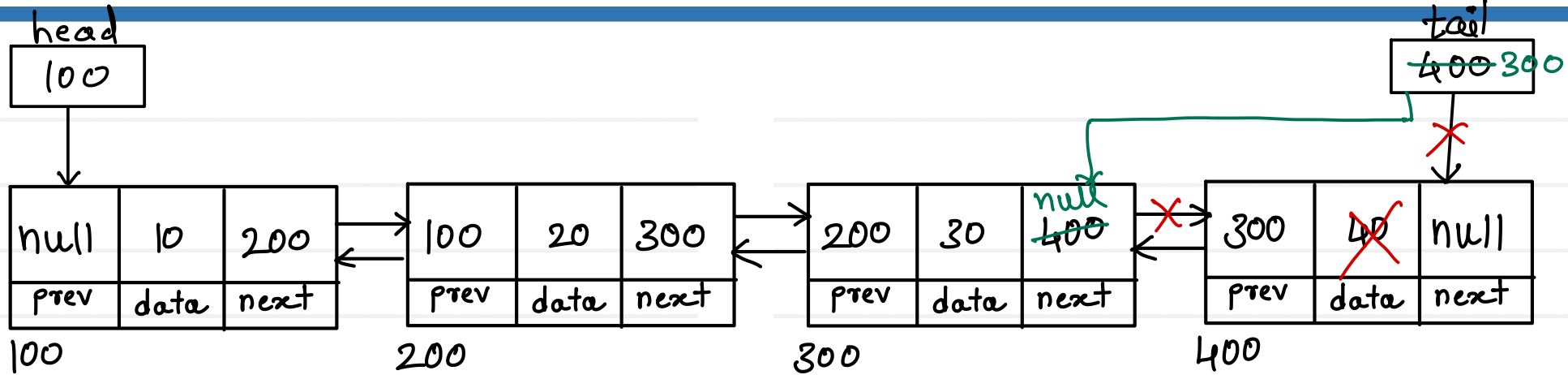
↓

10

$$T(n) = O(n)$$

1. if list is empty
return
2. if list has single node
head = tail = null
3. if list has multiple nodes
 - a. add null into prev of second node
 - b. move head on second node

Doubly Linear Linked List - Delete last



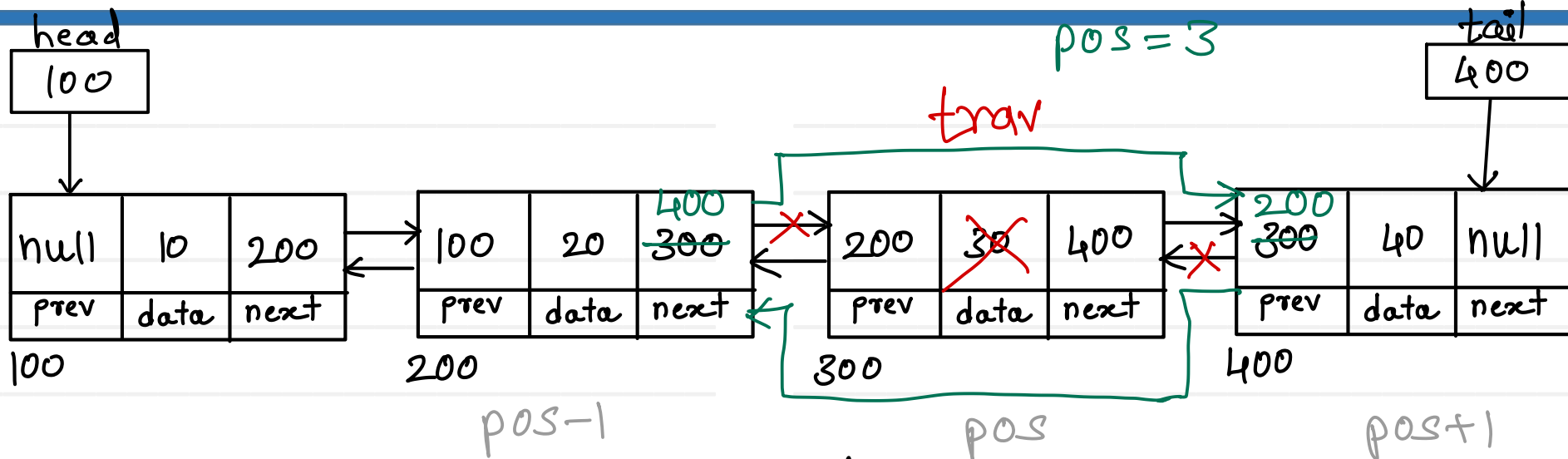
head = head.next
head.prev = null

10

$$T(n) = O(1)$$

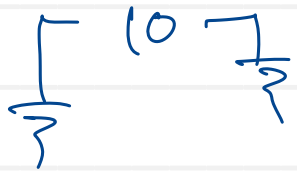
1. if list is empty
return
2. if list has single node
head = tail = null
3. if list has multiple nodes
 - a. add null into next of second last node
 - b. move tail on second last node

Doubly Linear Linked List - Delete position



1. if list is empty
return
2. if list has single node
head = tail = null.
3. if list has multiple nodes
 - a. traverse till pos node
 - b. add pos+1 node into next of pos-1 node
 - c. add pos-1 node into prev of pos+1 node

head = head.next
head.prev = null



$$T(n) = O(n)$$

Linked list - Applications

- linked list is a dynamic data structure because it can grow or shrink at runtime.
- Due to this dynamic nature, linked list is used to implement other data structures like
 - Stack
 - Queue
 - Hash table
 - Graph

Stack

LIFO

1. Add first
delete first

2. Add last
delete last

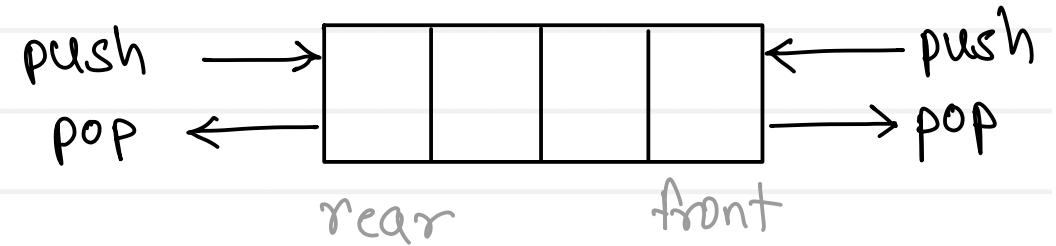
Queue

FIFO

1. Add first
delete last

2. Add last
delete first

Deque (Double Ended Queue)



1. Input restricted deque
- one input end is closed

2. Output restricted deque
- one output end is closed

Array

- Array space inside memory is continuous
- Array can not grow or shrink at runtime
- Random access of elements is allowed
- Insert or delete, needs shifting of array elements
- Array needs less space

Linked list

- Linked list space inside memory is not continuous
- Linked list can grow or shrink at runtime
- Random access of elements is not allowed
- Insert or delete, need not shifting of array elements
- Linked list needs more space



Thank you!!!

Devendra Dhande

devendra.dhande@sunbeaminfo.com



Sunbeam Institute of Information Technology

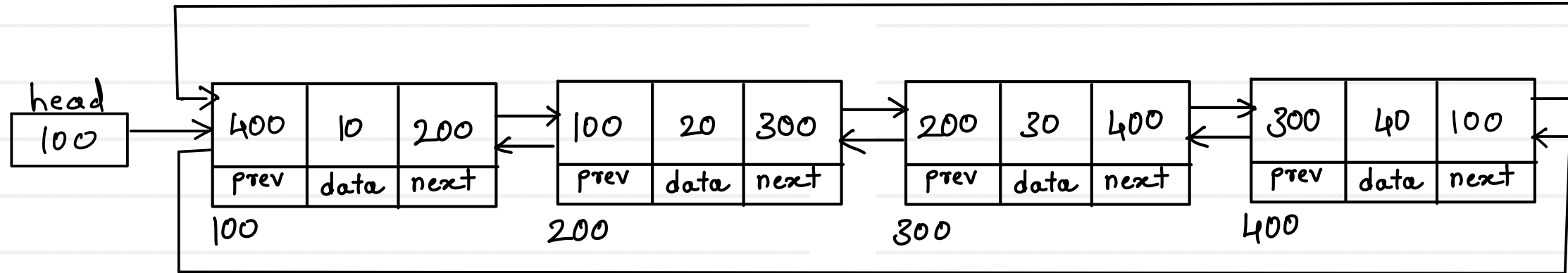
Pune and Karad

Module – Data Structures and Algorithms

Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

Doubly Circular Linked List - Display

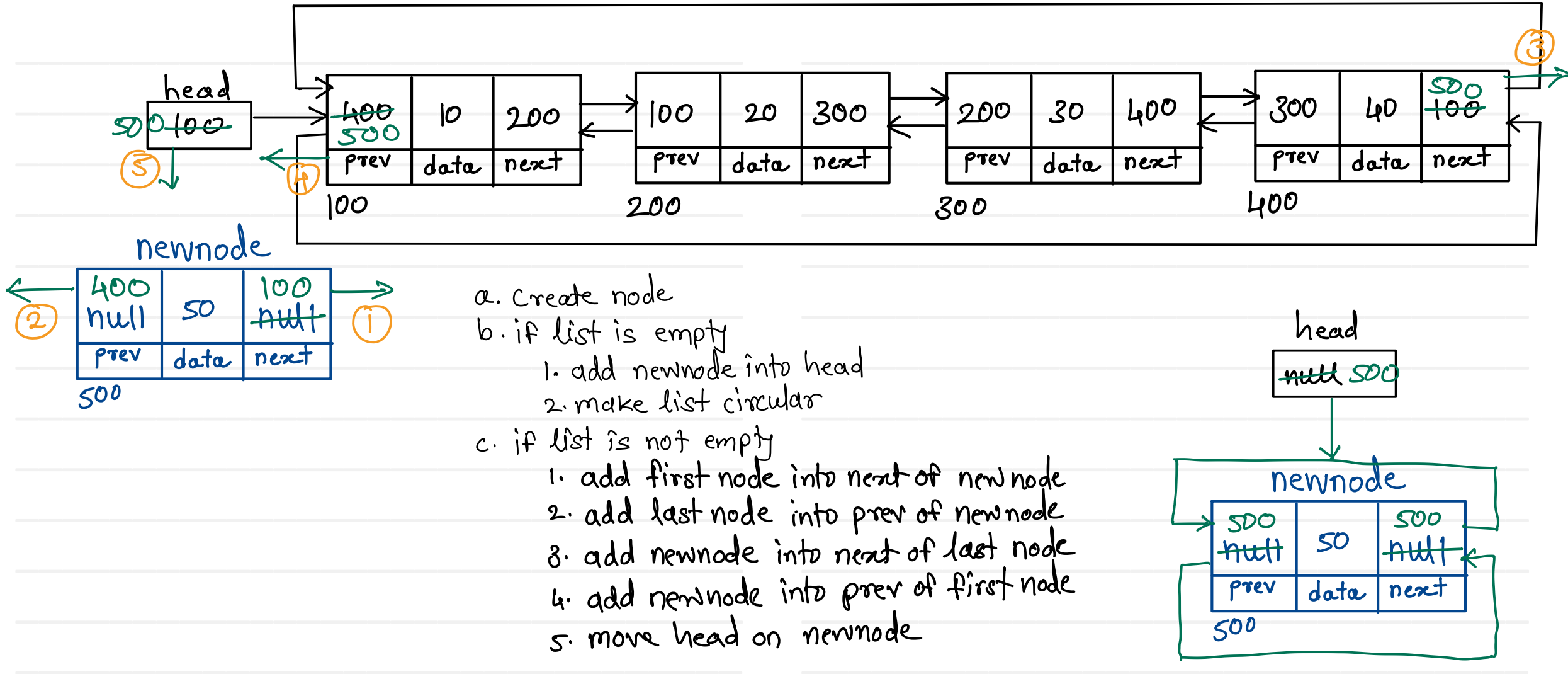


1. Create trav & start at first node
2. print current node data
3. go on next node
4. repeat step 2 & 3 till last node

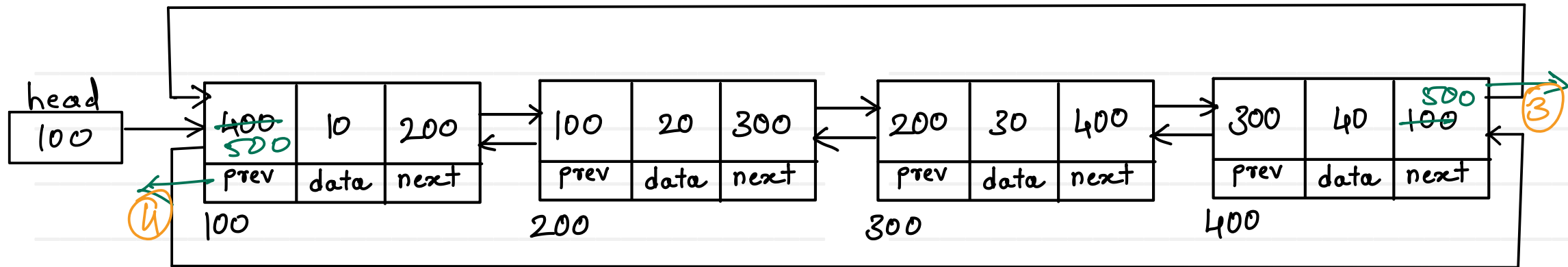
1. Create trav & start at last node
2. print current node
3. go on prev node
4. repeat step 2 & 3 till first node

$$T(n) = O(n)$$

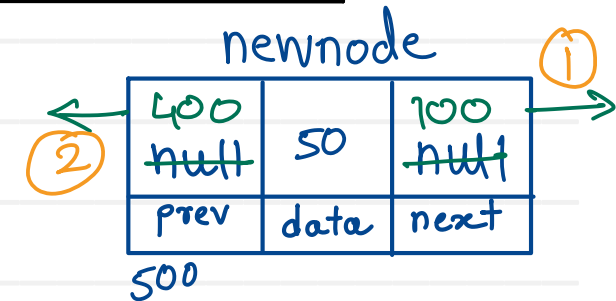
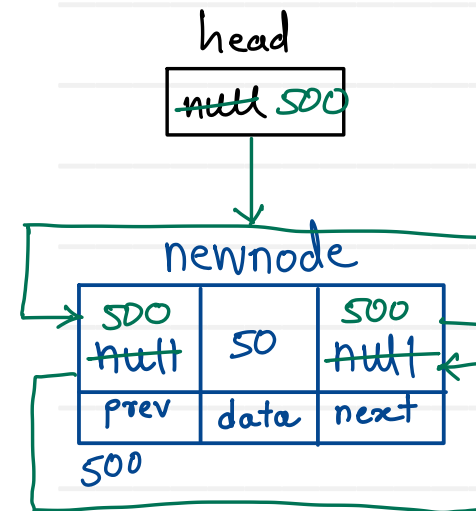
Doubly Circular Linked List - Add first



Doubly Circular Linked List - Add last

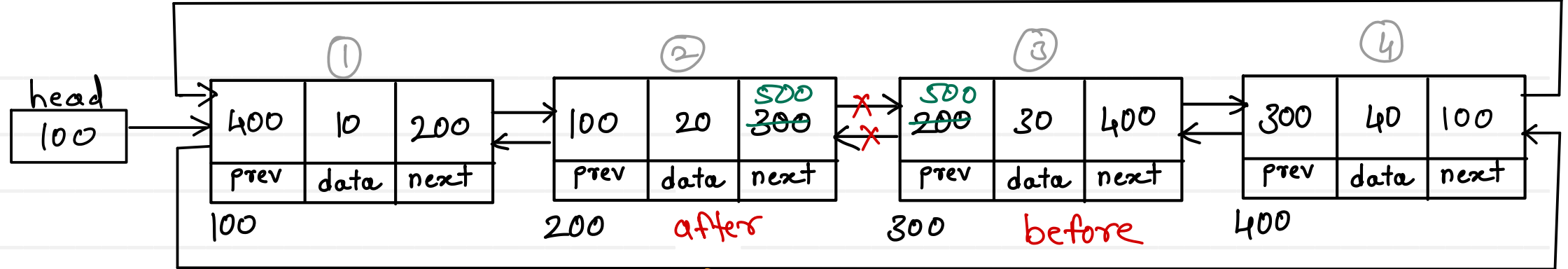


- Create node
- if list is empty
 - add newnode into head
 - make list circular
- if list is not empty
 - add first node into next of new node
 - add last node into prev of new node
 - add newnode into next of last node
 - add newnode into prev of first node

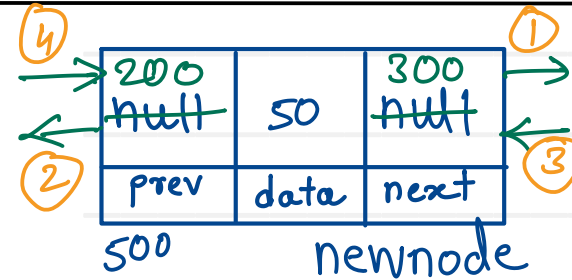


$$T(n) = O(1)$$

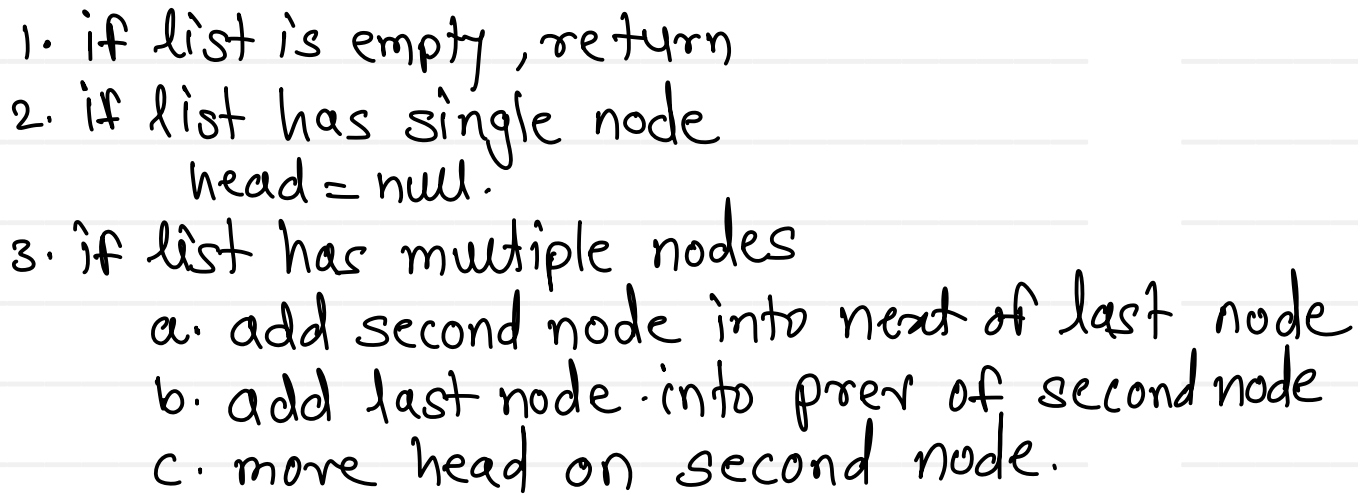
Doubly Circular Linked List - Add position



$\text{newnode.next} = \text{before};$
 $\text{newnode.prev} = \text{after};$
 $\text{before.prev} = \text{newnode};$
 $\text{after.next} = \text{newnode};$

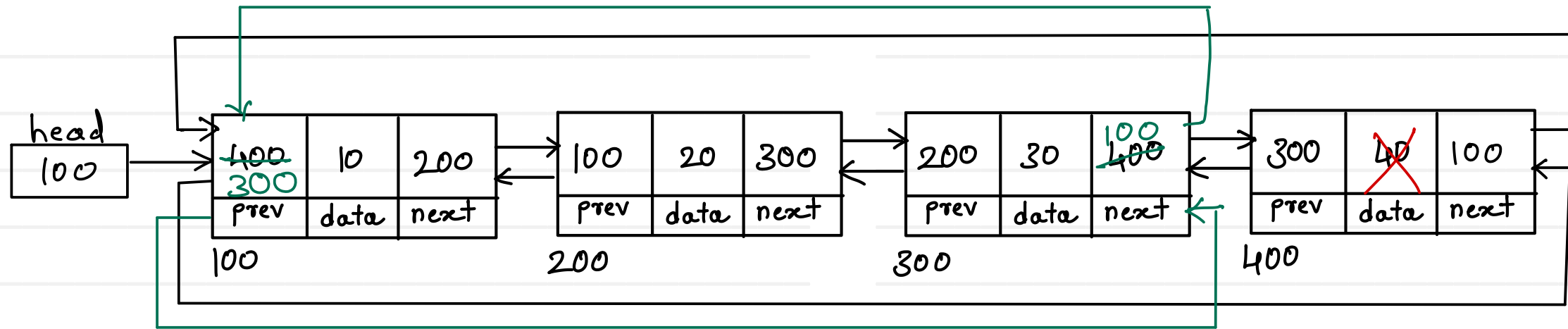


$$T(n) = O(n)$$





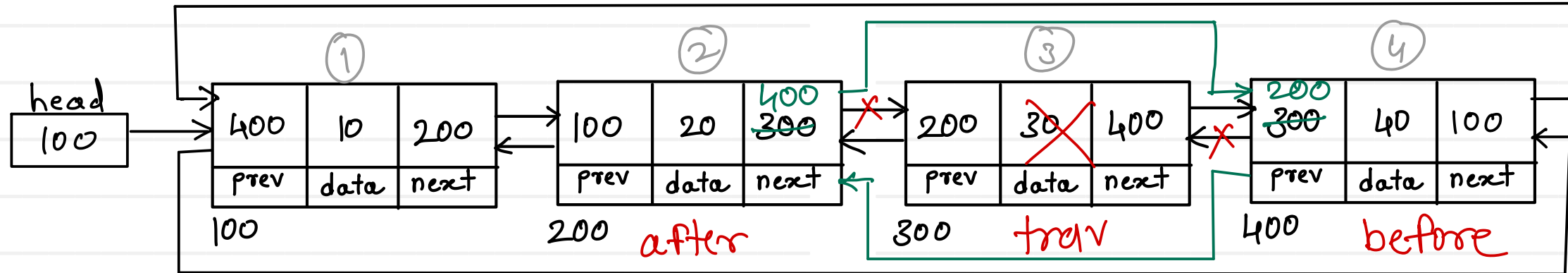
Doubly Circular Linked List - Delete last



1. if list is empty, return
2. if list has single node
head = null;
3. if list has multiple nodes
 - a. add first node into next of second last node
 - b. add second last node into prev of first node

$$T(n) = O(1)$$

Doubly Circular Linked List - Delete position



$after = trav.prev;$
 $before = trav.next;$
 $after.next = before;$
 $before.prev = after;$

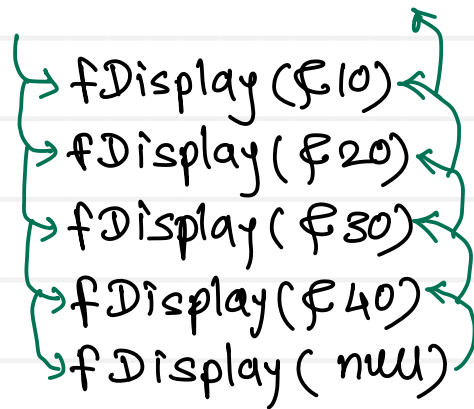
$$T(n) = O(n)$$

Singly linear linked list - Display reverse

head
↓
10 → 20 → 30 → 40 → ⊥

Tail Recursion

```
void fDisplay(Node trav) {  
    if (trav == null)  
        return;  
    sysout(trav.data);  
    fDisplay(trav.next);  
}
```

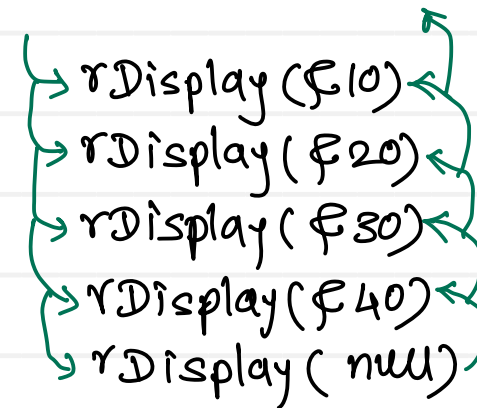


10 20 30 40

head
↓
10 → 20 → 30 → 40 → ⊥

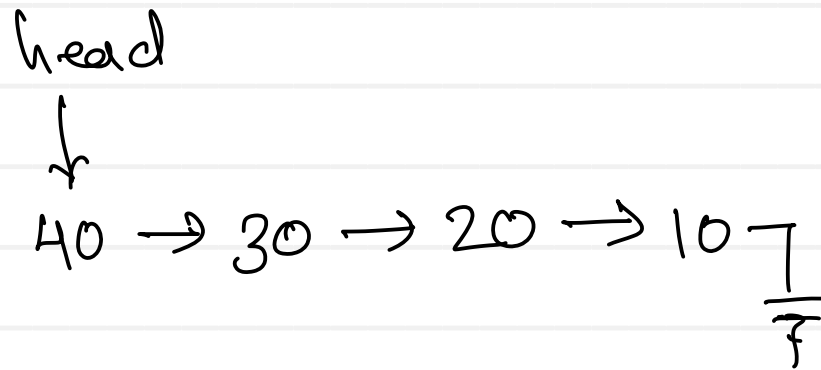
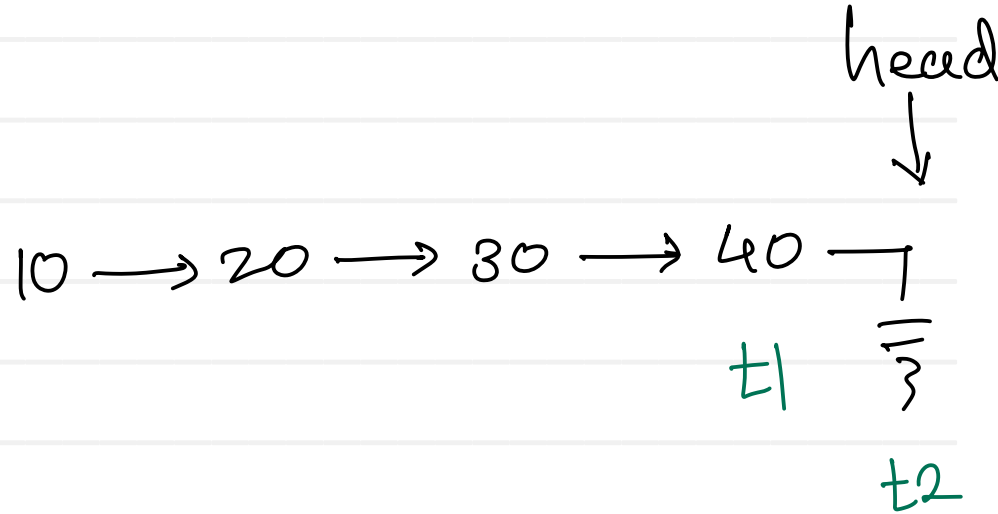
Non tail Recursion

```
void rDisplay(Node trav) {  
    if (trav == null)  
        return;  
    rDisplay(trav.next);  
    sysout(trav.data);  
}
```



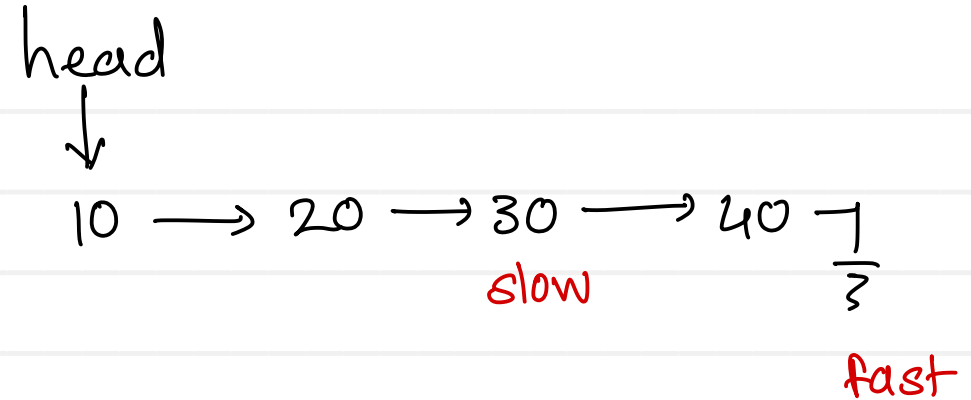
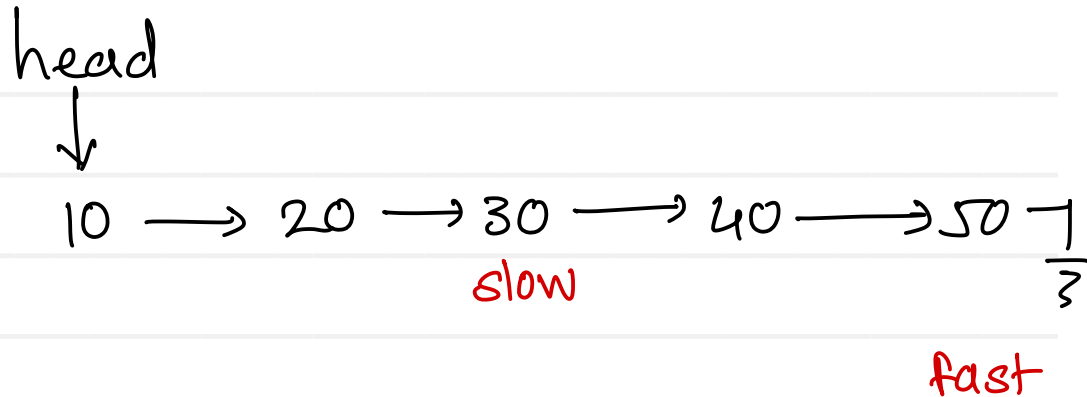
40 30 20 10

Singly linear linked list - Reverse



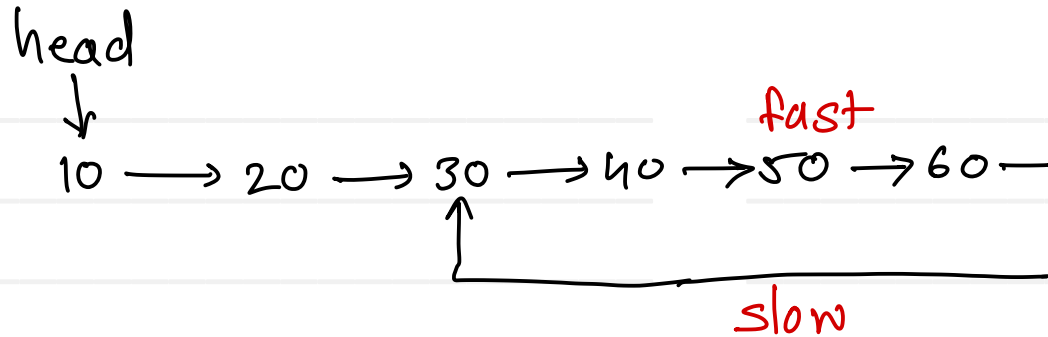
```
Node t1 = null;  
Node t2 = head;  
while (t2 != null) {  
    head = t2.next;  
    t2.next = t1;  
    t1 = t2;  
    t2 = head;  
}  
head = t1;
```


Singly linear linked list - Find mid



```
void findMid( ) {
    Node fast = head, slow = head;
    while ( fast != null && fast.next != null ) {
        fast = fast.next.next;
        slow = slow.next;
    }
    sysout( slow.data );
}
```

Singly linear linked list - Detect loop



```
boolean hasLoop( ) {
```

```
    Node fast = head, slow = head;
```

```
    while ( fast != null && fast.next != null ) {
```

```
        fast = fast.next.next;
```

```
        slow = slow.next;
```

```
        if ( fast == slow )
```

```
            return true;
```

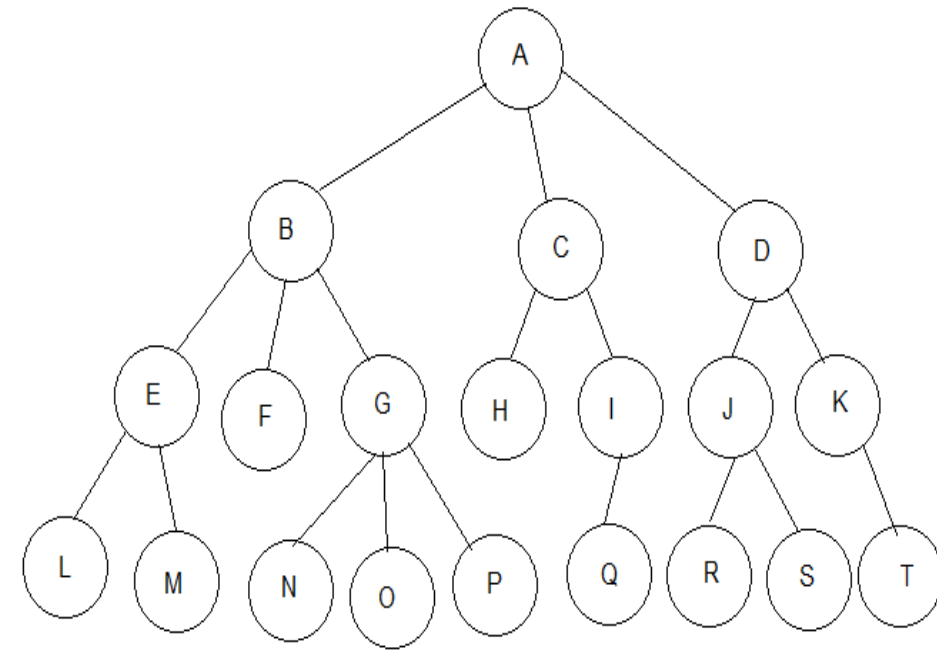
```
    }
```

```
    return false;
```

```
}
```

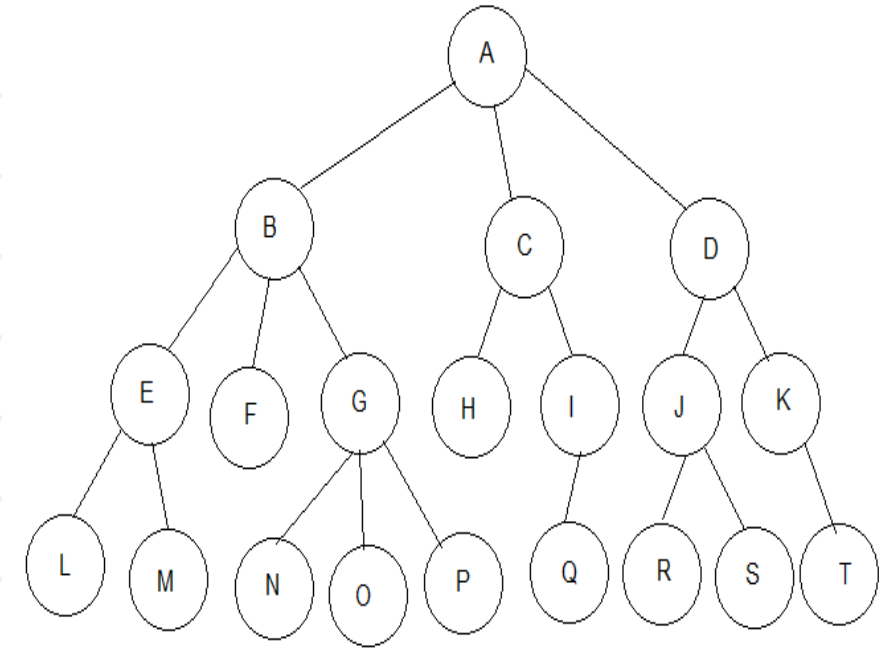
Tree - Terminologies

- **Tree** is a **non linear** data structure which is a finite set of nodes with one specially designated node is called as “**root**” and remaining nodes are partitioned into m disjoint subsets where each of subset is a tree..
- **Root** is a **starting point** of the tree.
- All nodes are connected in **Hierarchical manner (multiple levels)**.
- **Parent node:-** having other child nodes connected
- **Child node:-** immediate descendant of a node
- **Leaf node:-**
 - Terminal node of the tree.
 - Leaf node does not have child nodes.
- **Ancestors:-** all nodes in the path from root to that node.
- **Descendants:-** all nodes accessible from the given node
- **Siblings:-** child nodes of the same parent



Tree - Terminologies

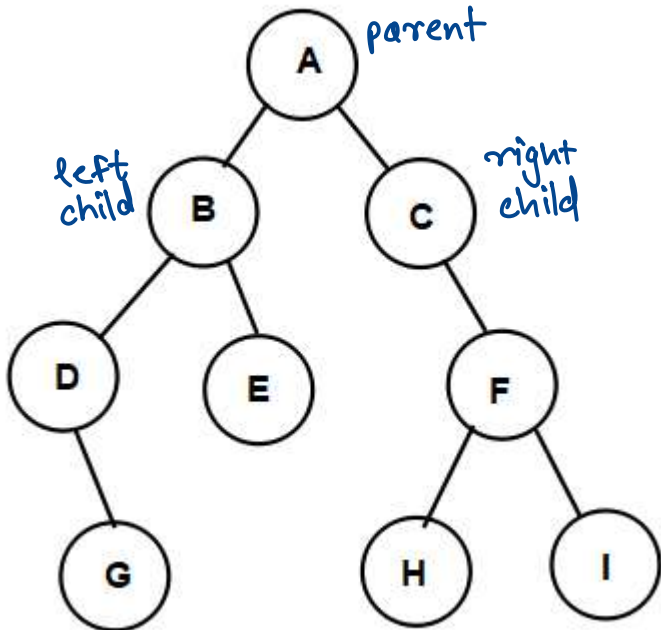
- **Degree of a node** :- number of child nodes for any given node.
- **Degree of a tree** :- Maximum degree of any node in tree.
- **Level of a node** :- indicates position of the node in tree hierarchy
 - Level of child = Level of parent + 1
 - Level of root = 0
- **Height of node** :- number of links from node to longest leaf.
- **Depth of node** :- number of links from root to that node
- **Height of a tree** :- Maximum height of a node
- **Depth of a tree** :- Maximum depth of a node
- Tree with zero nodes (ie empty tree) is called as “**Null tree**”. Height of Null tree is -1.
 - Tree can grow up to any level and any node can have any number of Childs.
 - That's why operations on tree becomes un efficient.
 - Restrictions can be applied on it to achieve efficiency and hence there are different types of trees.



Tree - Terminologies

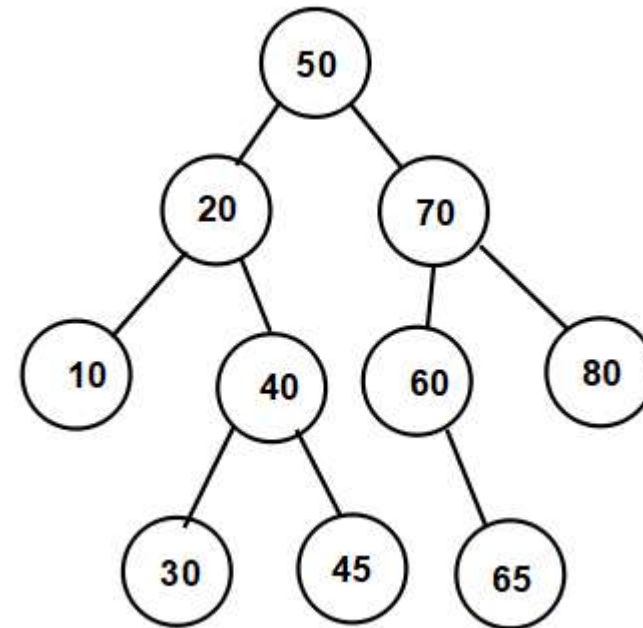
- **Binary Tree**

- Tree in which each node has maximum two child nodes
- Binary tree has degree 2. Hence it is also called as 2- tree



- **Binary Search Tree**

- Binary tree in which left child node is always smaller and right child node is always greater or equal to the parent node.
- Searching is faster
- Time complexity : $O(h)$ h – height of tree



Binary Search Tree - Implementation

Node :

data →

left → reference

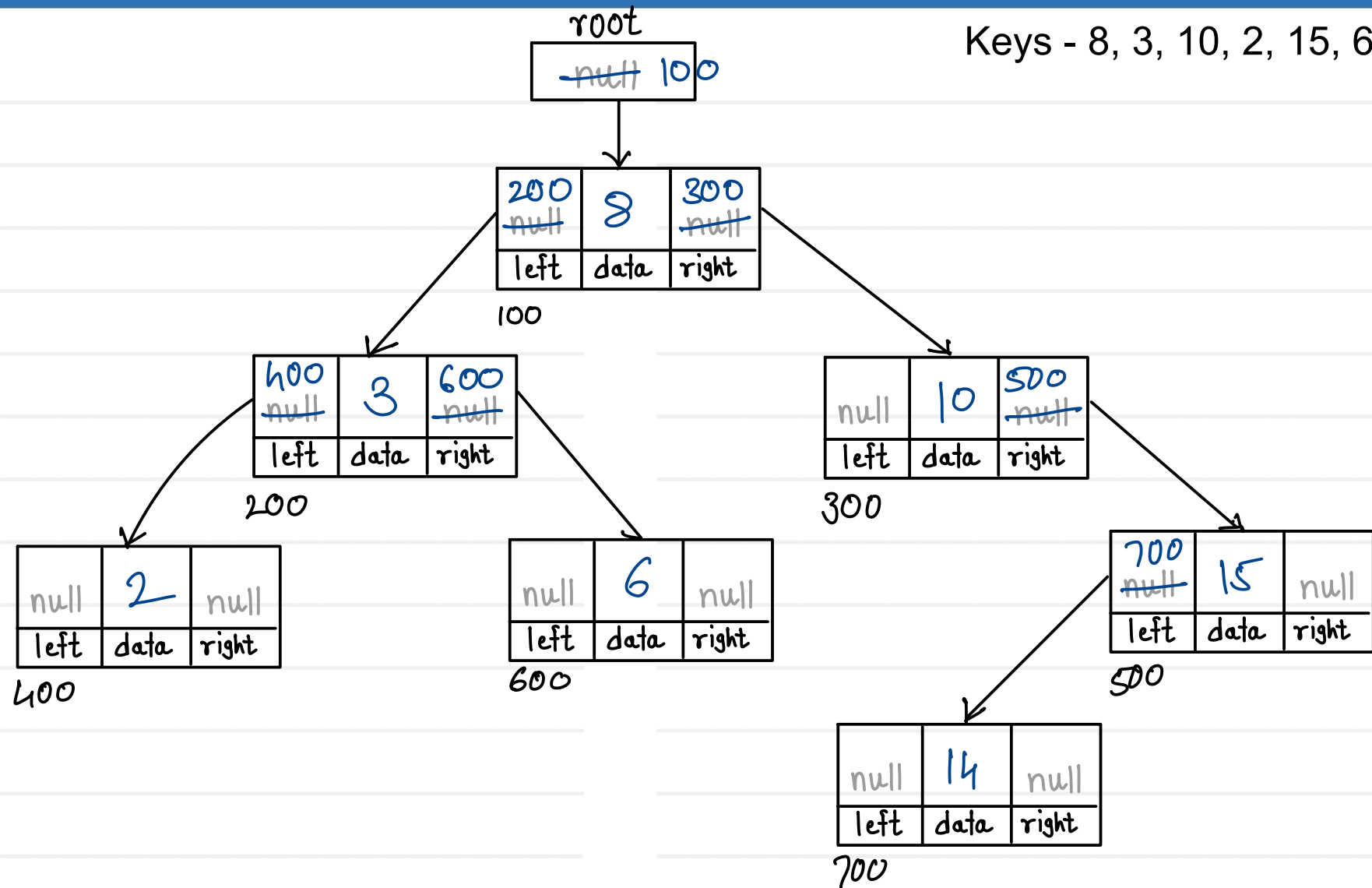
right → reference

```
class Node {  
    int data;  
    Node left;  
    Node right;  
}
```

```
class BST {  
    static class Node {  
        int data;  
        Node left;  
        Node right;  
    }  
    Node root;  
    public BST() { ... }  
    public add() { ... }  
    public delete() { ... }  
    public search() { ... }  
    public traverse() { ... }  
}
```

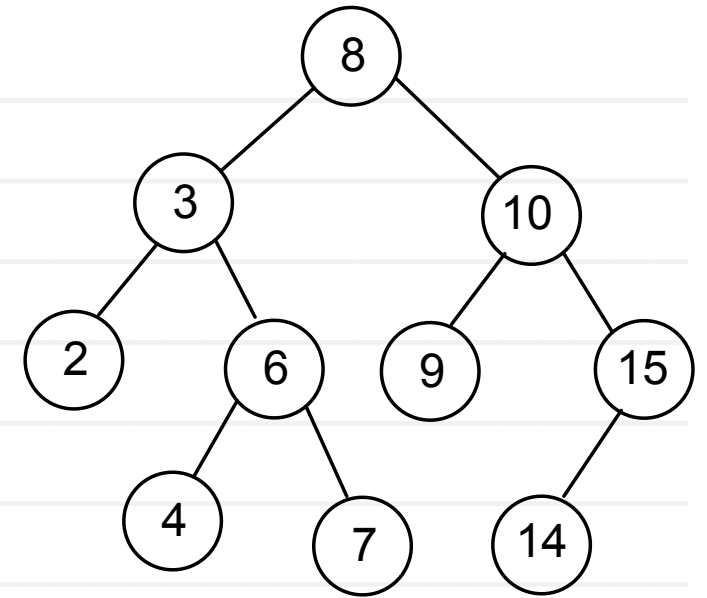
Binary Search Tree - Add Node

Keys - 8, 3, 10, 2, 15, 6, 14, 4, 7, 9

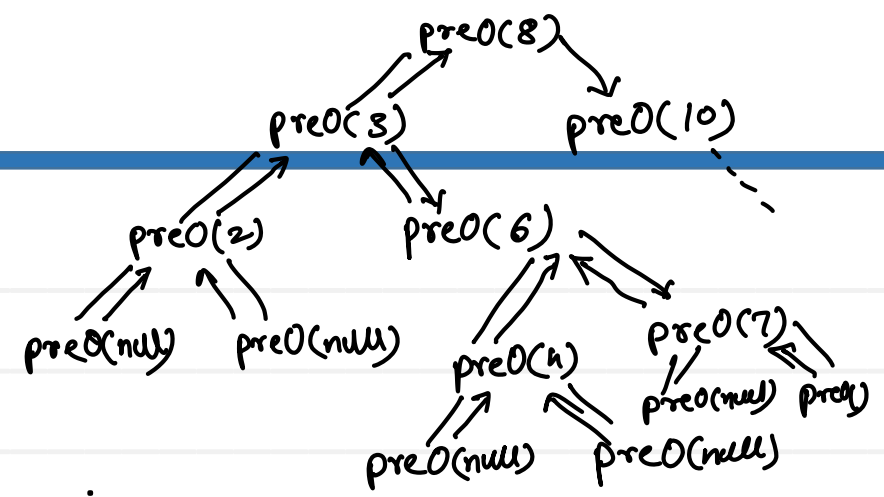


Binary Search Tree – Add Node

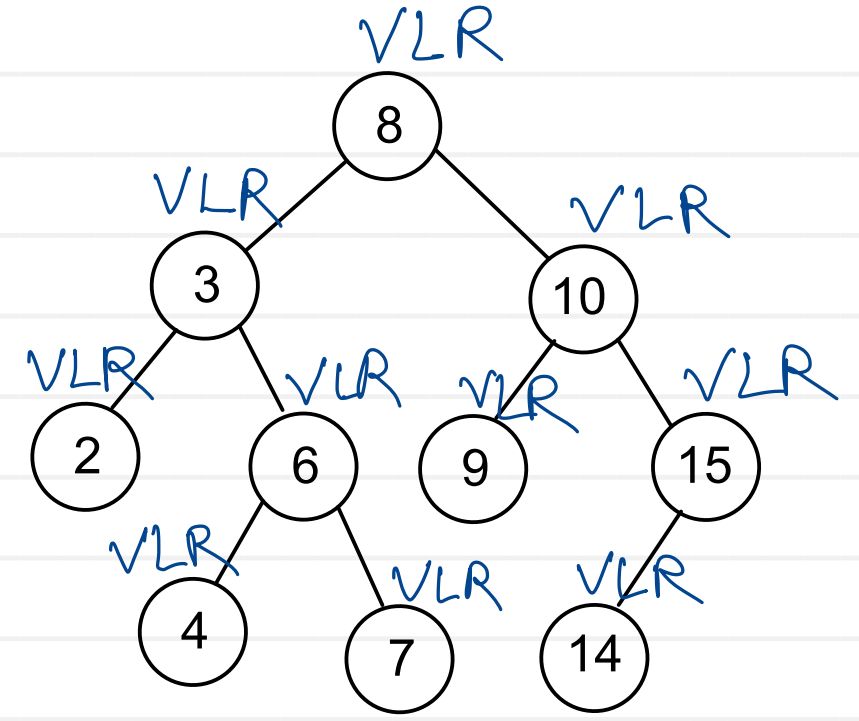
```
//1. create node for given value
//2. if BSTree is empty
    // add newnode into root itself
//3. if BSTree is not empty
    //3.1 create trav reference and start at root node
    //3.2 if value is less than current node data (trav.data)
        //3.2.1 if left of current node is empty
            // add newnode into left of current node
        //3.2.2 if left of current node is not empty
            // go into left of current node
    //3.3 if value is greater or equal than current node data (trav.data)
        //3.3.1 if right of current node is empty
            // add newnode into right of current node
        //3.3.2 if right of current node is not empty
            // go into right of current node
    //3.4 repeat step 3.2 and 3.3 till node is not getting added into BSTree
```



- **Pre-Order:- V L R**
- **In-order:- L V R**
- **Post-Order:- L R V**
- The traversal algorithms can be implemented easily using recursion.
- Non-recursive algorithms for implementing traversal needs stack to store node pointers.

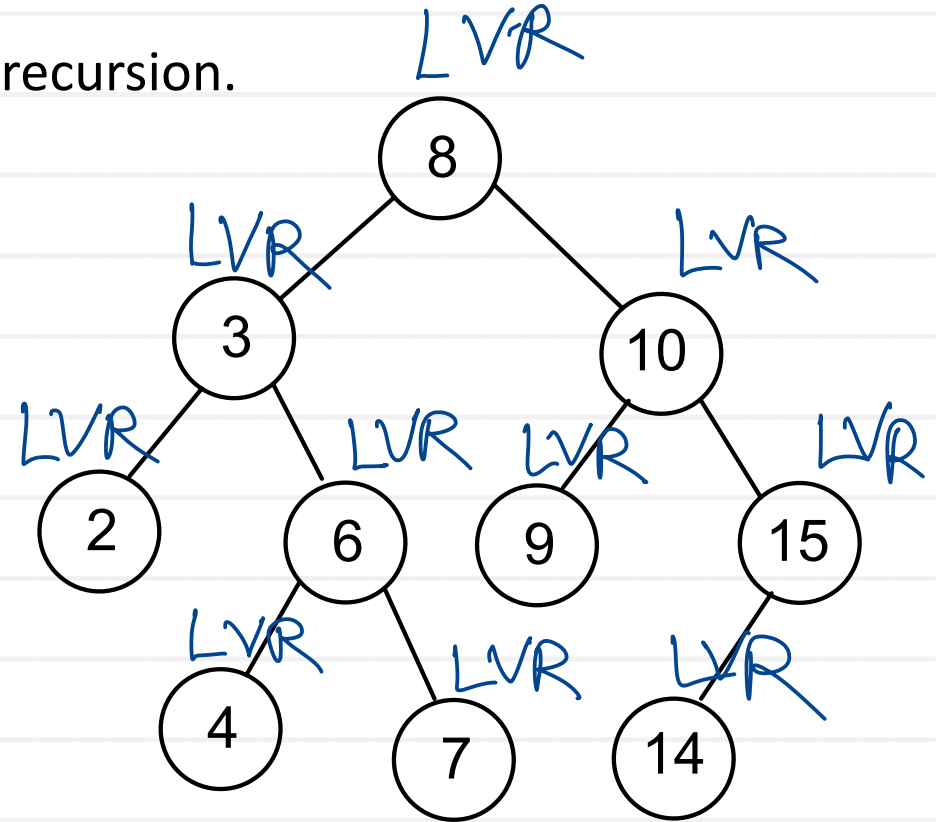


• **Pre-Order :-** 8, 3, 2, 6, 4, 7, 10, 9, 15, 14



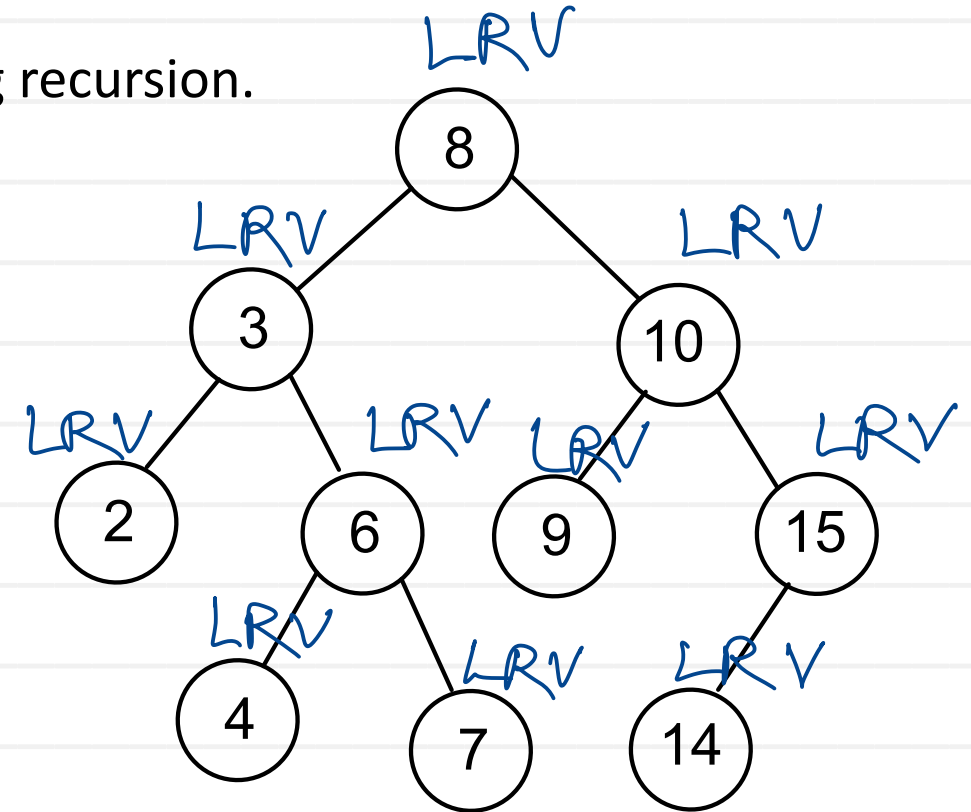
- **Pre-Order:-** V L R
- **In-order:-** L V R
- **Post-Order:-** L R V
- The traversal algorithms can be implemented easily using recursion.
- Non-recursive algorithms for implementing traversal needs stack to store node pointers.

• **In-Order :-** 2, 3, 4, 6, 7, 8, 9, 10, 14, 15

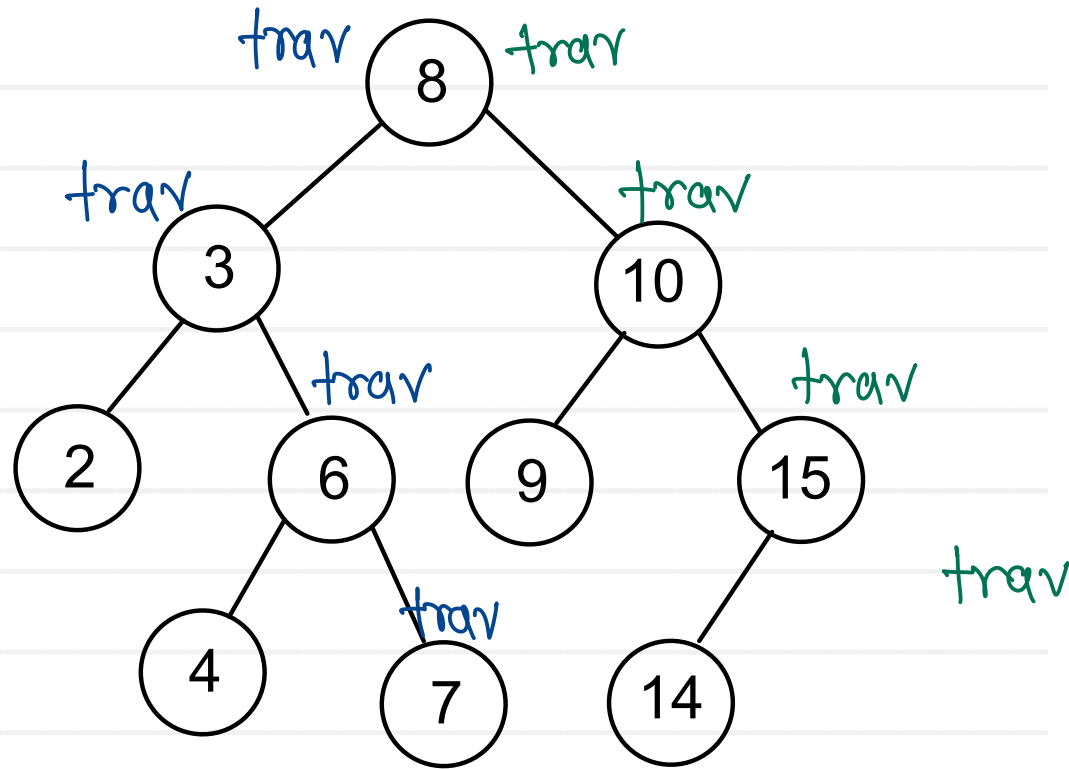


- **Pre-Order:-** V L R
- **In-order:-** L V R
- **Post-Order:-** L R V
- The traversal algorithms can be implemented easily using recursion.
- Non-recursive algorithms for implementing traversal needs stack to store node pointers.

• **Post-Order :-** 2, 4, 7, 6, 3, 9, 14, 15, 10, 8



Binary Search Tree - Binary Search



1. Start from root
2. If key is equal to current node data return current node
3. If key is less than current node data search key into left sub tree of current node
4. If key is greater than current node data search key into right sub tree of current node
5. Repeat step 2 to 4 till leaf node

Key = 7 — Key is found
Key = 16 — Key is not found



Thank you!!!

Devendra Dhande

devendra.dhande@sunbeaminfo.com



Sunbeam Institute of Information Technology

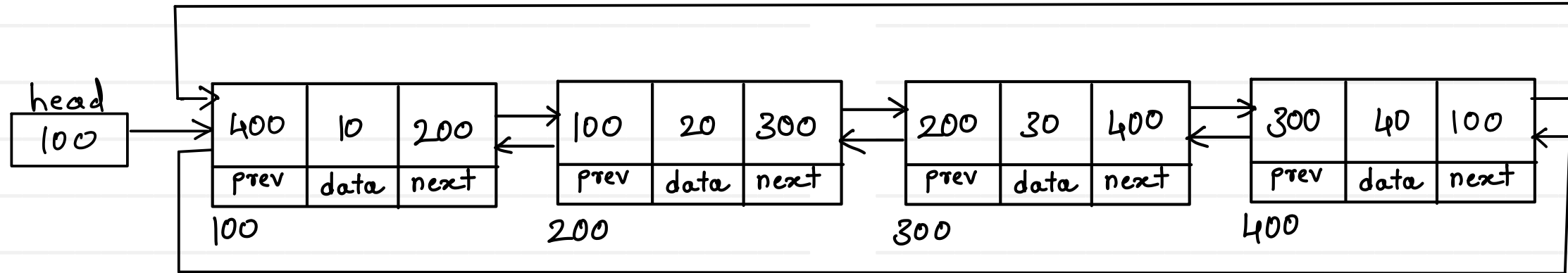
Pune and Karad

Module – Data Structures and Algorithms

Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

Doubly Circular Linked List - Display

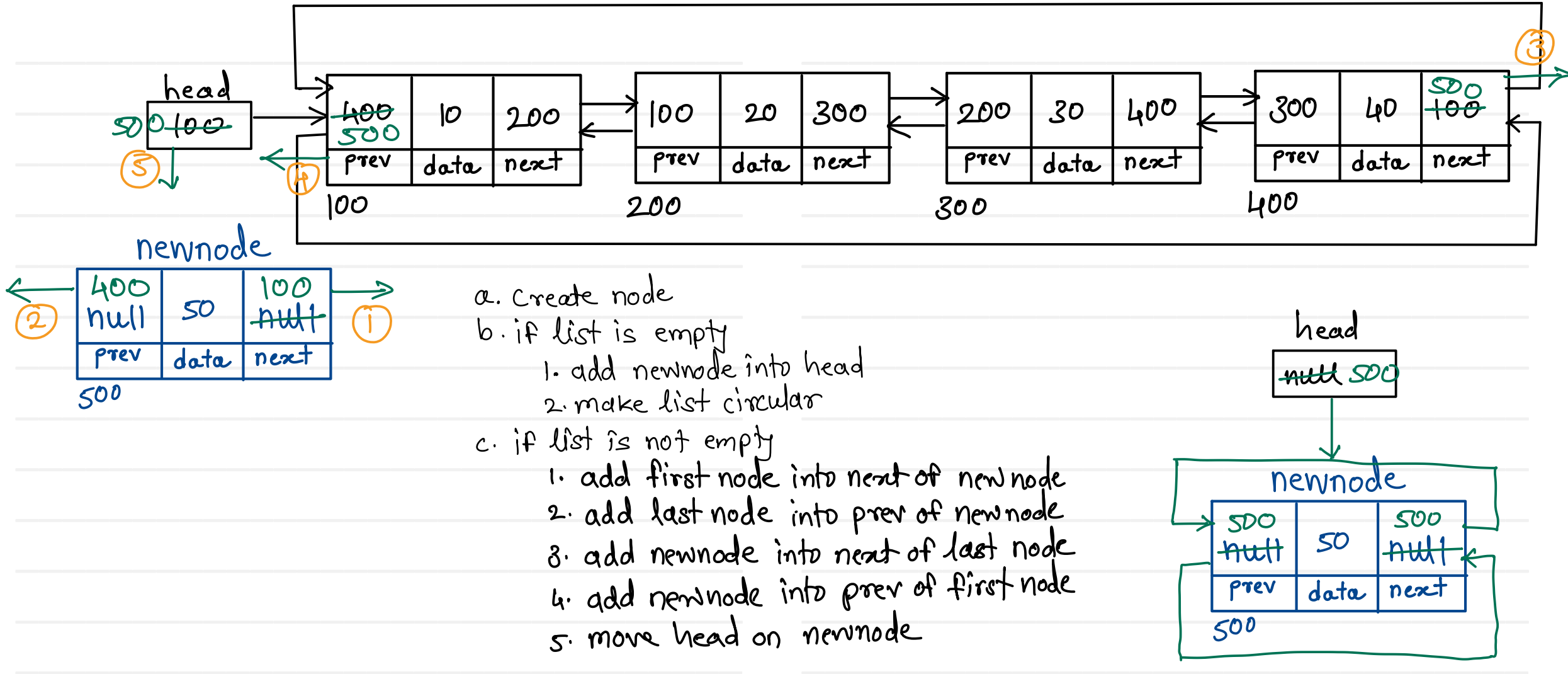


1. Create trav & start at first node
2. print current node data
3. go on next node
4. repeat step 2 & 3 till last node

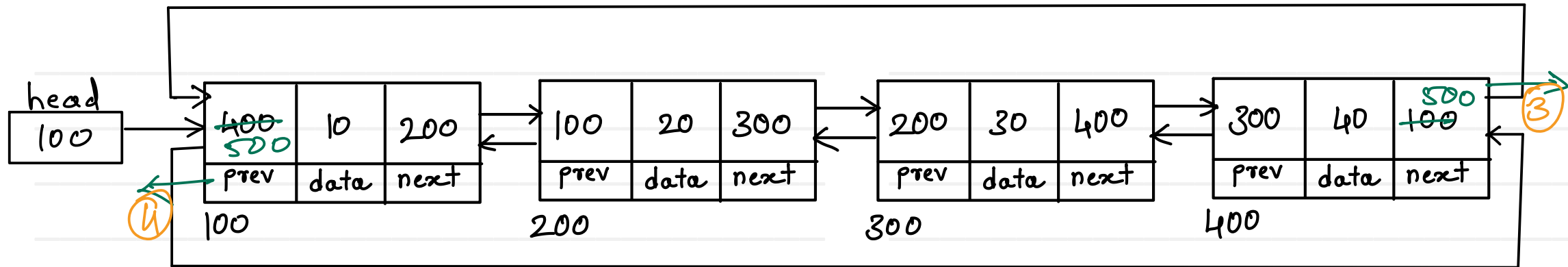
1. Create trav & start at last node
2. print current node
3. go on prev node
4. repeat step 2 & 3 till first node

$$T(n) = O(n)$$

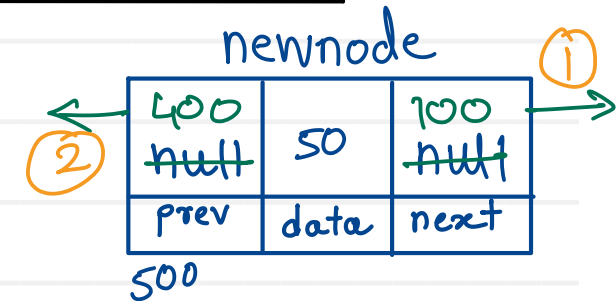
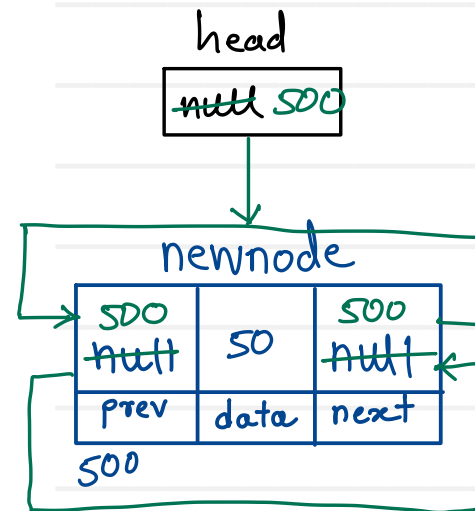
Doubly Circular Linked List - Add first



Doubly Circular Linked List - Add last

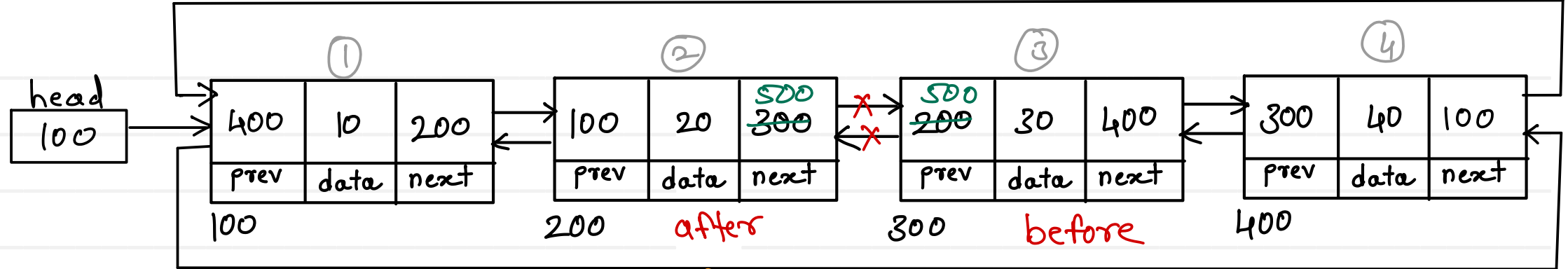


- Create node
- if list is empty
 - add newnode into head
 - make list circular
- if list is not empty
 - add first node into next of new node
 - add last node into prev of new node
 - add newnode into next of last node
 - add newnode into prev of first node

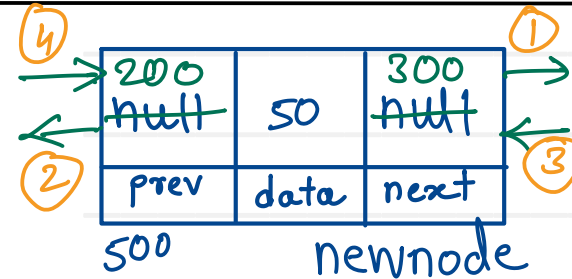


$$T(n) = O(1)$$

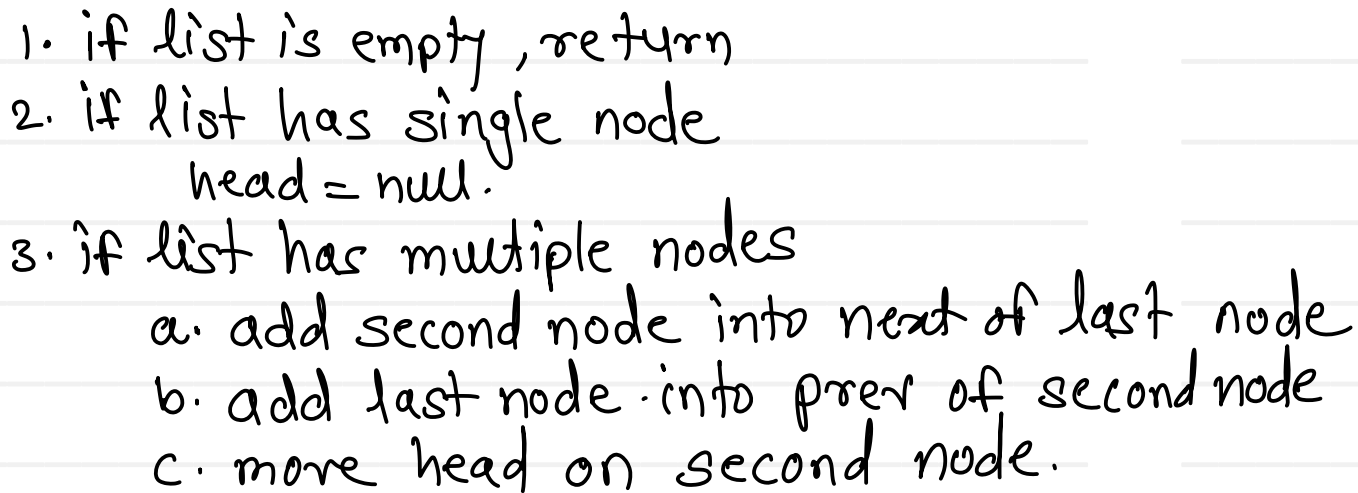
Doubly Circular Linked List - Add position



$\text{newnode.next} = \text{before};$
 $\text{newnode.prev} = \text{after};$
 $\text{before.prev} = \text{newnode};$
 $\text{after.next} = \text{newnode};$

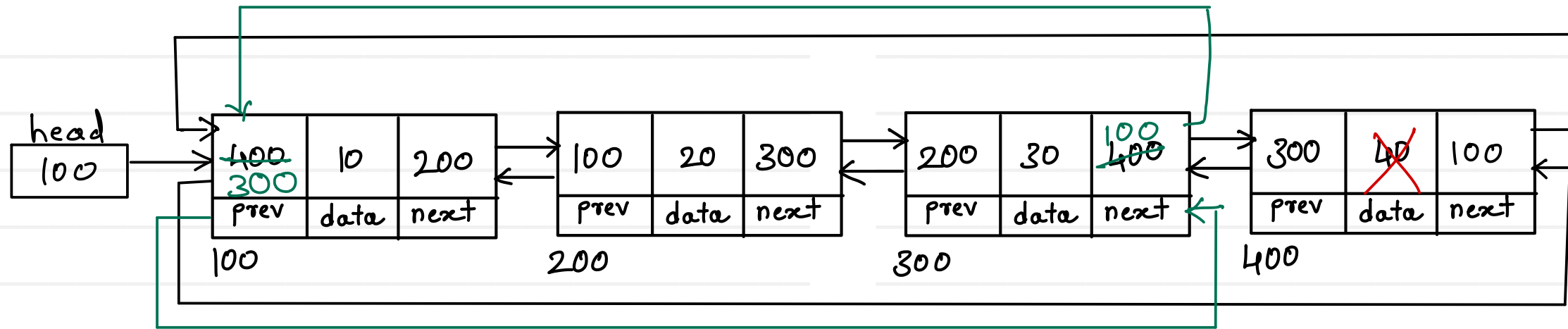


$$T(n) = O(n)$$





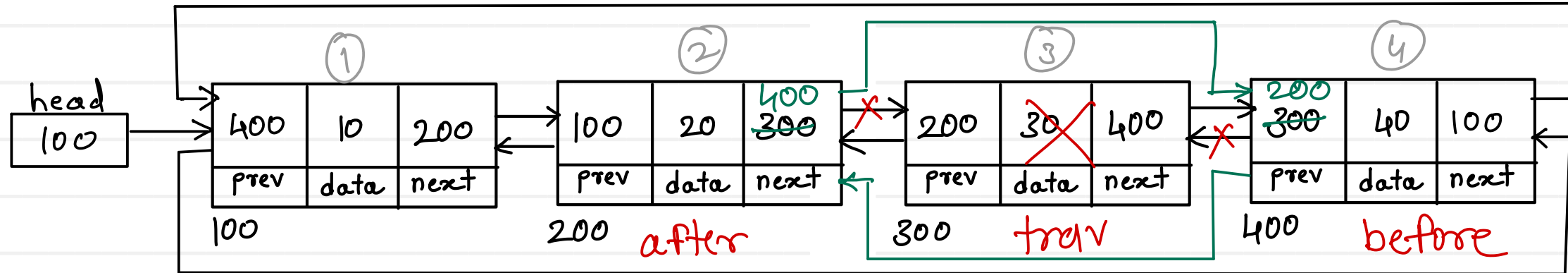
Doubly Circular Linked List - Delete last



1. if list is empty, return
2. if list has single node
head = null;
3. if list has multiple nodes
 - a. add first node into next of second last node
 - b. add second last node into prev of first node

$$T(n) = O(1)$$

Doubly Circular Linked List - Delete position



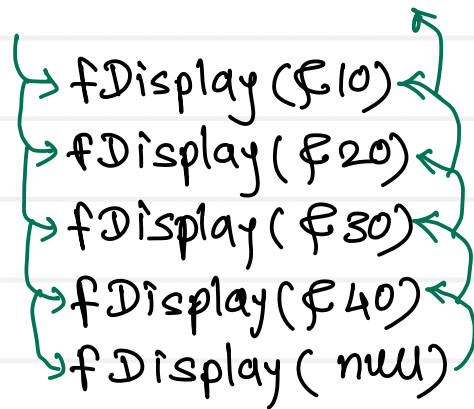
$after = trav.prev;$
 $before = trav.next;$
 $after.next = before;$
 $before.prev = after;$

$$T(n) = O(n)$$

Singly linear linked list - Display reverse

head Tail Recursion
 ↓
 10 → 20 → 30 → 40 → Null

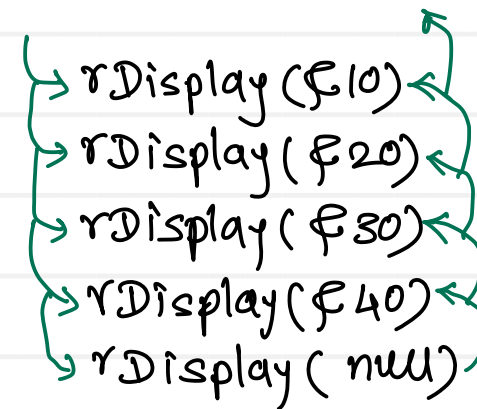
```
void fDisplay(Node trav) {
    if (trav == null)
        return;
    sysout(trav.data);
    fDisplay(trav.next);
}
```



10 20 30 40

head Non tail Recursion
 ↓
 10 → 20 → 30 → 40 → Null

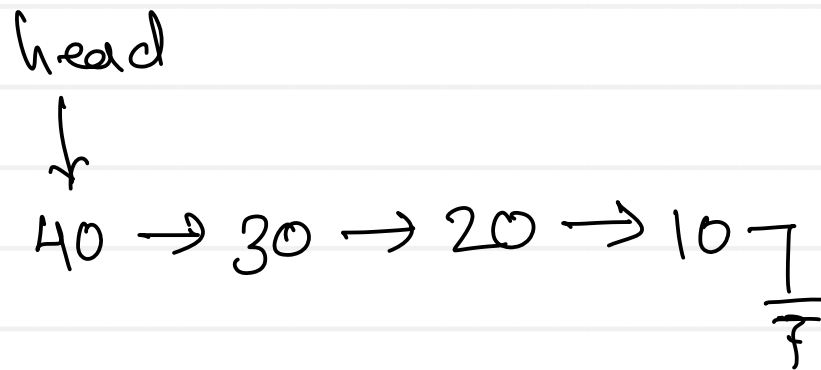
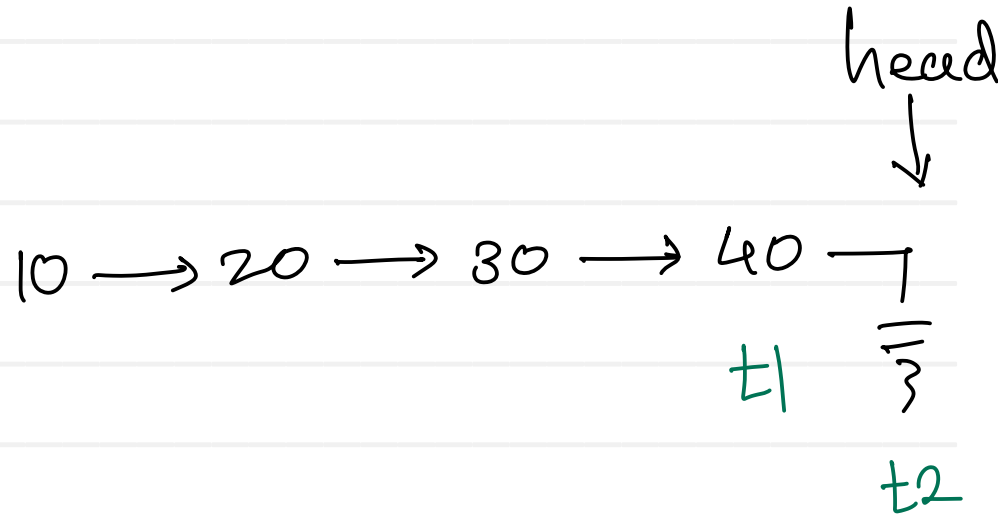
```
void rDisplay(Node trav) {
    if (trav == null)
        return;
    rDisplay(trav.next);
    sysout(trav.data);
}
```



40 30 20 10



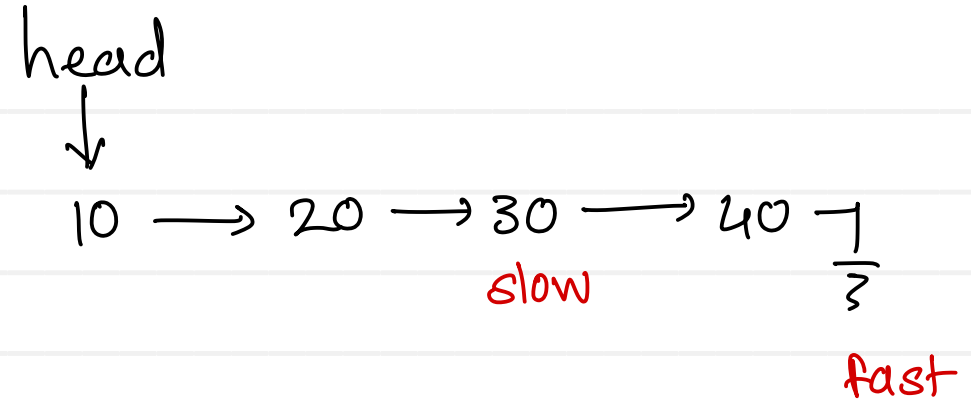
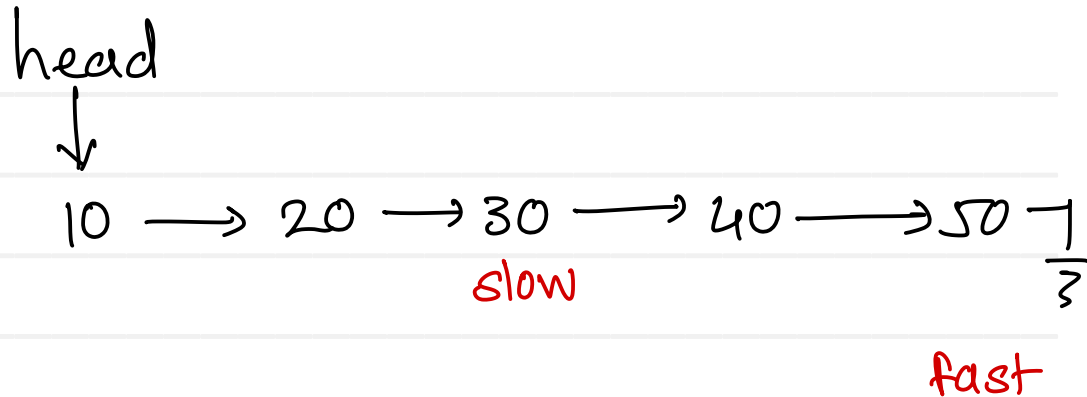
Singly linear linked list - Reverse



```
Node t1 = null;
Node t2 = head;
while (t2 != null) {
    head = t2.next;
    t2.next = t1;
    t1 = t2;
    t2 = head;
}
head = t1;
```

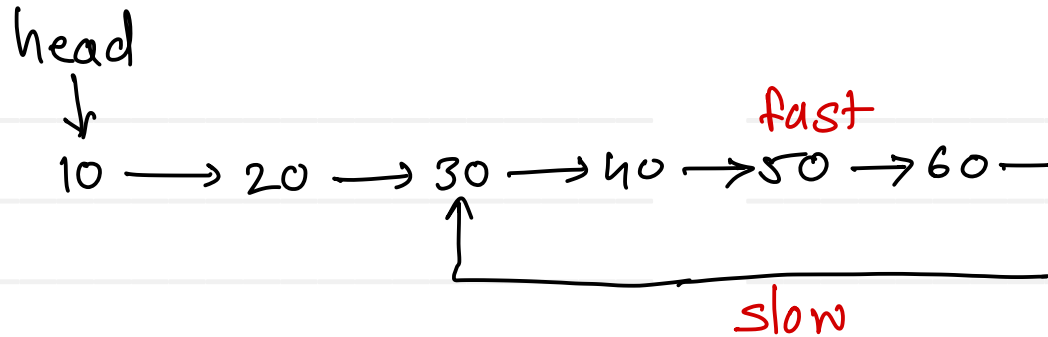


Singly linear linked list - Find mid



```
void findMid( ) {  
    Node fast = head, slow = head;  
    while ( fast != null && fast.next != null ) {  
        fast = fast.next.next;  
        slow = slow.next;  
    }  
    sysout( slow.data );  
}
```


Singly linear linked list - Detect loop



```
boolean hasLoop( ) {
```

```
    Node fast = head, slow = head;
```

```
    while ( fast != null && fast.next != null ) {
```

```
        fast = fast.next.next;
```

```
        slow = slow.next;
```

```
        if ( fast == slow )
```

```
            return true;
```

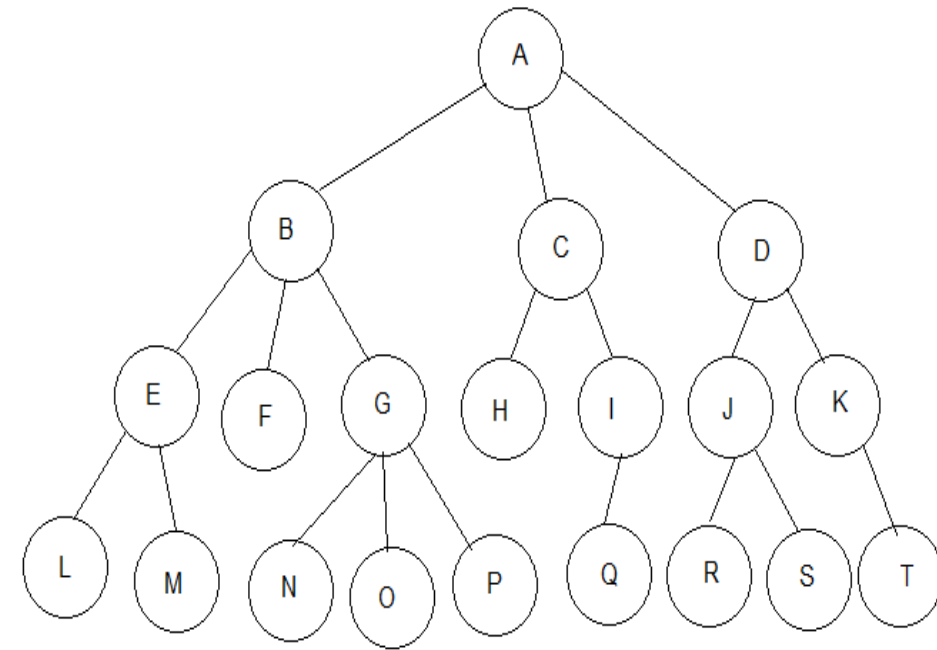
```
    }
```

```
    return false;
```

```
}
```

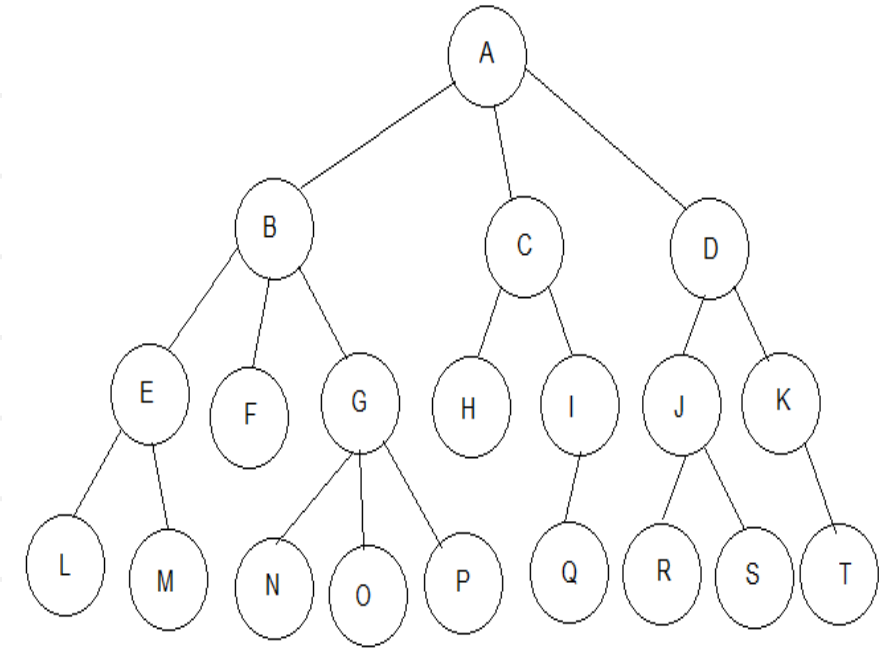
Tree - Terminologies

- **Tree** is a **non linear** data structure which is a finite set of nodes with one specially designated node is called as “**root**” and remaining nodes are partitioned into m disjoint subsets where each of subset is a tree..
- **Root** is a **starting point** of the tree.
- All nodes are connected in **Hierarchical manner (multiple levels)**.
- **Parent node:-** having other child nodes connected
- **Child node:-** immediate descendant of a node
- **Leaf node:-**
 - Terminal node of the tree.
 - Leaf node does not have child nodes.
- **Ancestors:-** all nodes in the path from root to that node.
- **Descendants:-** all nodes accessible from the given node
- **Siblings:-** child nodes of the same parent



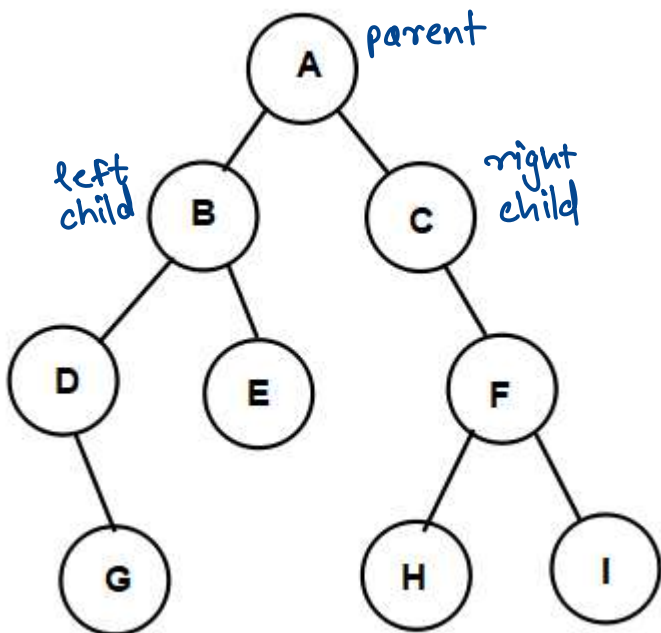
Tree - Terminologies

- **Degree of a node** :- number of child nodes for any given node.
- **Degree of a tree** :- Maximum degree of any node in tree.
- **Level of a node** :- indicates position of the node in tree hierarchy
 - Level of child = Level of parent + 1
 - Level of root = 0
- **Height of node** :- number of links from node to longest leaf.
- **Depth of node** :- number of links from root to that node
- **Height of a tree** :- Maximum height of a node
- **Depth of a tree** :- Maximum depth of a node
- Tree with zero nodes (ie empty tree) is called as “**Null tree**”. Height of Null tree is -1.
 - Tree can grow up to any level and any node can have any number of Childs.
 - That's why operations on tree becomes un efficient.
 - Restrictions can be applied on it to achieve efficiency and hence there are different types of trees.



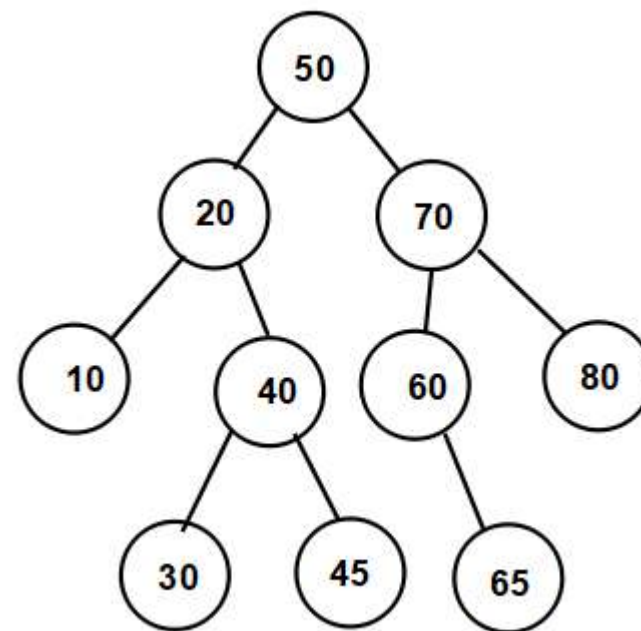
- **Binary Tree**

- Tree in which each node has maximum two child nodes
- Binary tree has degree 2. Hence it is also called as 2- tree



- **Binary Search Tree**

- Binary tree in which left child node is always smaller and right child node is always greater or equal to the parent node.
- Searching is faster
- Time complexity : $O(h)$ h – height of tree



Binary Search Tree - Implementation

Node :

data →

left → reference

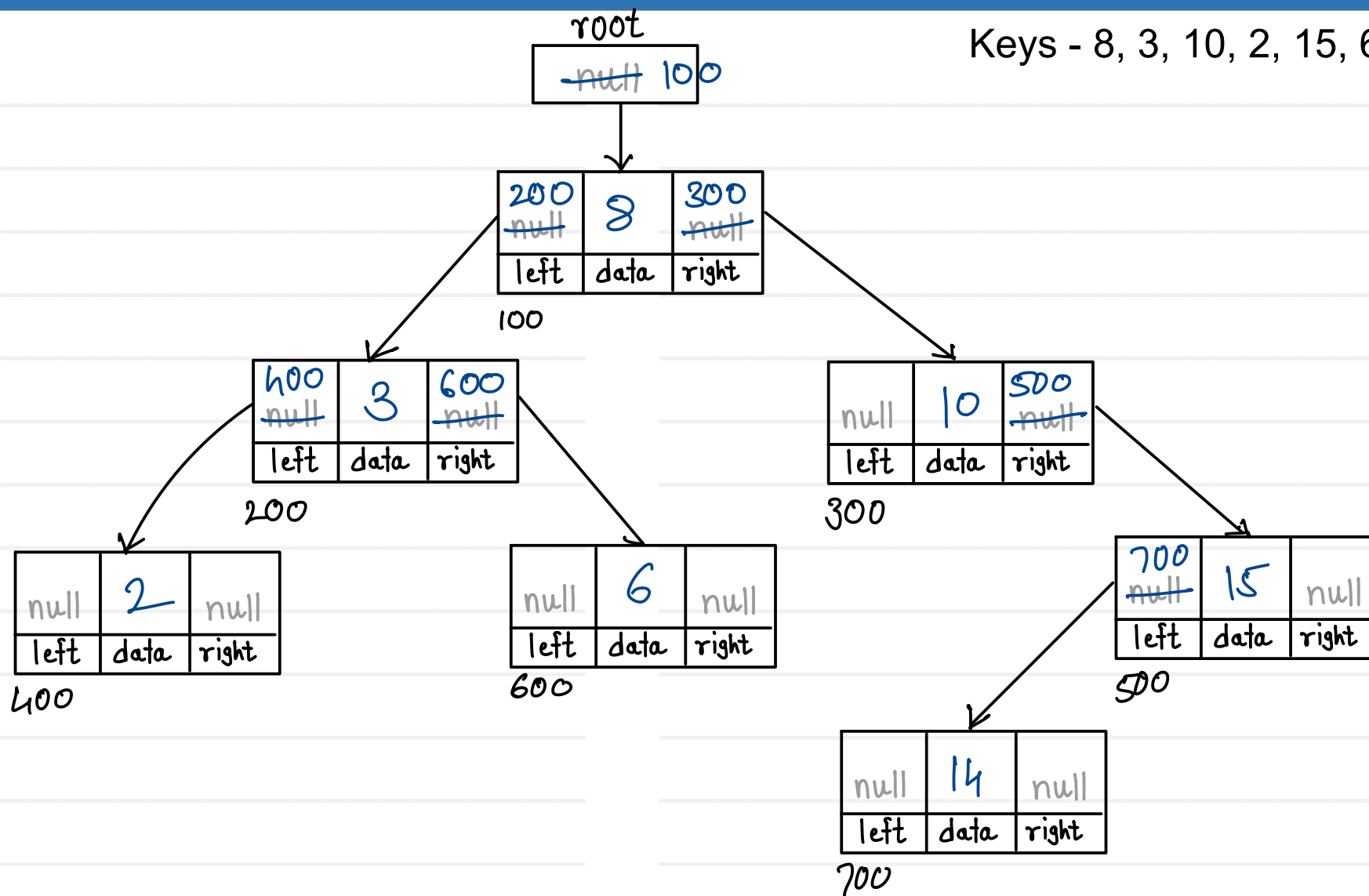
right → reference

```
class Node {  
    int data;  
    Node left;  
    Node right;  
}
```

```
class BST {  
    static class Node {  
        int data;  
        Node left;  
        Node right;  
    }  
    Node root;  
    public BST() { ... }  
    public add() { ... }  
    public delete() { ... }  
    public search() { ... }  
    public traverse() { ... }  
}
```

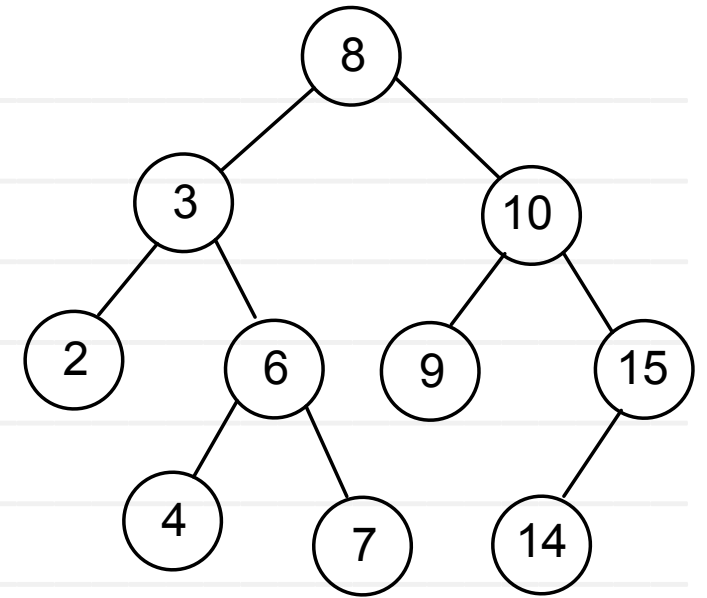
Binary Search Tree - Add Node

Keys - 8, 3, 10, 2, 15, 6, 14, 4, 7, 9



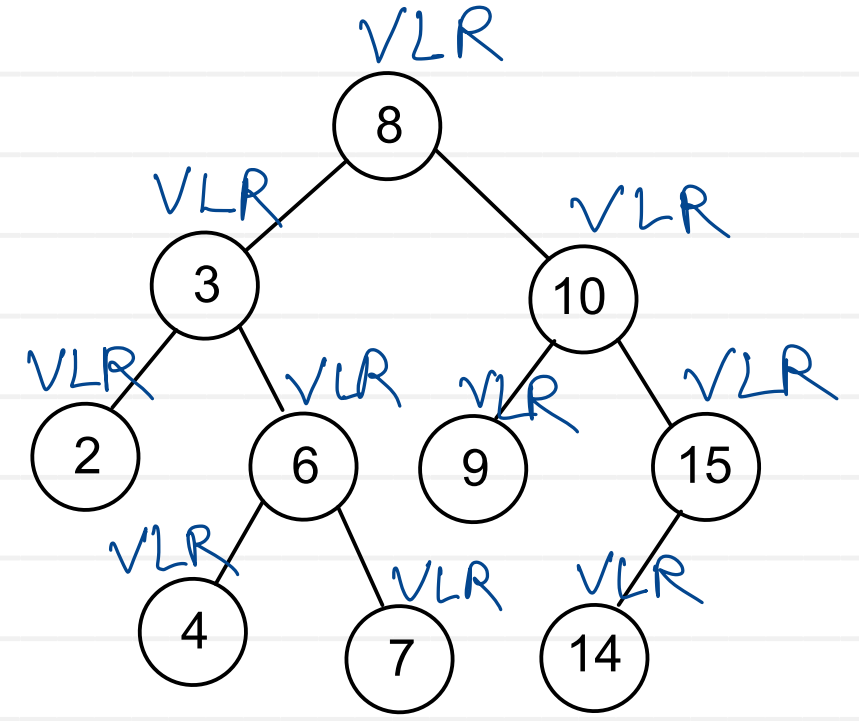
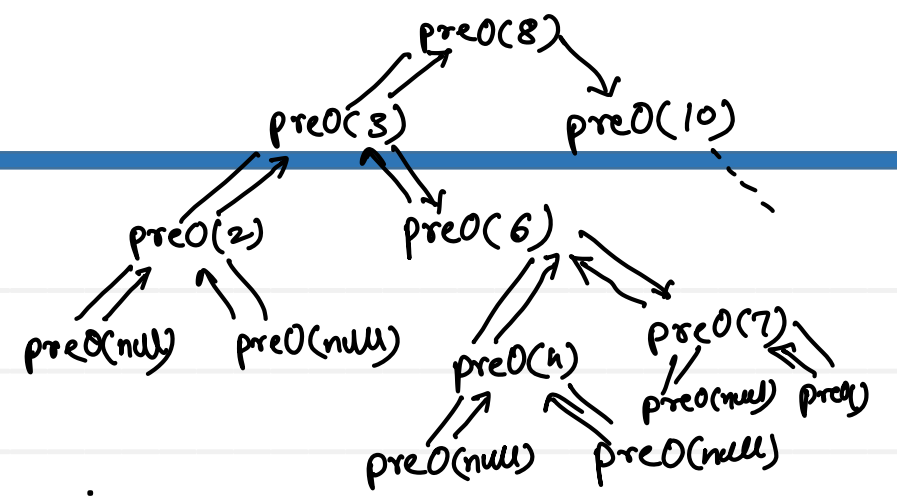
Binary Search Tree – Add Node

```
//1. create node for given value
//2. if BSTree is empty
    // add newnode into root itself
//3. if BSTree is not empty
    //3.1 create trav reference and start at root node
    //3.2 if value is less than current node data (trav.data)
        //3.2.1 if left of current node is empty
            // add newnode into left of current node
        //3.2.2 if left of current node is not empty
            // go into left of current node
    //3.3 if value is greater or equal than current node data (trav.data)
        //3.3.1 if right of current node is empty
            // add newnode into right of current node
        //3.3.2 if right of current node is not empty
            // go into right of current node
    //3.4 repeat step 3.2 and 3.3 till node is not getting added into BSTree
```



- **Pre-Order:- V L R**
- **In-order:- L V R**
- **Post-Order:- L R V**
- The traversal algorithms can be implemented easily using recursion.
- Non-recursive algorithms for implementing traversal needs stack to store node pointers.

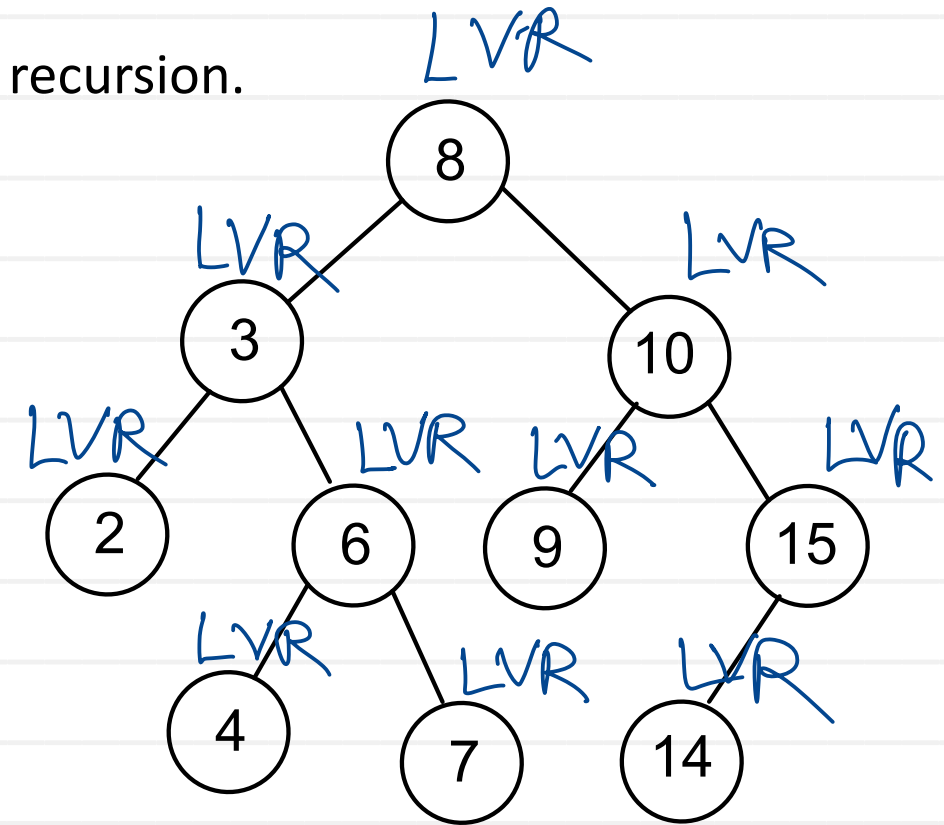
• **Pre-Order :-** 8, 3, 2, 6, 4, 7, 10, 9, 15, 14



Tree Traversal Techniques

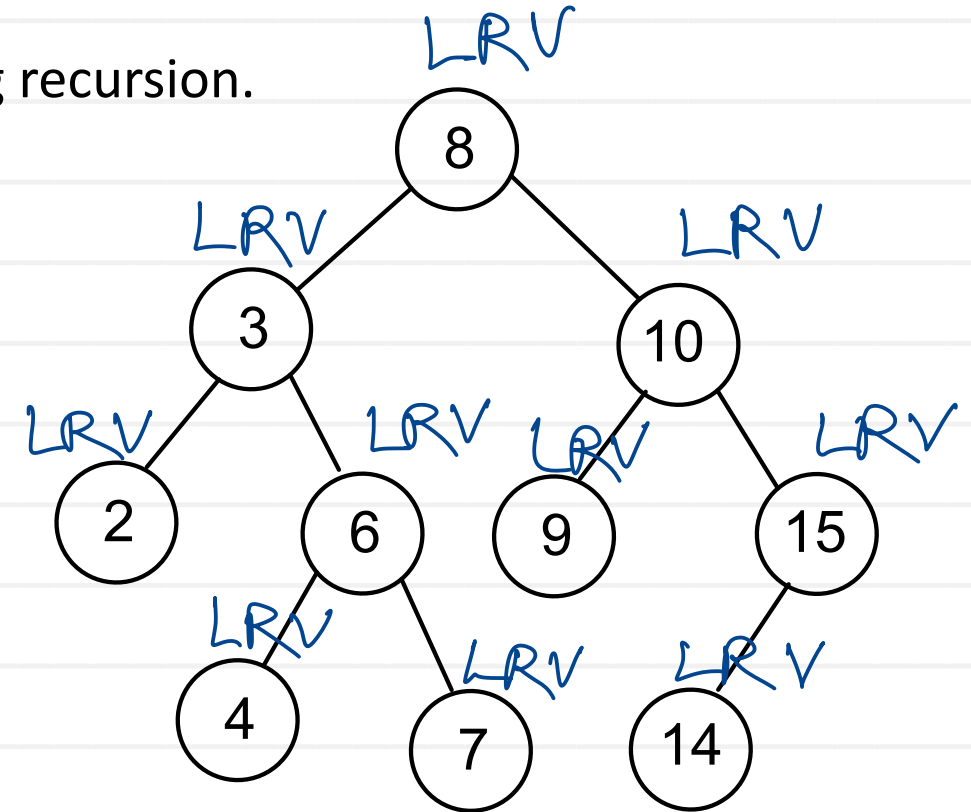
- **Pre-Order:-** V L R
- **In-order:-** L V R
- **Post-Order:-** L R V
- The traversal algorithms can be implemented easily using recursion.
- Non-recursive algorithms for implementing traversal needs stack to store node pointers.

• **In-Order :-** 2, 3, 4, 6, 7, 8, 9, 10, 14, 15

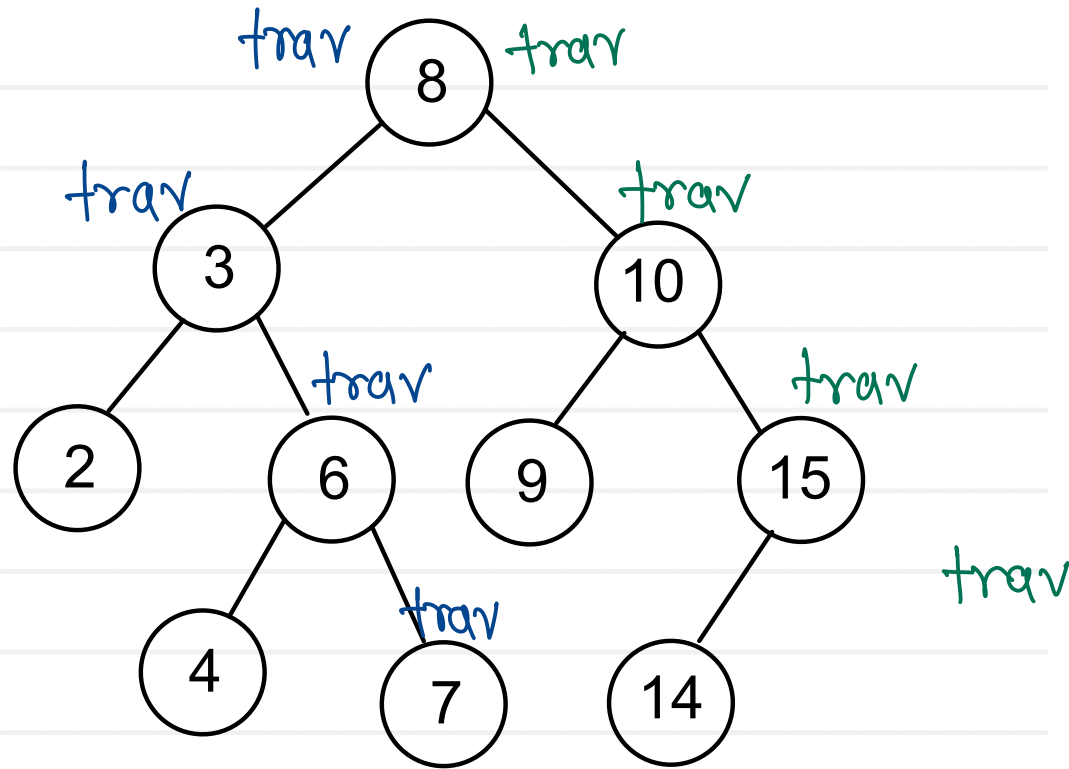


- **Pre-Order:-** V L R
- **In-order:-** L V R
- **Post-Order:-** L R V
- The traversal algorithms can be implemented easily using recursion.
- Non-recursive algorithms for implementing traversal needs stack to store node pointers.

• **Post-Order :-** 2, 4, 7, 6, 3, 9, 14, 15, 10, 8



Binary Search Tree - Binary Search



1. Start from root
2. If key is equal to current node data return current node
3. If key is less than current node data search key into left sub tree of current node
4. If key is greater than current node data search key into right sub tree of current node
5. Repeat step 2 to 4 till leaf node

Key = 7 — Key is found
Key = 16 — Key is not found



Thank you!!!

Devendra Dhande

devendra.dhande@sunbeaminfo.com



Sunbeam Institute of Information Technology

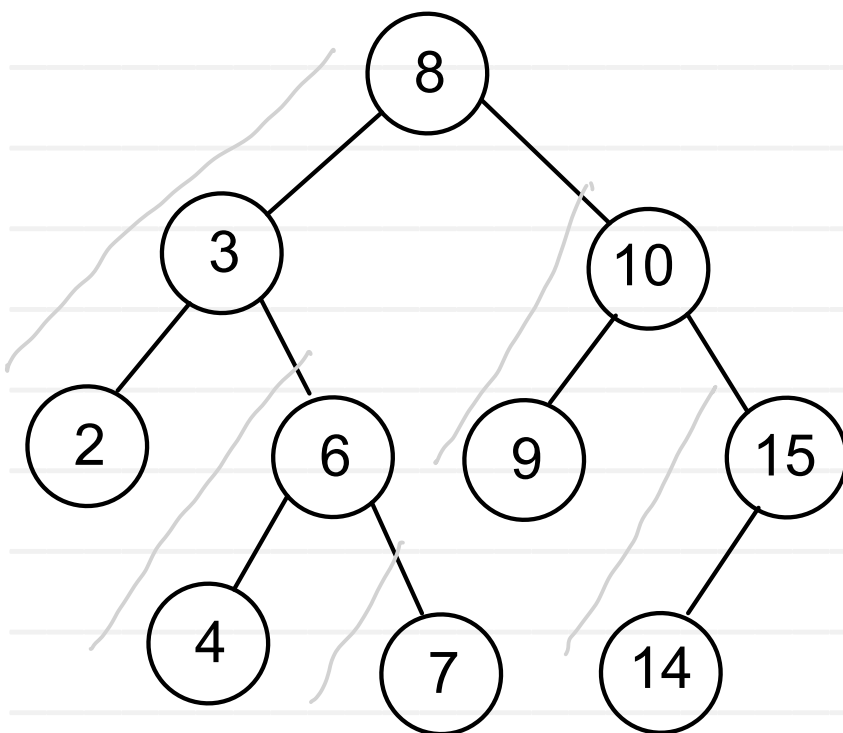
Pune and Karad

Module – Data Structures and Algorithms

Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

Binary Search Tree - DFS Traversal



Stack

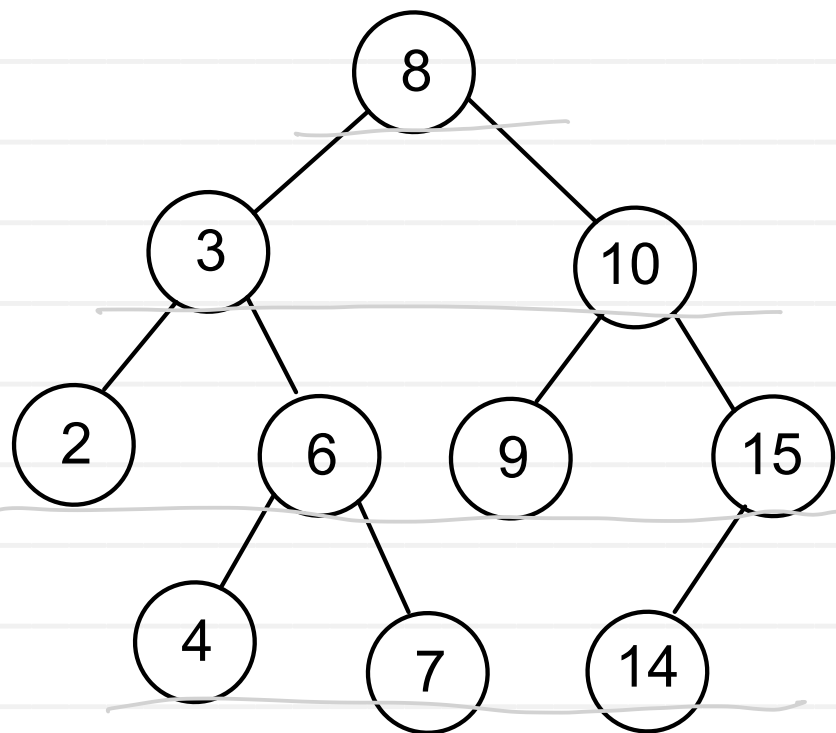
14
9
15
11
7
2
6
3
10
8

(Depth First Search)

1. Push root node on stack
2. Pop one node from stack
3. Visit (print) popped node
4. If right exists, push it on stack
5. If left exists, push it on stack
6. While stack is not empty, repeat step 2 to 5

Traversal: 8, 3, 2, 6, 4, 7, 10, 9, 15, 14

Binary Search Tree - BFS Traversal



Queue

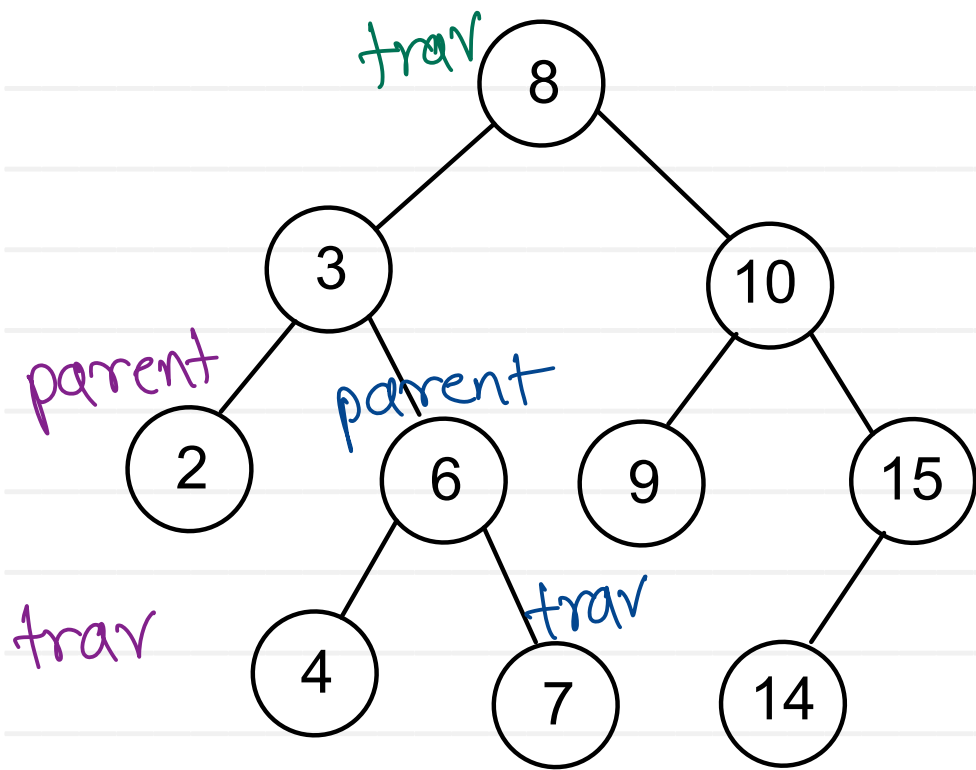
14
7
4
15
9
6
2
10
3
8

(Breadth First Search)

1. Push root node on queue
2. Pop one node from queue
3. Visit (print) popped node
4. If left exists, push it on queue
5. If right exists, push it on queue
6. While queue is not empty, repeat step 2 to 5

Traversal : 8 , 3 , 10 , 2 , 6 , 9 , 15 , 4 , 7 , 14

Binary Search Tree - Binary Search with Parent



Key = 1

trav	parent
\$8	null
\$3	\$8
\$2	\$3
null	\$2

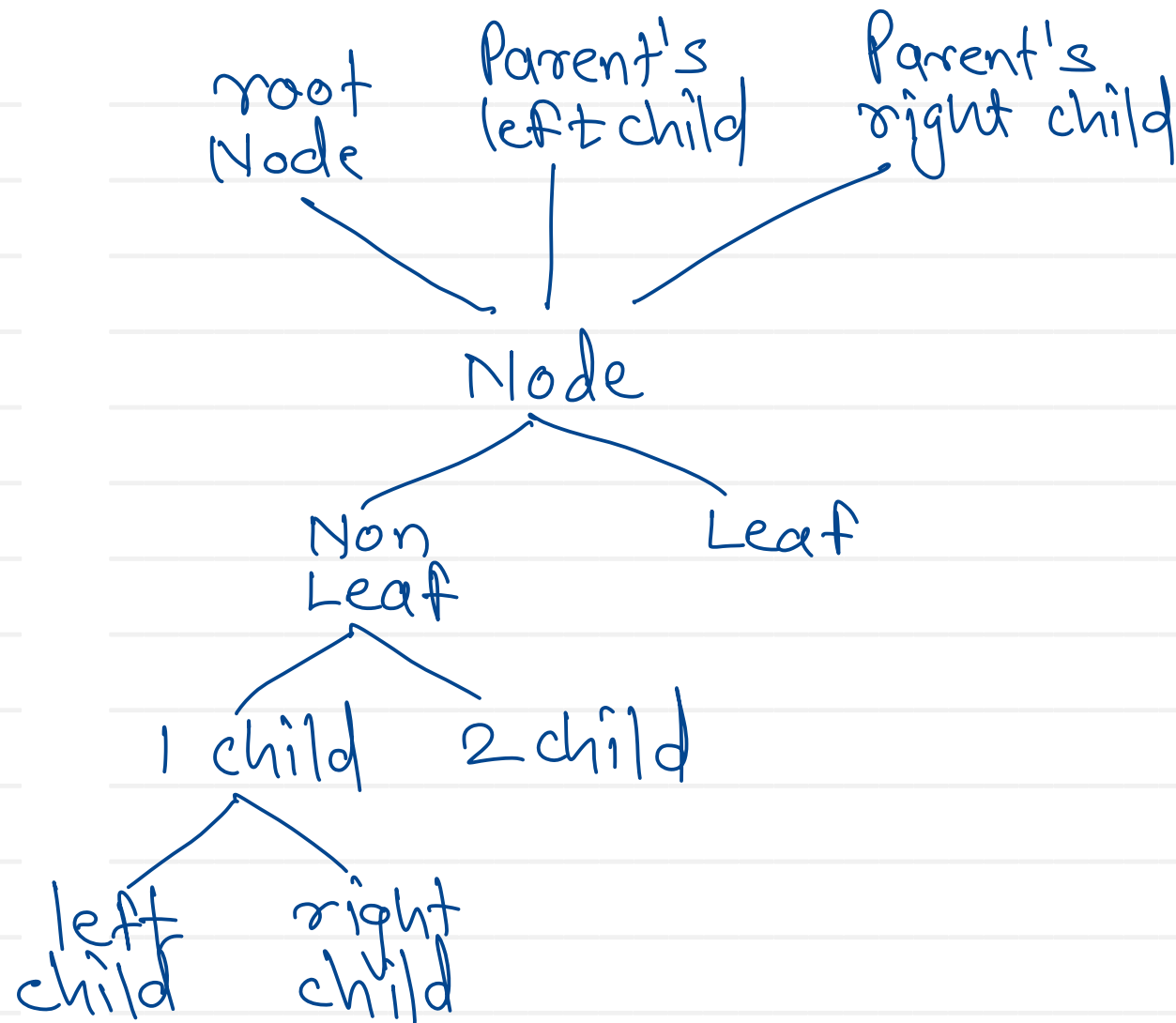
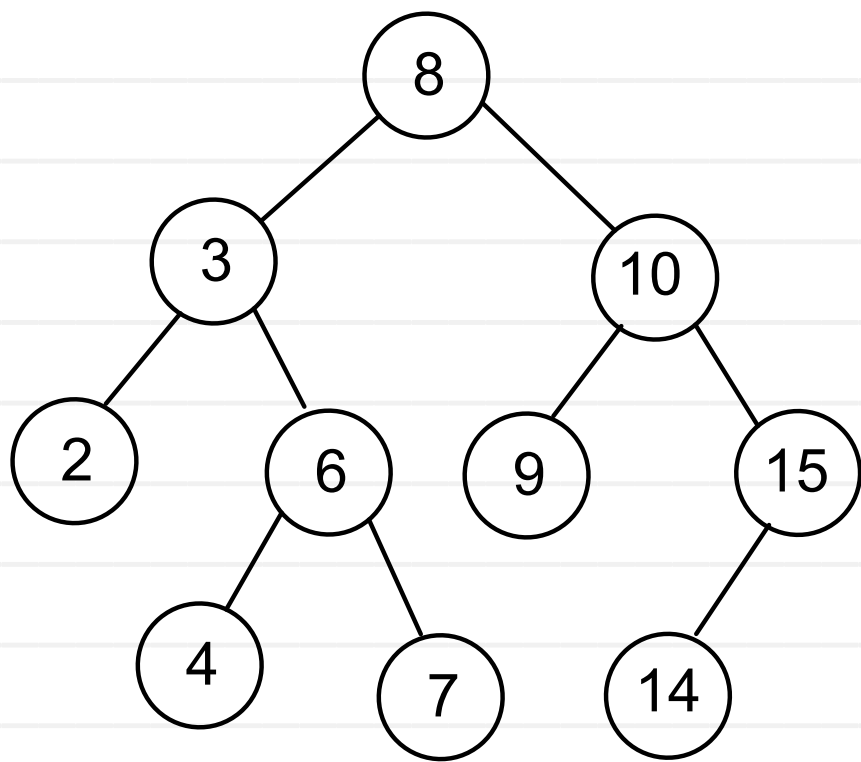
Key = 7

trav	parent
\$8	null
\$3	\$8
\$6	\$3
\$7	\$6

Key = 8

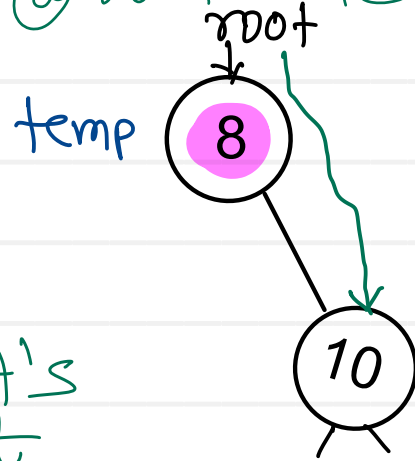
trav	parent
\$8	null

Binary Search Tree - Delete Node

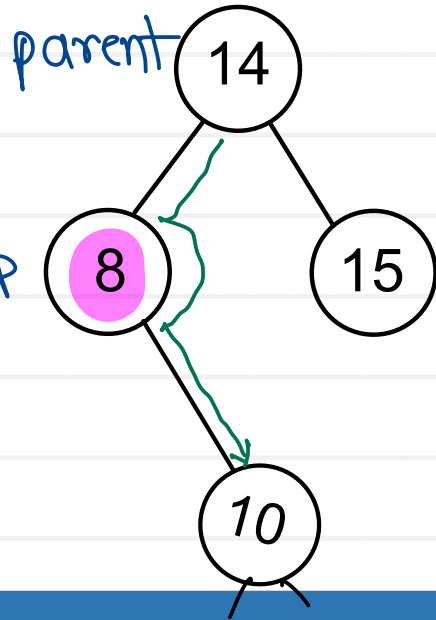


BST - Delete Single child node (Right child)

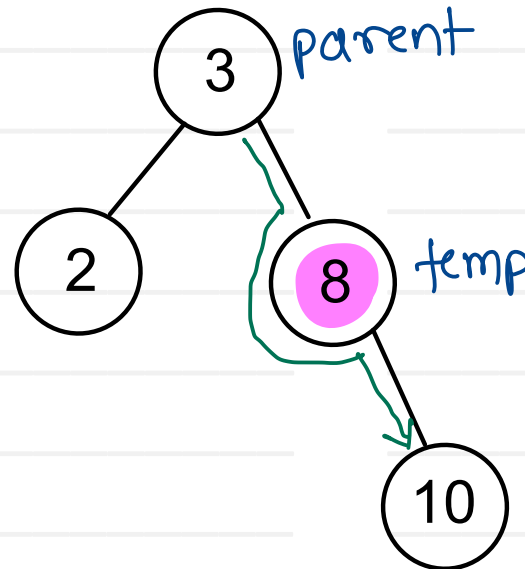
@root node



⑥ parent's left

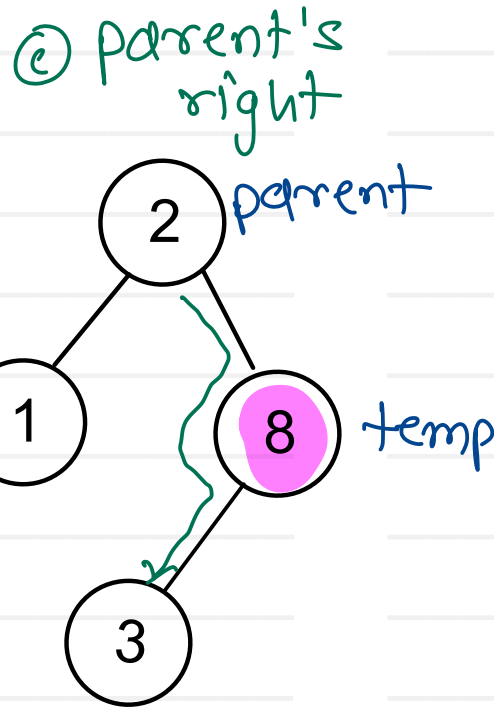
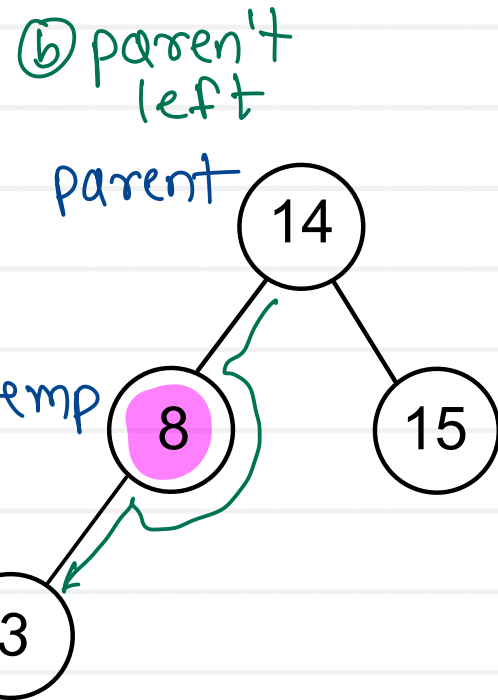
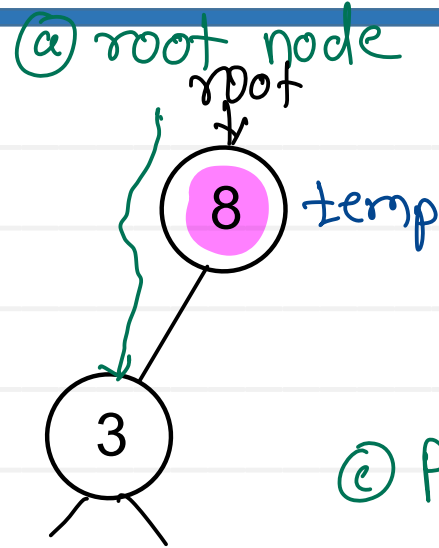


⑦ parent's right



```
if ( temp.left == null ) {
    if ( temp == root )
        root = temp.right;
    else if ( temp == parent.left )
        parent.left = temp.right;
    else if ( temp == parent.right )
        parent.right = temp.right;
}
```

BST- Delete Single child node (Left child)



```

if (temp.right == null) {
    if (temp == root)
        root = temp.left;
    else if (temp == parent.left)
        parent.left = temp.left;
    else if (temp == parent.right)
        parent.right = temp.left;
}
    
```



Thank you!!!

Devendra Dhande

devendra.dhande@sunbeaminfo.com



Sunbeam Institute of Information Technology

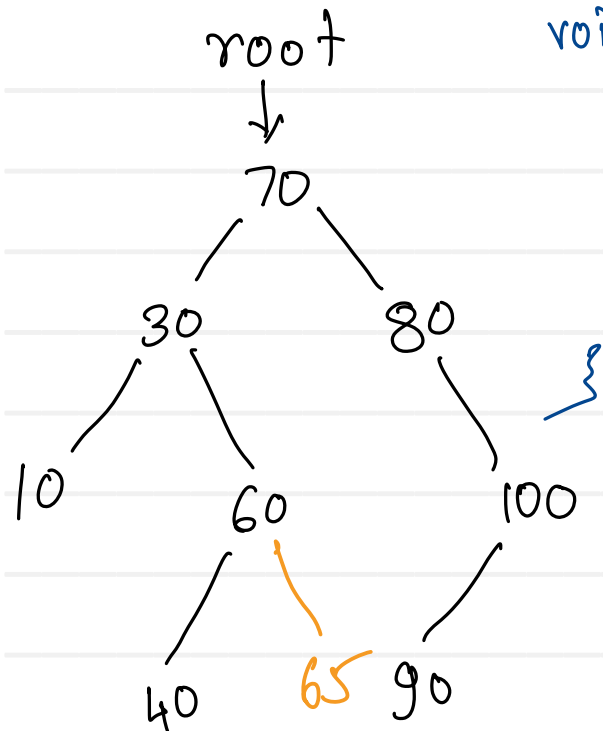
Pune and Karad

Module – Data Structures and Algorithms

Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

Add Node (Recursion)



```
void addNode(int value) {
    if (root == null)
        root = new Node(value);
```

```
    else
        addNode(value, root);
}
```

value = 65

addNode(65)

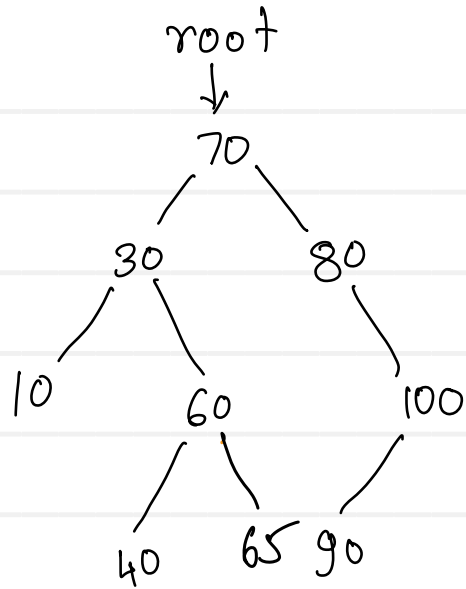
addNode(65, &70)

addNode(65, &30)

addNode(65, &60)

```
void addNode(int value, Node trav) {
    if (value < trav.data) {
        if (trav.left == null) {
            trav.left = new Node(value);
            return;
        }
        else
            addNode(value, trav.left);
    }
    else {
        if (trav.right == null) {
            trav.right = new Node(value);
            return;
        }
        else
            addNode(value, trav.right);
    }
}
```

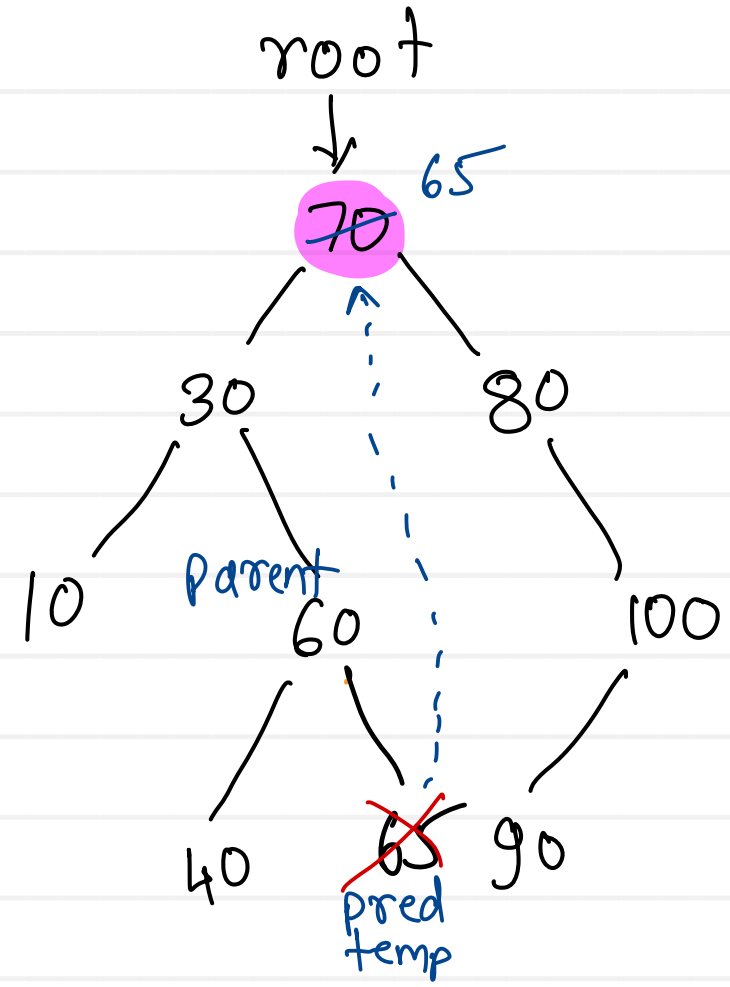
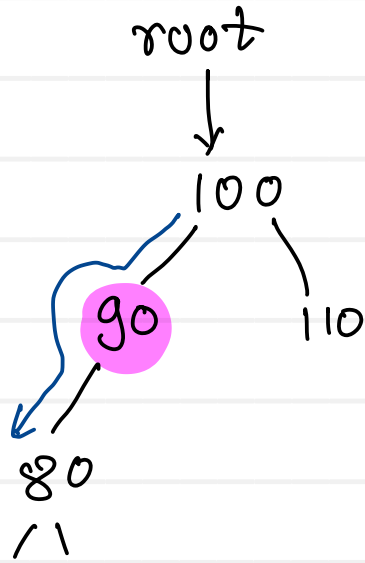
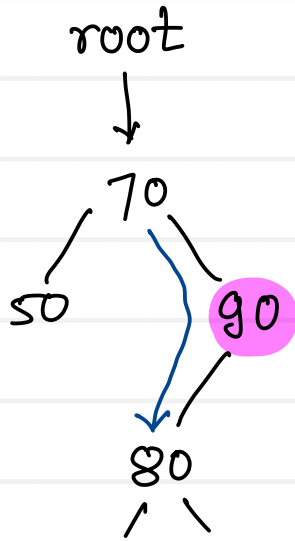
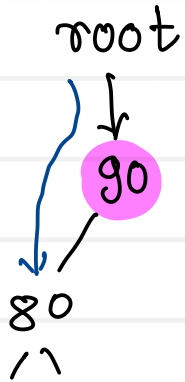
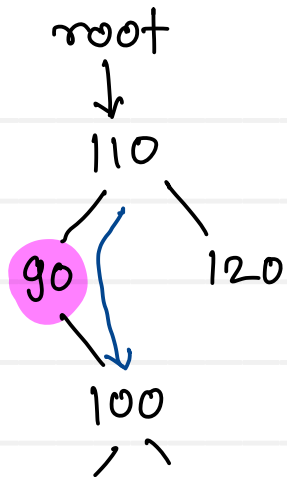
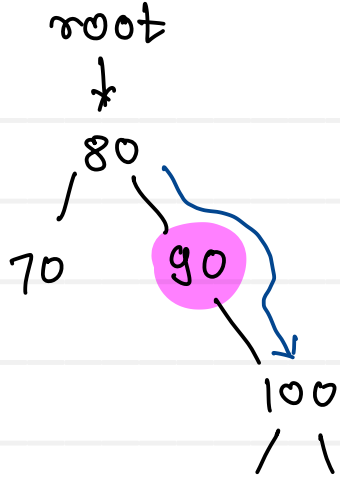
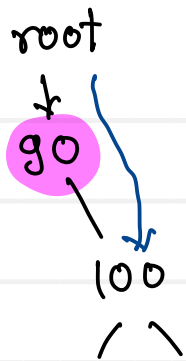
Binary Search (Recursive)



Key = 65
bSRec(65, 70)
bSRec(65, 30)
bSRec(65, 60)
bSRec(65, 65)

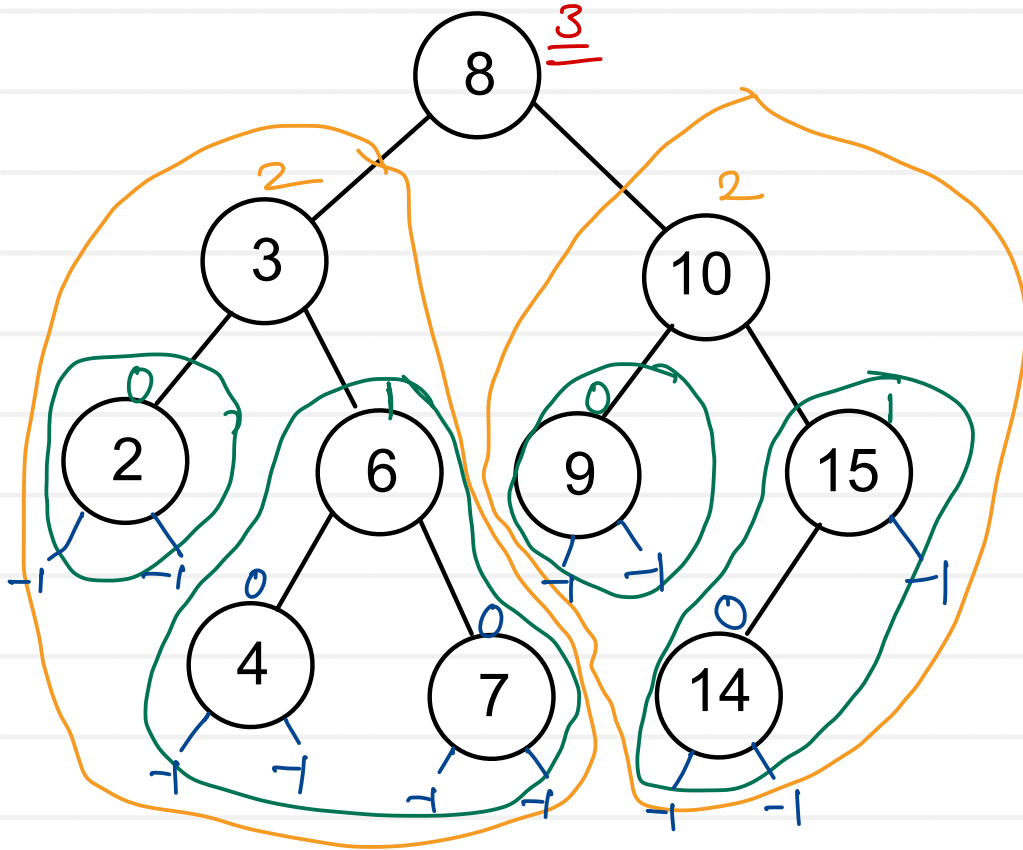
```
Node binarySearchRec(int key, Node trav) {
    if (trav == null)
        return null;
    if (key == trav.data)
        return trav;
    else if (key < trav.data)
        return binarySearchRec(key, trav.left);
    else
        return binarySearchRec(key, trav.right);
}
```

Delete Node



Binary Search Tree - Height

Height of root = MAX (height (left sub tree), height (right sub tree)) + 1



1. If left or right sub tree is absent then return -1
2. Find height of left sub tree
3. Find height of right sub tree
4. Find max height
5. Add one to max height and return

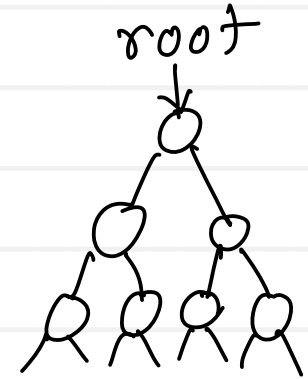
BST - Time complexity of operations

No. of elements = n
height of tree = h

$$n = 2^{h+1} - 1$$

Add	:	$O(h)$	$O(\log n)$
Search	:	$O(h)$	$O(\log n)$
Delete	:	$O(h)$	$O(\log n)$
Traverse	:		$O(n)$

h	n
-1	0
0	1
1	3
2	7
3	15
$\vdots$	$\vdots$



$$n = 2^h$$

$$\log n = \log 2^h$$

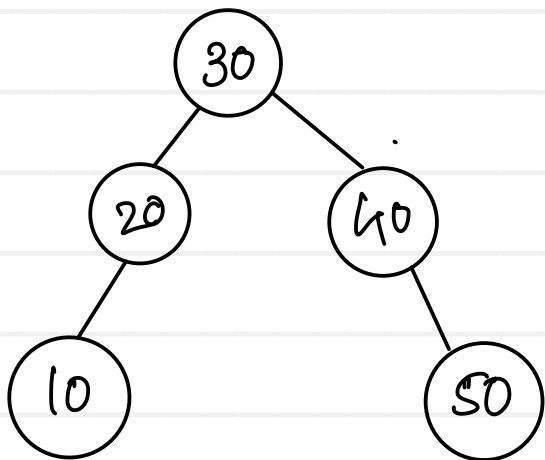
$$h = \frac{\log n}{\log 2}$$

$$\text{Time} \propto h$$

$$\text{Time} \propto \frac{\log n}{\log 2}$$

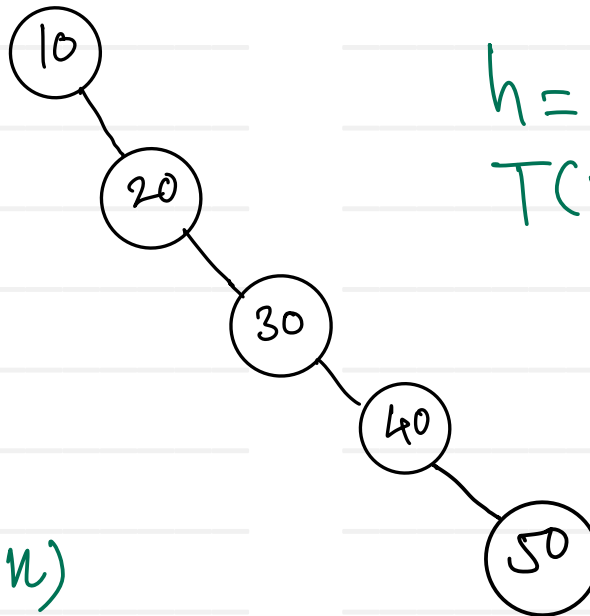
Skewed Binary Search Tree

Keys : 30, 40, 20, 50, 10



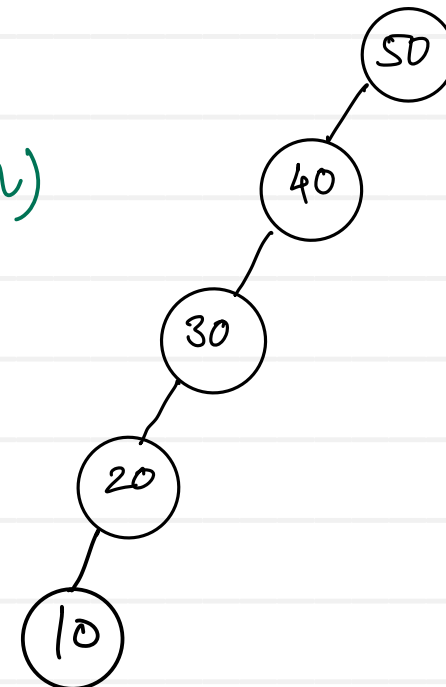
$$h = \log n \quad T(n) = O(\log n)$$

Keys : 10, 20, 30, 40, 50



$$h = n$$
$$T(n) = O(n)$$

Keys : 50, 40, 30, 20, 10



- In binary tree if only left or right links are used, tree grows only in one direction such tree is called as skewed binary tree
 - Left skewed binary tree
 - Right skewed binary tree
- Time complexity of any BST is $O(h)$
- Skewed BST have maximum height ie same as number of elements.
- Time complexity of searching in skewed BST is $O(n)$

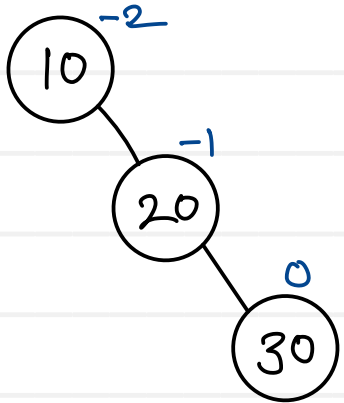
Balanced BST

- To speed up searching, height of BST should be minimum as possible
- If nodes in BST are arranged, so that its height is kept as less as possible, is called as Balanced BST

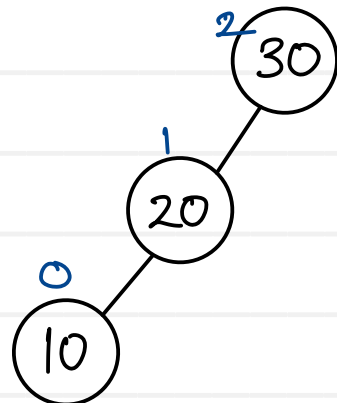
$$\text{Balance factor (nodes)} = \text{Height (left sub tree)} - \text{Height (right sub tree)}$$

- tree is balanced if balance factors of all the nodes is either -1, 0 or +1
- balance factors = $\{-1, 0, +1\}$
- A tree can be balanced by applying series of left or right rotations on imbalance nodes (node having balance factor other than -1, 0 or +1)

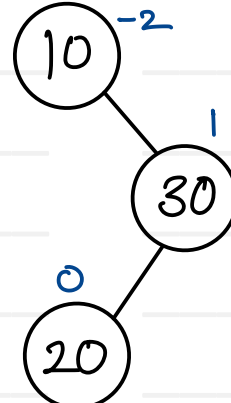
Keys : 10, 20, 30



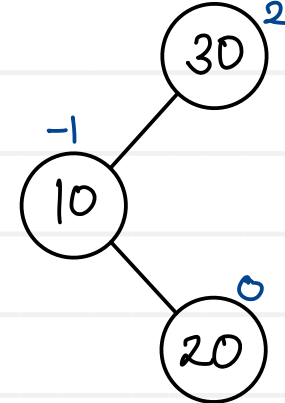
Keys : 30, 20, 10



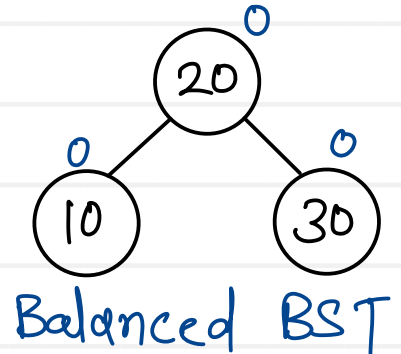
Keys : 10, 30, 20

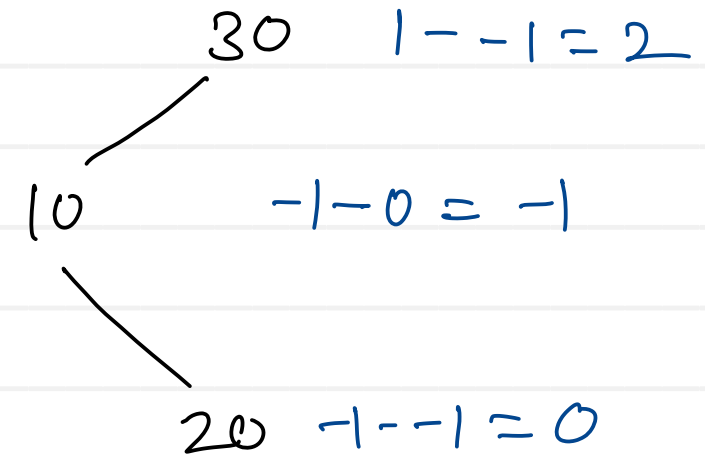
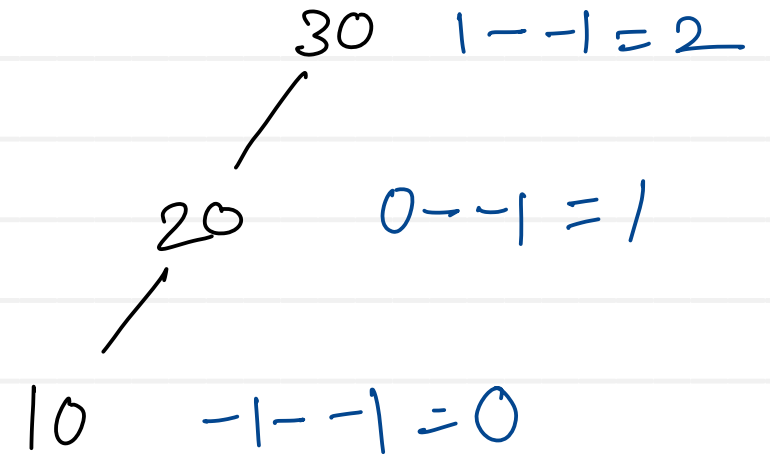
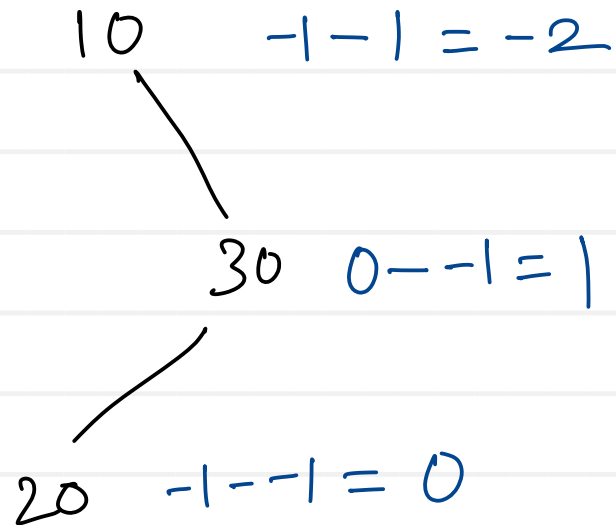
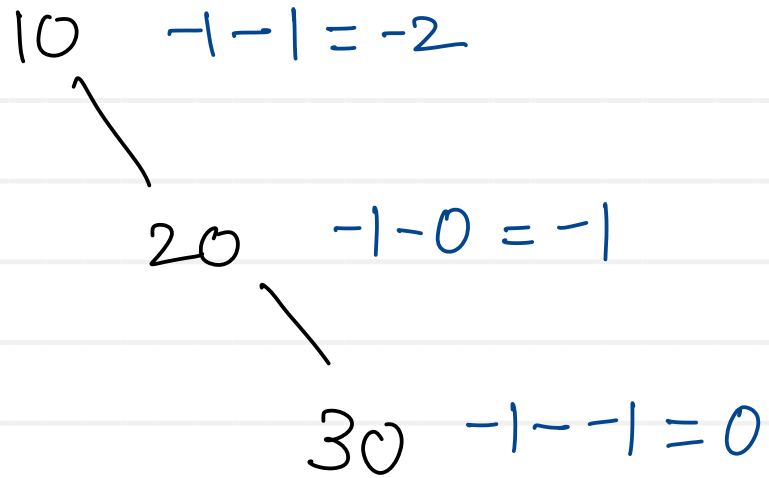


Keys : 30, 10, 20

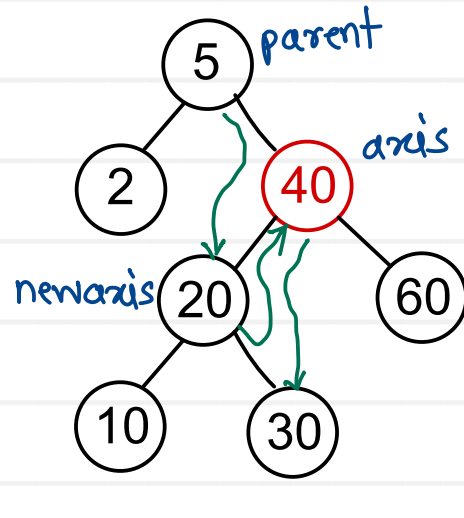
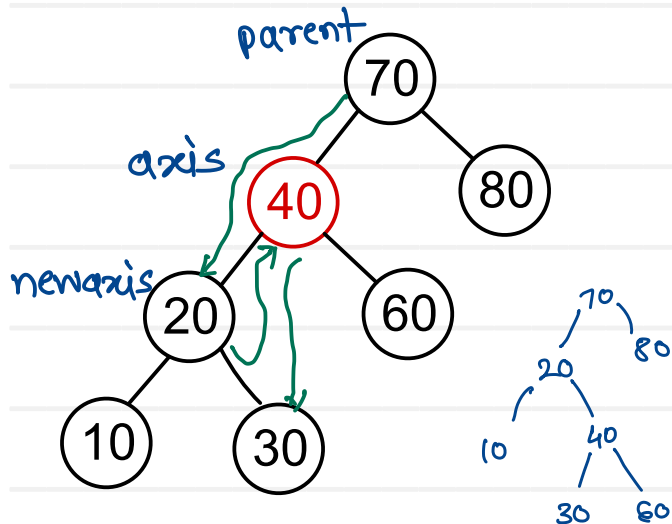
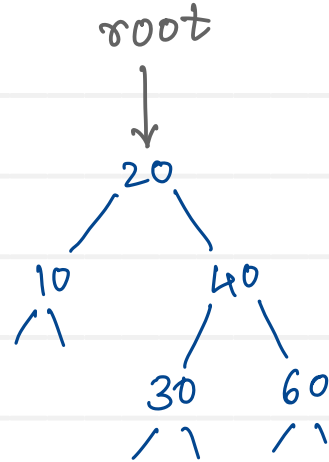
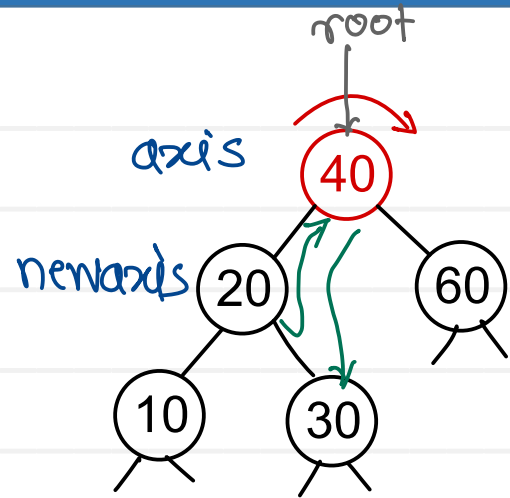


Keys : 20, 10, 30
Keys : 20, 30, 10





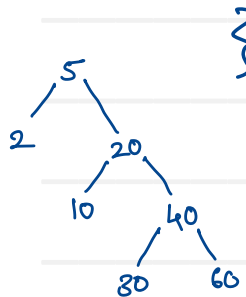
Right Rotation

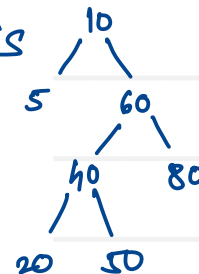
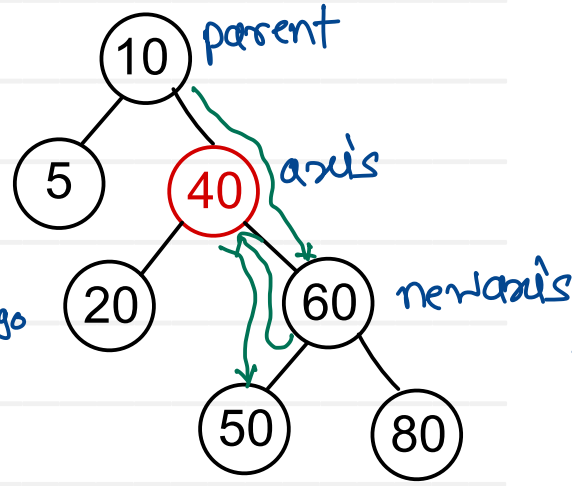
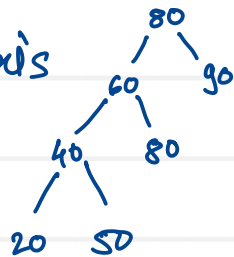
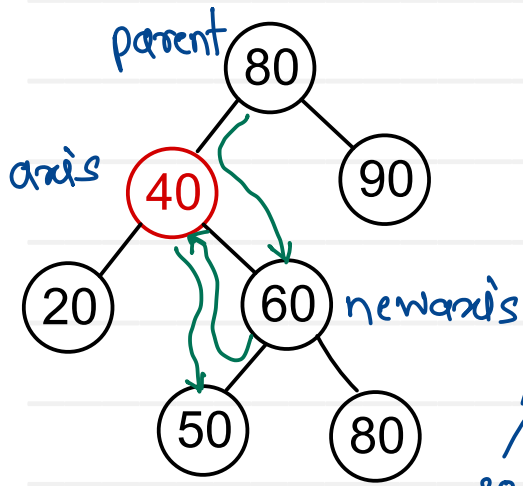
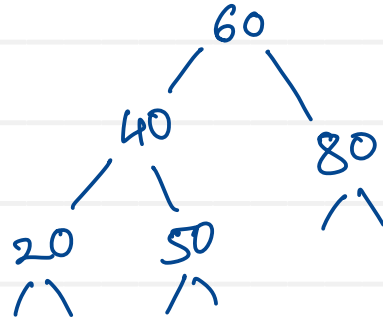
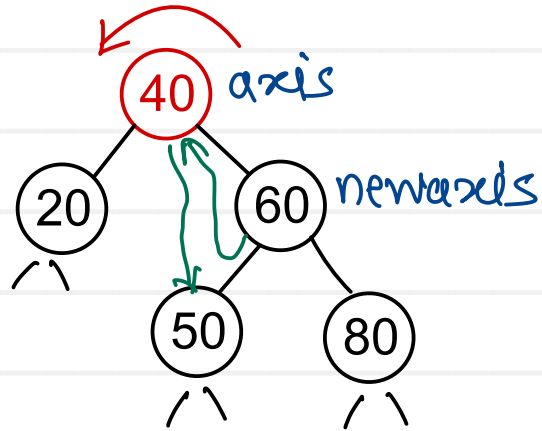


```

rightRotation(Node axis, Node parent) {
    newaxis = axis.left;
    axis.left = newaxis.right;
    newaxis.right = axis;
    if (axis == root)
        root = newaxis;
    else if (axis == parent.left)
        parent.left = newaxis;
    else if (axis == parent.right)
        parent.right = newaxis;
}

```





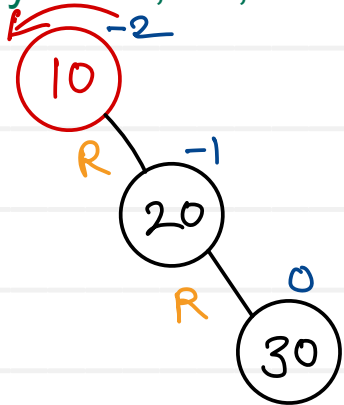
```

leftRotation(Node axis, Node parent){
    newaxis = axis.right;
    axis.right = newaxis.left;
    newaxis.left = axis;
    if(axis == root)
        root = newaxis;
    else if(axis == parent.left)
        parent.left = newaxis;
    else if(axis == parent.right)
        parent.right = newaxis;
}
    
```

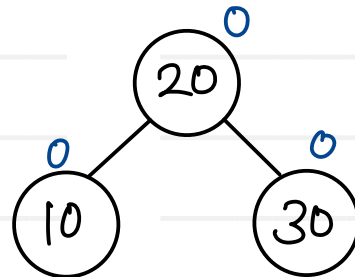
Rotation cases

RR Imbalance

Keys : 10, 20, 30

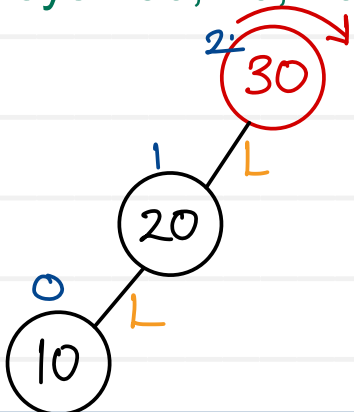


Left
Rotation

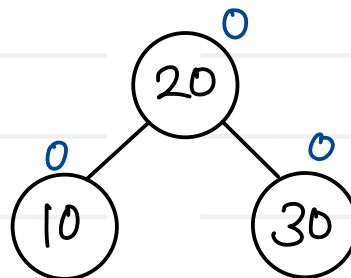


LL Imbalance

Keys : 30, 20, 10



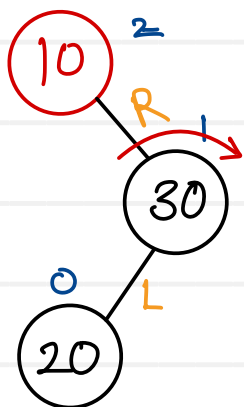
Right
Rotation



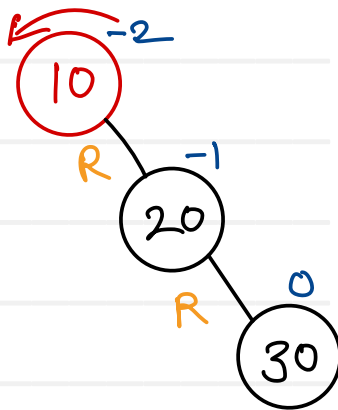
Rotation cases

RL Imbalance

Keys : 10, 30, 20

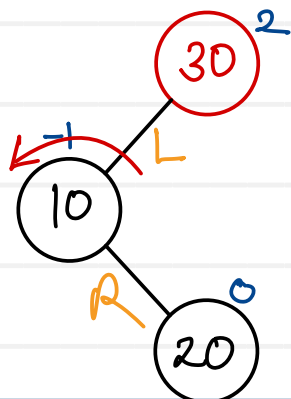


Right
Rotation

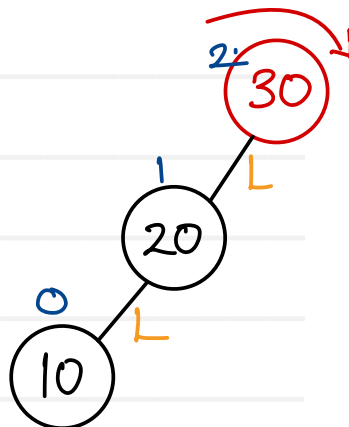


LR Imbalance

Keys : 30, 10, 20

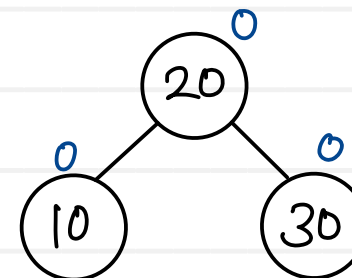


Left
Rotation

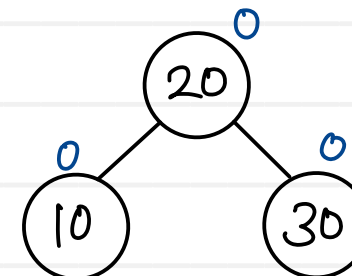


Double Rotation

Left
Rotation



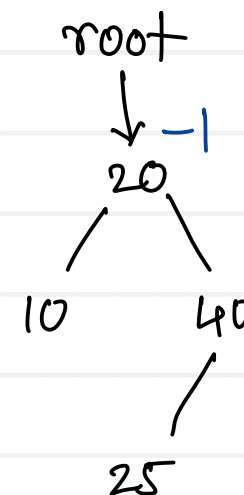
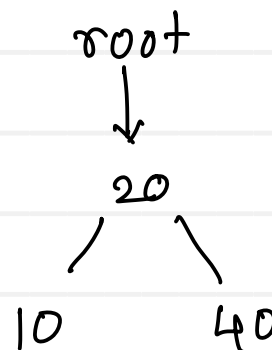
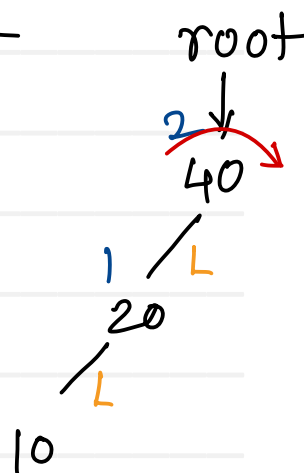
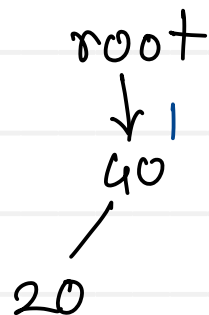
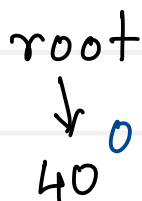
Right
Rotation



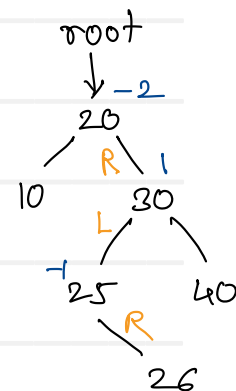
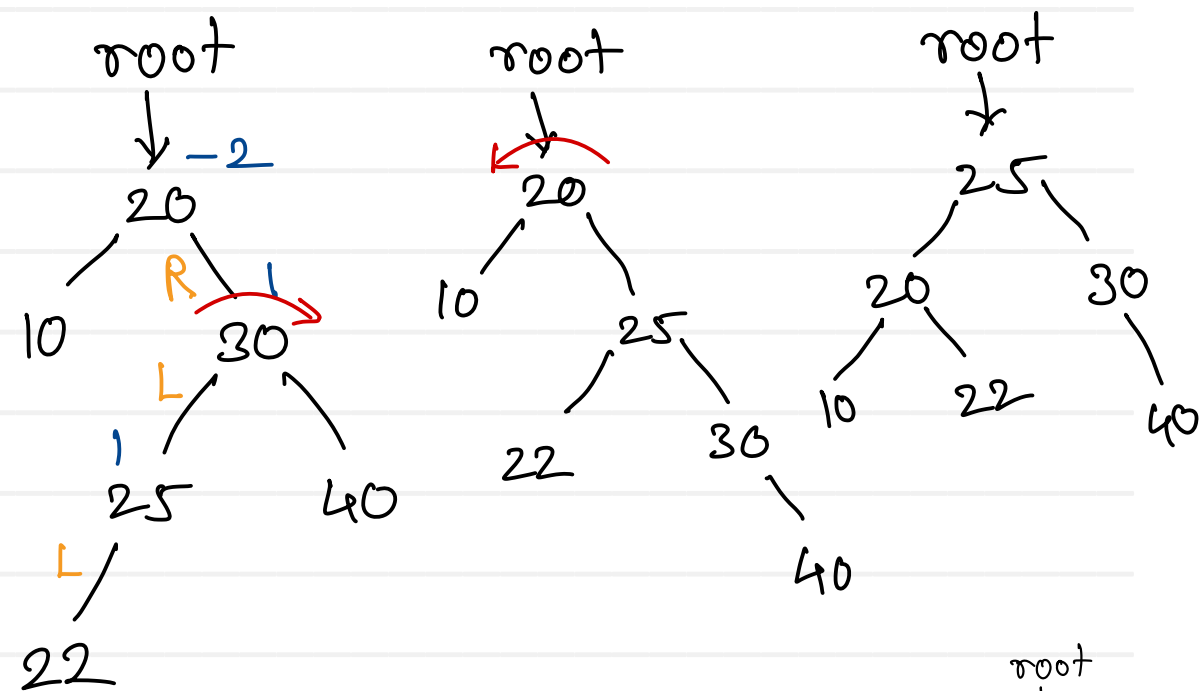
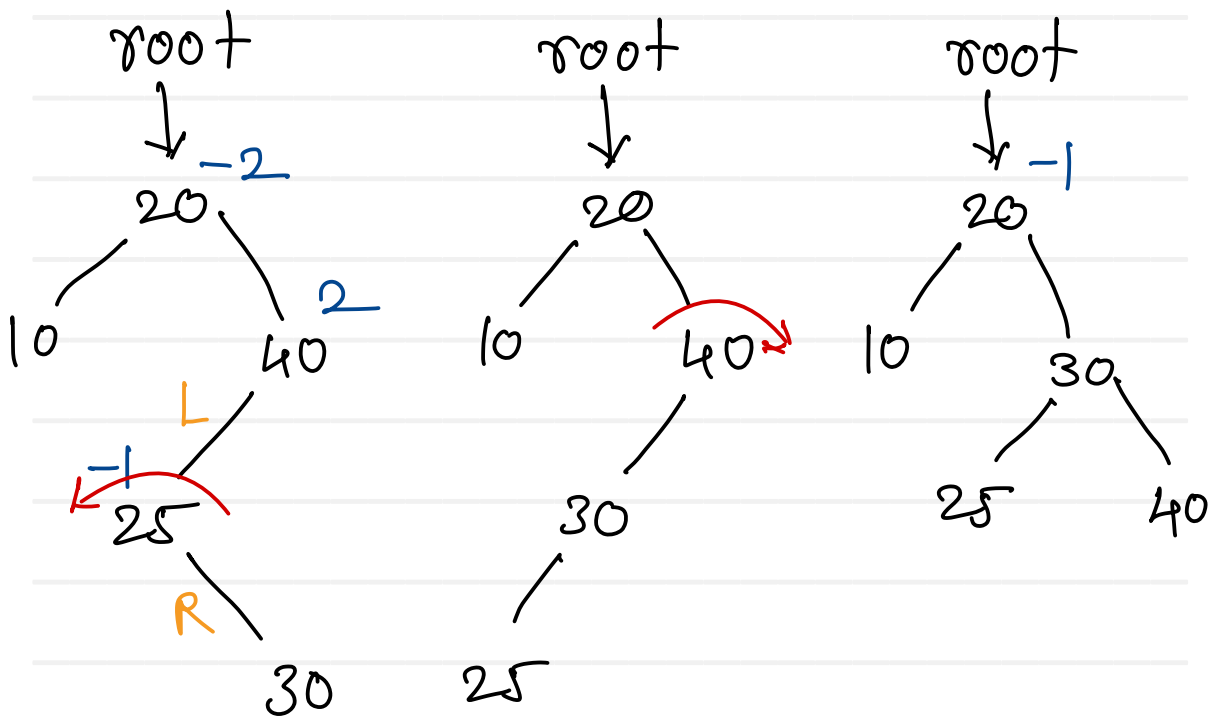
AVL Tree

- self balancing binary search tree
- on every insertion and deletion of a node, tree is getting balanced by applying rotations on imbalance nodes
- The difference bet heights of left and right sub trees can not be more than one for all nodes
- Balance factors of all the nodes are either -1 , 0 or +1
- All operations of AVL tree are performed in $O(\log n)$ time complexity

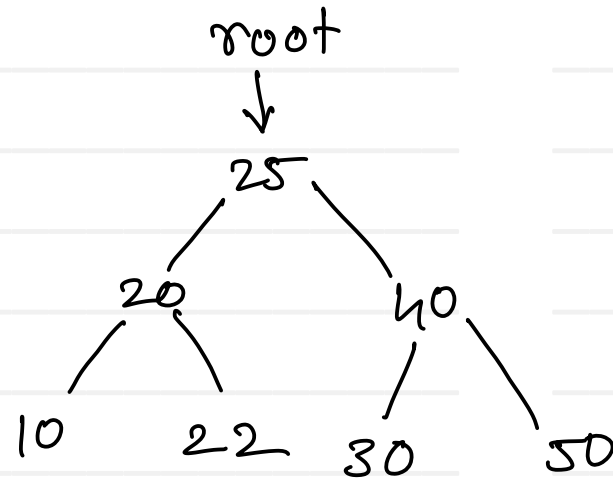
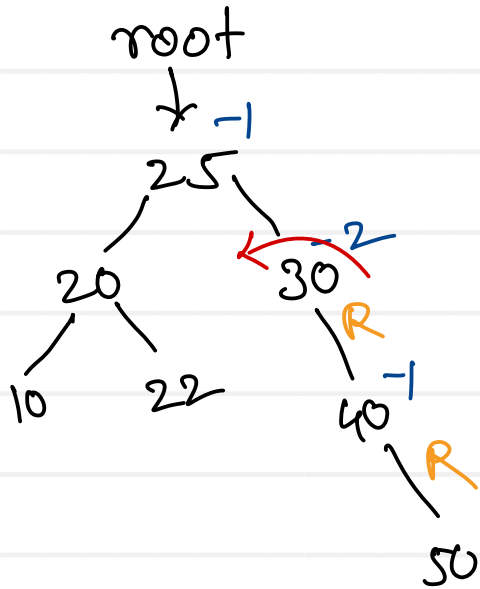
Keys : 40, 20, 10, 25, 30, 22, 50



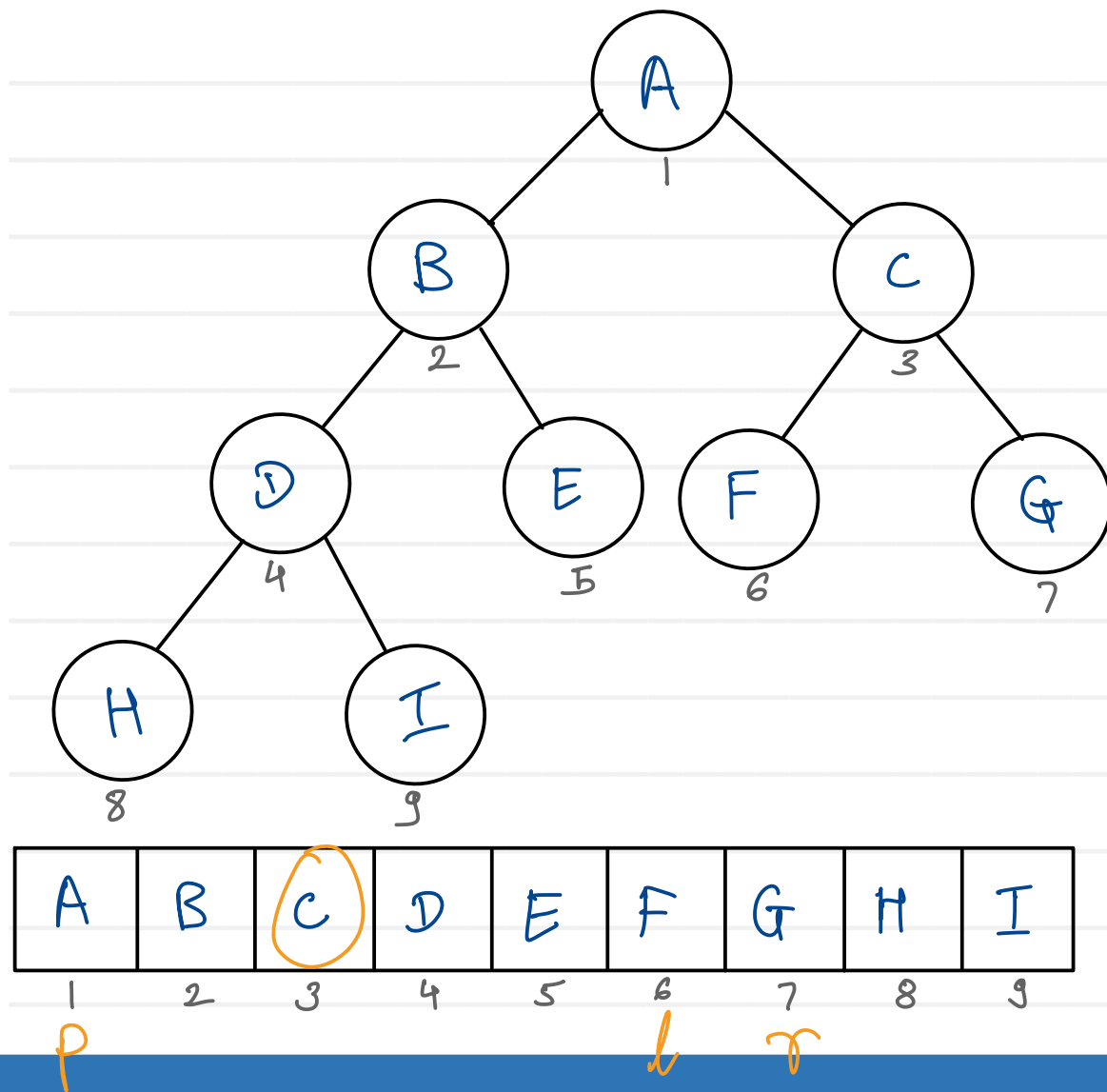
Keys : 40, 20, 10, 25, 30, 22, 50



Keys : 40, 20, 10, 25, 30, 22, 50

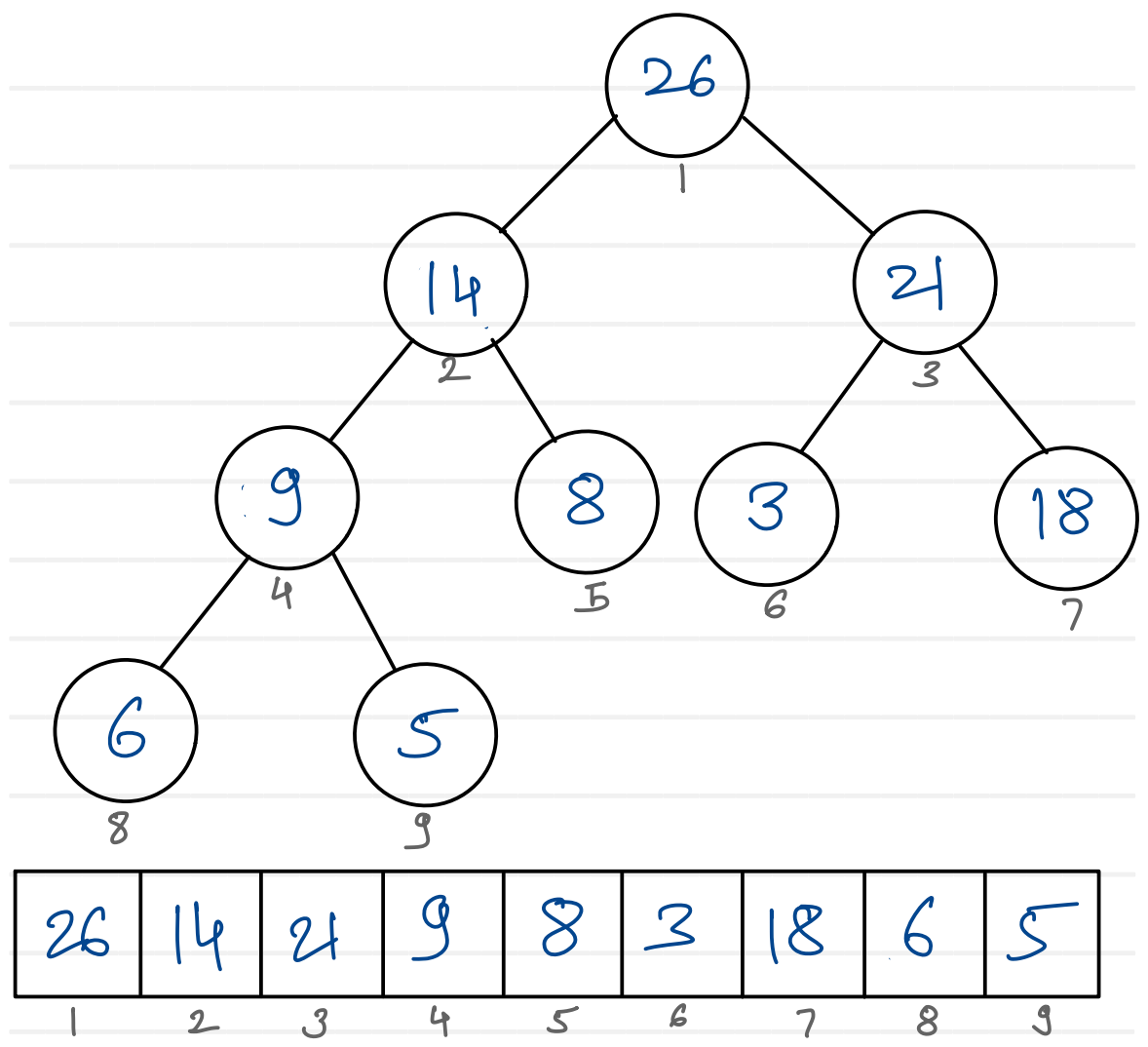


Almost Complete Binary Tree or Heap



- Almost Complete Binary Tree (height = h)
- All leaf nodes must be at level h or h-1
- All leaf nodes at level h must aligned as left as possible
- Array implementation of Almost Complete Binary Tree is called as heap

Node - i
 Parent - $i / 2$
 Left child - $i * 2$
 Right child - $i * 2 + 1$

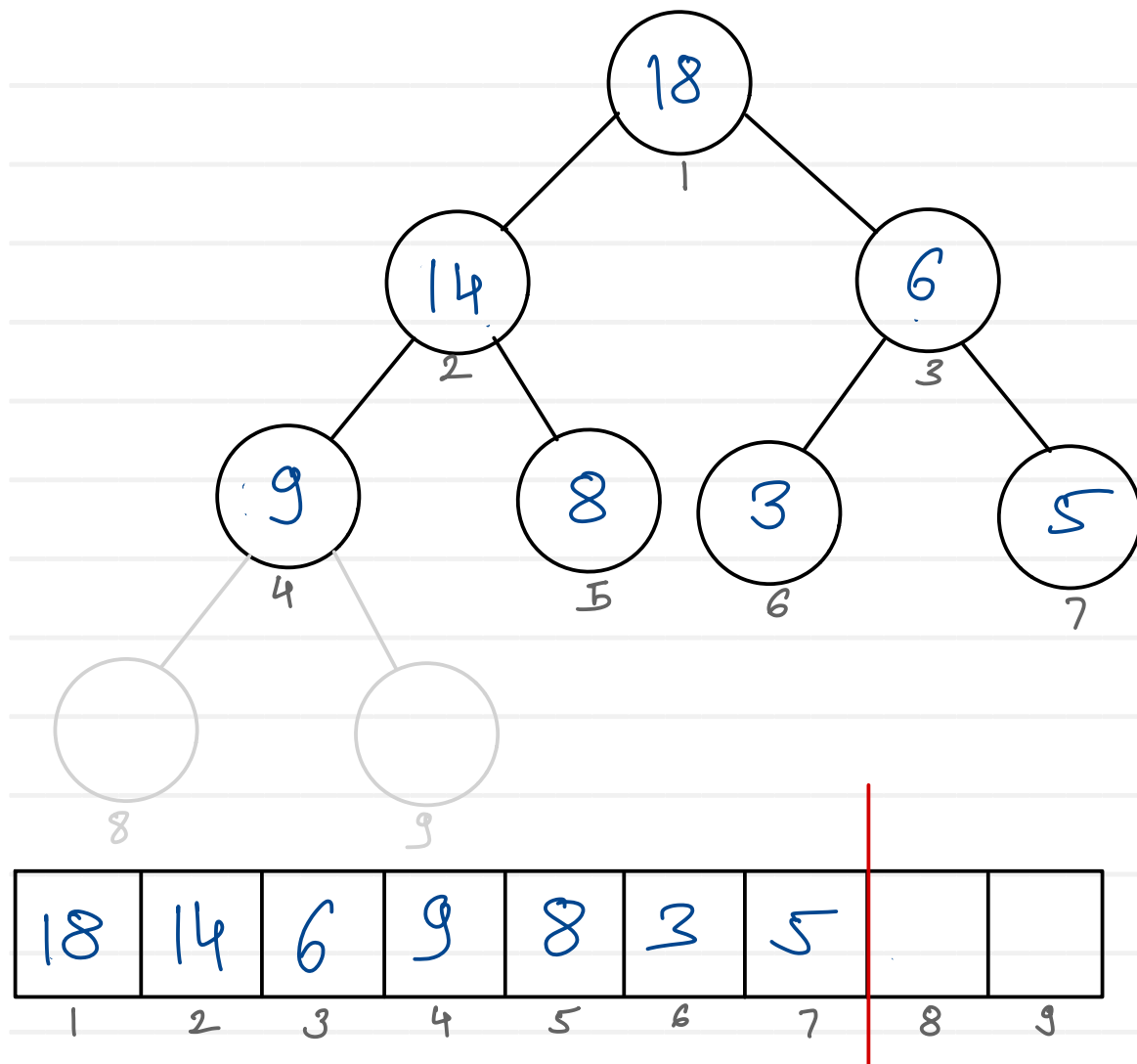


Keys : 6, 14, 3, 26, 8, 18, 21, 9, 5

1. Add new value at first index of array from left side
2. Adjust position of newly added value by comparing it with all its ancestors one by one.

$$T(n) = O(\log n)$$

Heap - Create heap (Delete)



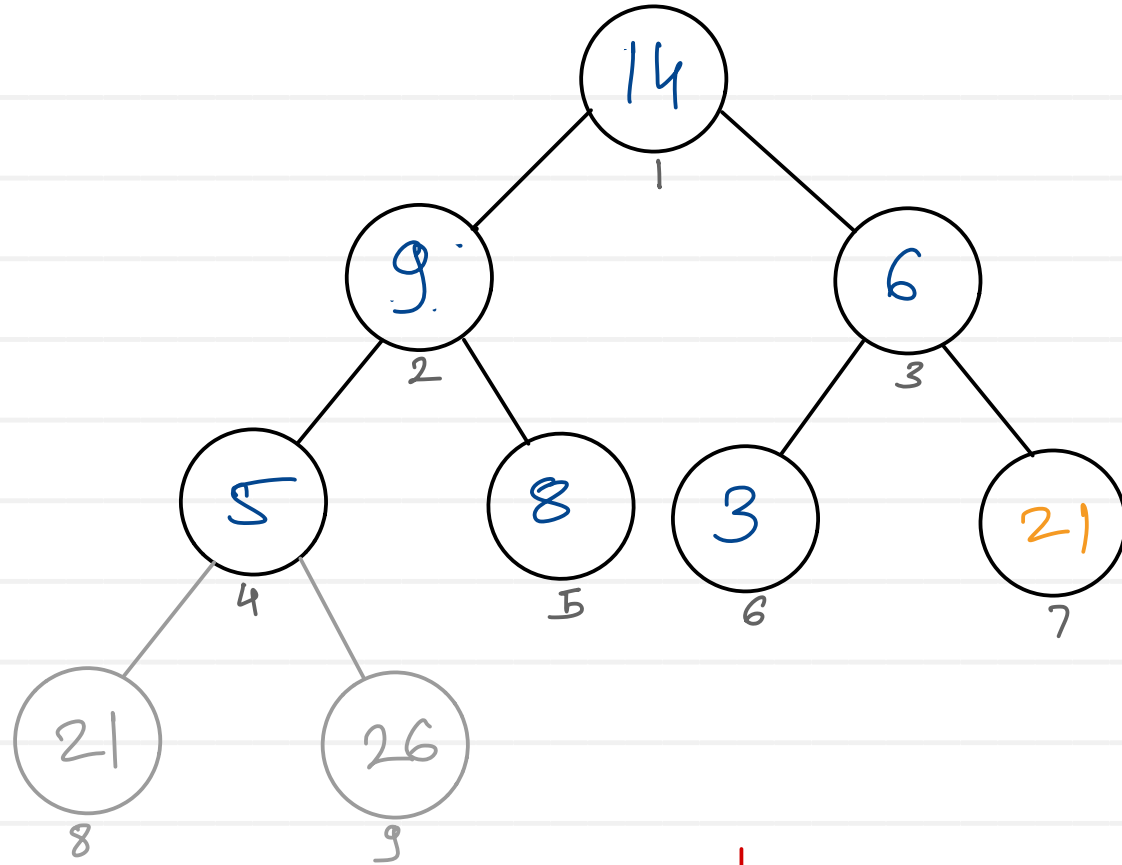
Max = 26

Max = 21

1. Place last element of heap at root position
2. Adjust position of root element by comparing it with all its descendents one by one

$$T(n) = O(\log n)$$

Heap sort



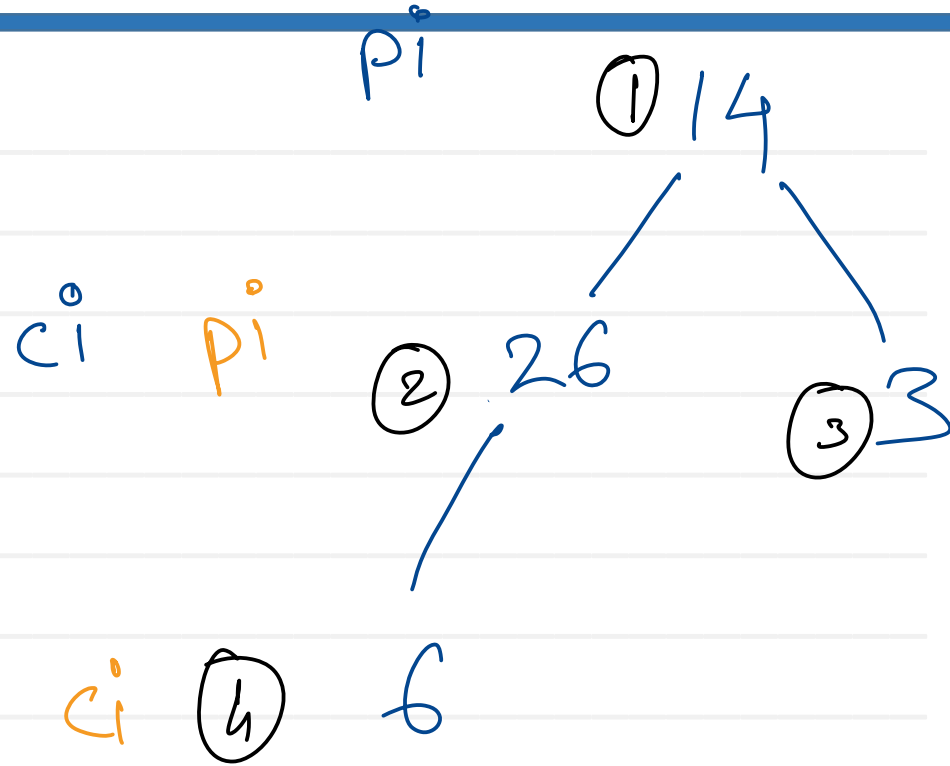
14	9	6	5	8	3	18	21	26
1	2	3	4	5	6	7	8	9

arr

6	14	3	26	8	18	21	9	5
1	2	3	4	5	6	7	8	9

1. make heap $\rightarrow n \log n$
 2. delete heap $\rightarrow n \log n$
- $2n \log n$

$$T(n) = O(n \log n)$$



c_i	p_i
4	2
2	1
1	0X



Thank you!!!

Devendra Dhande

devendra.dhande@sunbeaminfo.com



Sunbeam Institute of Information Technology
Pune and Karad
PG - DESD

Module – Data Structures

Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com



Sunbeam Infotech

www.sunbeaminfo.com

BST - Types

- **Skewed Binary tree**

- In binary tree if only left or right links are used, then tree grows in only one direction and such tree is called as skewed binary tree.
 - Left skewed binary tree
 - Right skewed binary tree
- Tree has maximum height that is same as number of elements
- Time complexity of searching is $O(n)$. (Like linked list)

- **Balanced BST**

- If nodes in BST are arranged so that its height is kept as less as possible, is called as balanced BST.
- Balance Factor = Height of left subtree – Height of right subtree
- In balanced BST, balance factor of each node is -1, 0 or +1
- A tree can be balanced by applying series of left or right rotations.

- **AVL Tree**

- AVL tree is a self balancing binary search tree.
- Node are balanced on each insert and delete operation.
- Difference between heights of left and right subtrees can not be more than 1 for all nodes.



Sunbeam Infotech

www.sunbeaminfo.com

BST - Types

• AVL Tree

- AVL tree is a self balancing binary search tree.
- Node are rebalanced on each insert and delete operation.
- Difference between heights of left and right subtrees can not be more than 1 for all nodes.
- Most of BST operations are done in $O(h)$ i.e. $O(\log n)$ time.
- Need more number of rotations as compared to Red & Black tree.
- Construct an AVL Tree by inserting nodes : 40, 20, 10, 25, 30, 22, 50



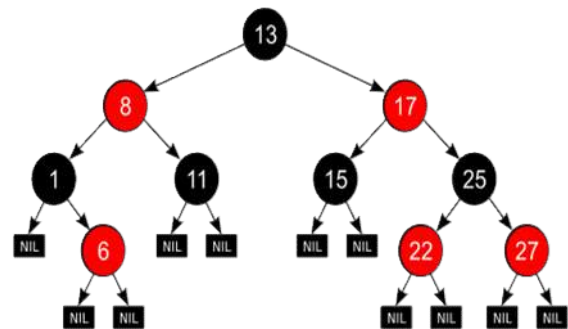
Sunbeam Infotech

www.sunbeaminfo.com

BST - Types

• Red and Black Tree

- Red and Black tree is a self balancing binary search tree.
- Each node follows some rules:
 - Every node has a color either red or black.
 - Root of tree is always black.
 - Two adjacent cannot be red nodes (Parent color should be different than child).
 - Every path from a node (including root) to any of its descendant NULL node has the equal number of black nodes.
- Most of BST operations are done in $O(h)$ i.e. $O(\log n)$ time.
- For frequent insert/delete, RB tree is preferred over AVL tree.



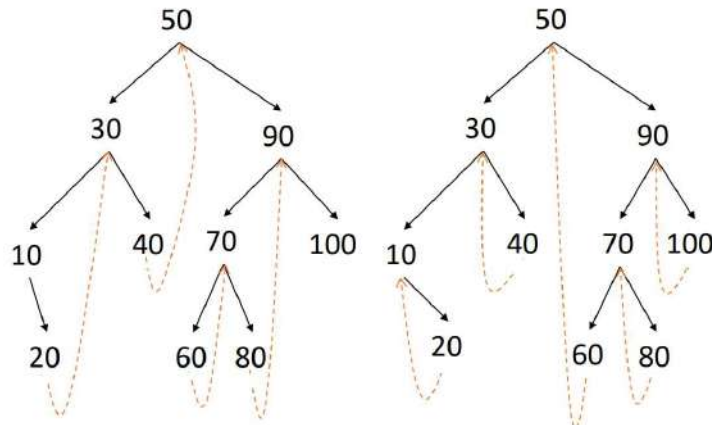
Sunbeam Infotech

www.sunbeaminfo.com

BST - Types

• Threaded Tree

- Typical BST in-order traversal involves recursion or stack. It slows execution and also need more space.
- Threaded BST keep address of in-order successor or predecessor addresses instead of NULL to speed up in-order traversal (using a loop).
 - Left threaded BST
 - Right threaded BST
 - In-threaded BST



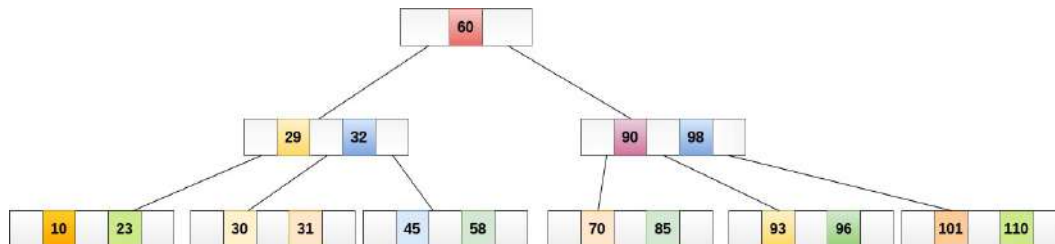
Sunbeam Infotech

www.sunbeaminfo.com

Tree Types

• B Tree (Tree with degree m/m-way tree)

- A B-Tree of order m can have at most m-1 keys and m children.
- B tree store large number of keys in a single node. This allows storing number of values keeping height minimal.
- Note that in B-Tree all leaf nodes are at same level.
- B-Tree is commonly used for indexing into file systems and databases. It ensures quick data searching and speed up disk access.



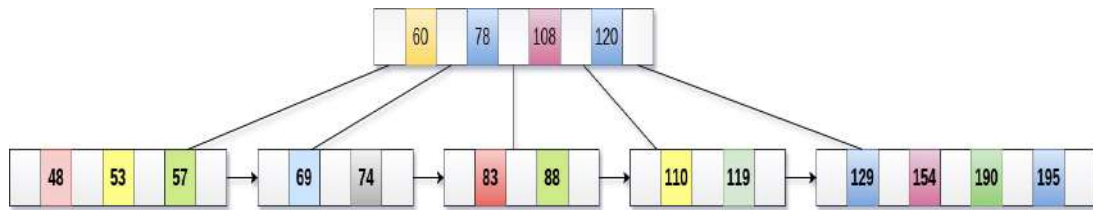
Sunbeam Infotech

www.sunbeaminfo.com

Tree Types

• B+ Tree

- Extension of B-Tree for efficient insert, delete and search operation.
- Data is stored in leaf nodes only and all leaf nodes are linked together for sequential access.
- Faster searching, simplified deletion (as only from leaf nodes).
- B+ Tree is commonly used for indexing into file systems and databases. It ensures quick data searching and speed up disk access.



Sunbeam Infotech

www.sunbeaminfo.com

Tree : Types

• Strictly Binary Tree

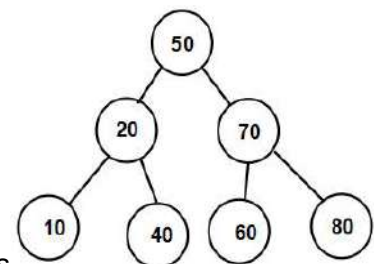
- Binary tree in which each non leaf node has exact two child nodes.

• Full Binary Tree

- Binary tree with its full capacity for the given height.
- In other words, adding one more node will increase height of the tree.
- It is always complete as well as strictly binary tree.
- Number of elements = $2^h - 1$

• Complete Binary Tree

- The binary tree which follows two conditions
 - All leaf nodes are at level h or $h-1$.
 - All leaf nodes at last level (h) are aligned to left as much as possible.

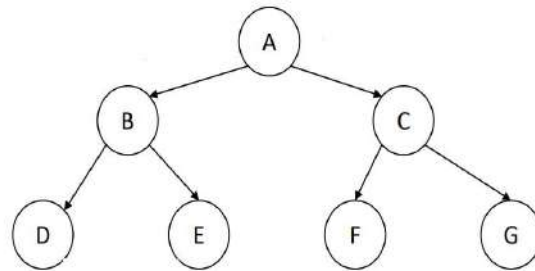


Sunbeam Infotech

www.sunbeaminfo.com

Heap

- Heap is array implementation of complete binary tree.
- Parent child relation is maintained through index calculations
- If a node is at index - i
- Its left child at index = $2 * i$
- Its right child at index = $2 * i + 1$
- Its parent at index = $i / 2$



A	B	C	D	E	F	G
1	2	3	4	5	6	7

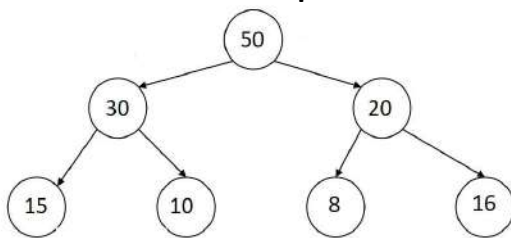


Sunbeam Infotech

www.sunbeaminfo.com

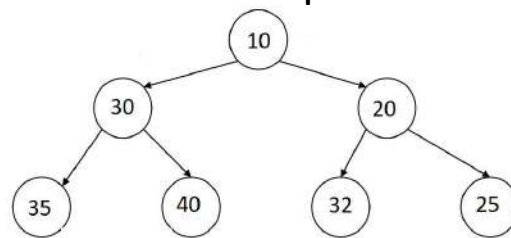
Heap Types – Max and Min

Max Heap



50	30	20	15	10	8	16
1	2	3	4	5	6	7

Min Heap



10	30	20	35	40	32	25
1	2	3	4	5	6	7

- Max heap is a heap data structure in which each node is greater than both of its child nodes.
- Min heap is a heap data structure in which each node is smaller than both of its child nodes.



Sunbeam Infotech

www.sunbeaminfo.com

Heap Sort

- Heap sort is 2 step process
 - Create heap
 - Delete heap

10	20	15	30	40
1	2	3	4	5

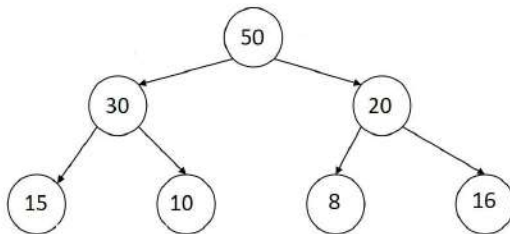


Sunbeam Infotech

www.sunbeaminfo.com

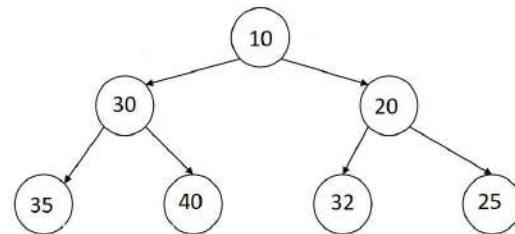
Priority Queues

**Higher number
Higher priority**



- In Max heap always root element which has highest value is removed

**Lower number
Higher priority**



- In Min heap always root element which has lowest value is removed



Sunbeam Infotech

www.sunbeaminfo.com



Thank you!

Devendra Dhande

<devendra.dhande@sunbeaminfo.com>



Sunbeam Infotech

www.sunbeaminfo.com



Sunbeam Institute of Information Technology

Pune and Karad

Module – Data Structures and Algorithms

Trainer - Devendra Dhande

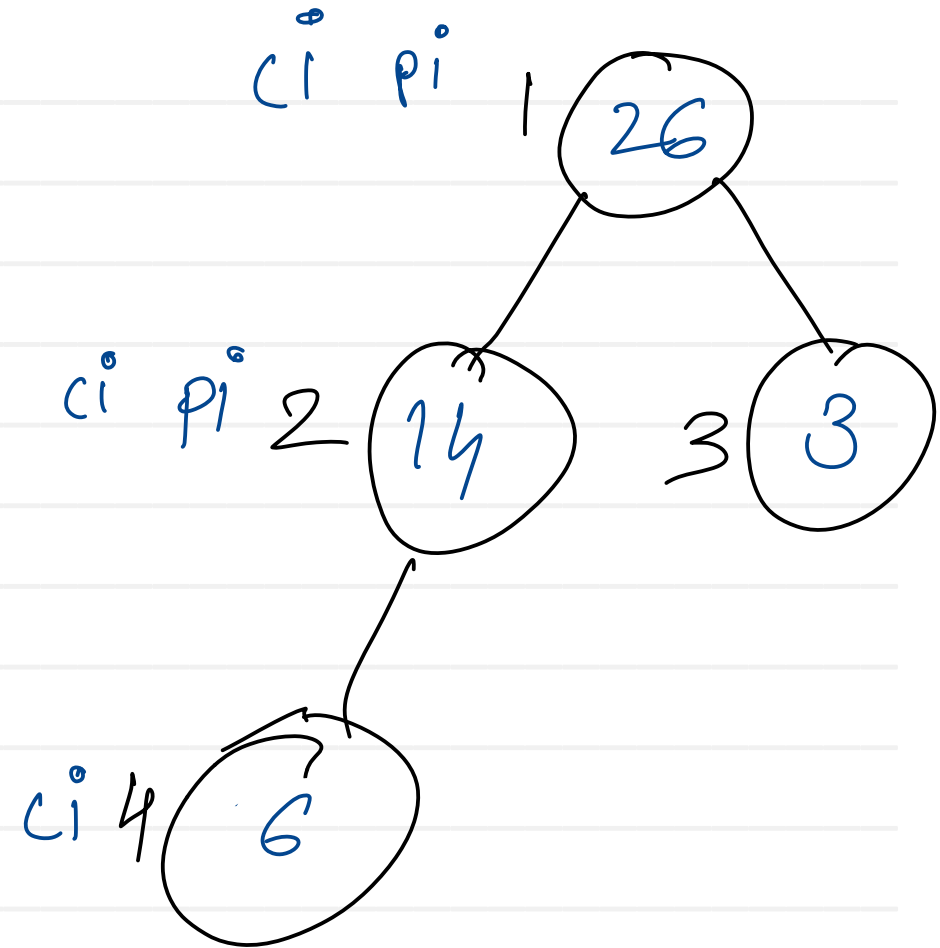
Email – devendra.dhande@sunbeaminfo.com

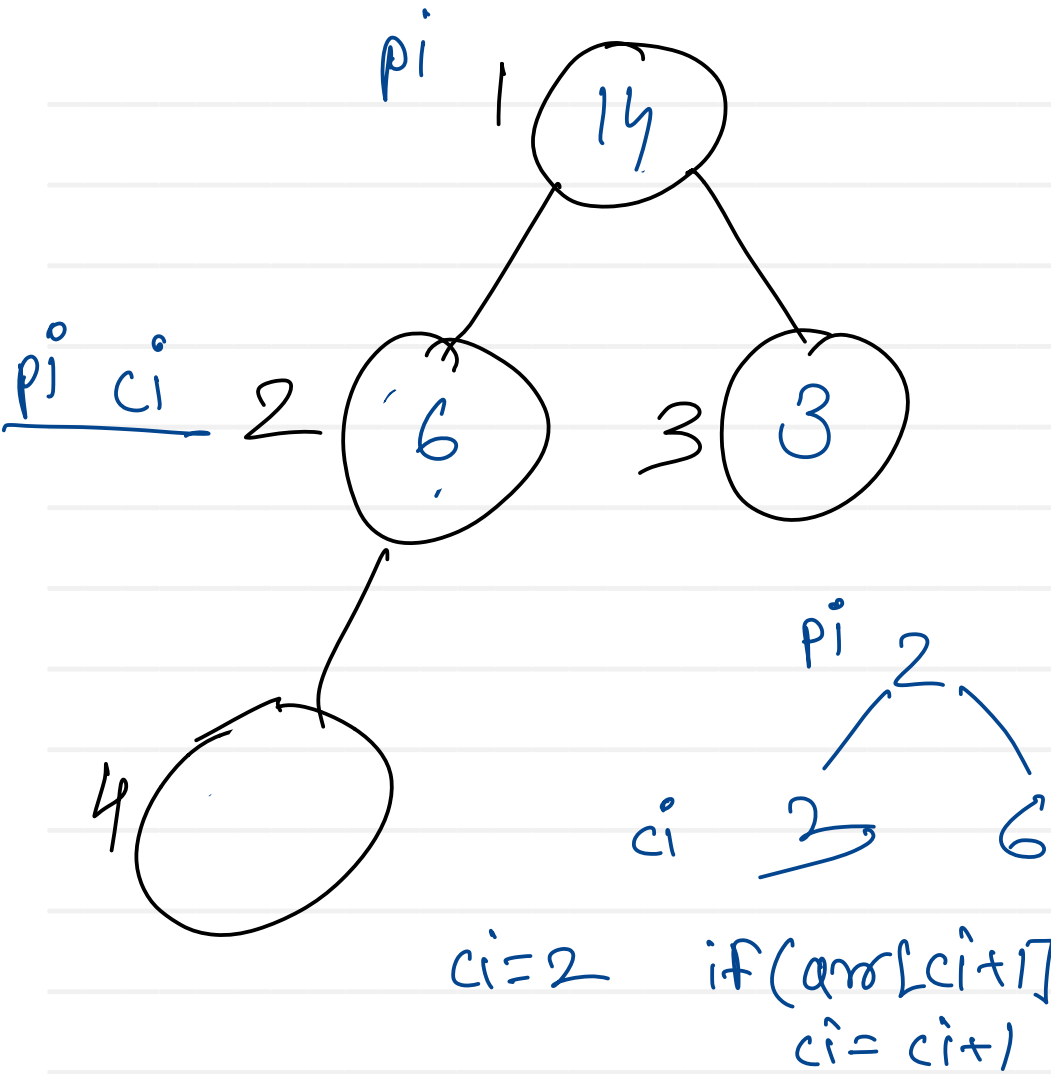
Heap - Add

$p_i = 0$

```

size = 0
add(value) {
    size++;
    arr[size] = value;
    ci = size;
    pi = ci / 2;
    while (pi >= 1) {
        if (arr[pi] > arr[ci])
            break;
        int temp = arr[pi];
        arr[pi] = arr[ci];
        arr[ci] = temp;
        ci = pi;
        pi = ci / 2;
    }
}
    
```



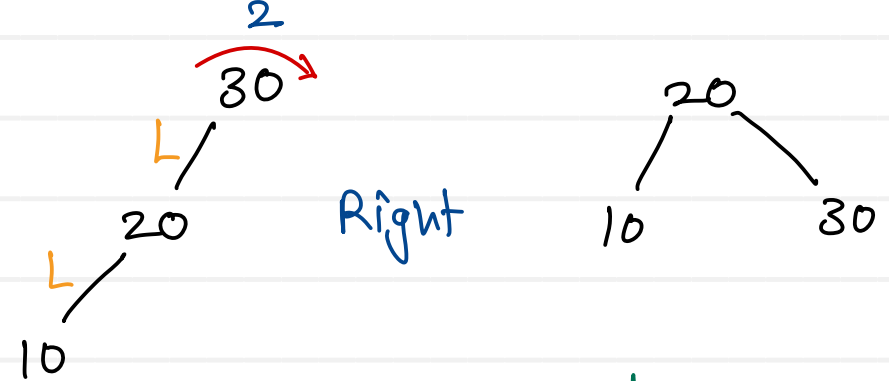


```

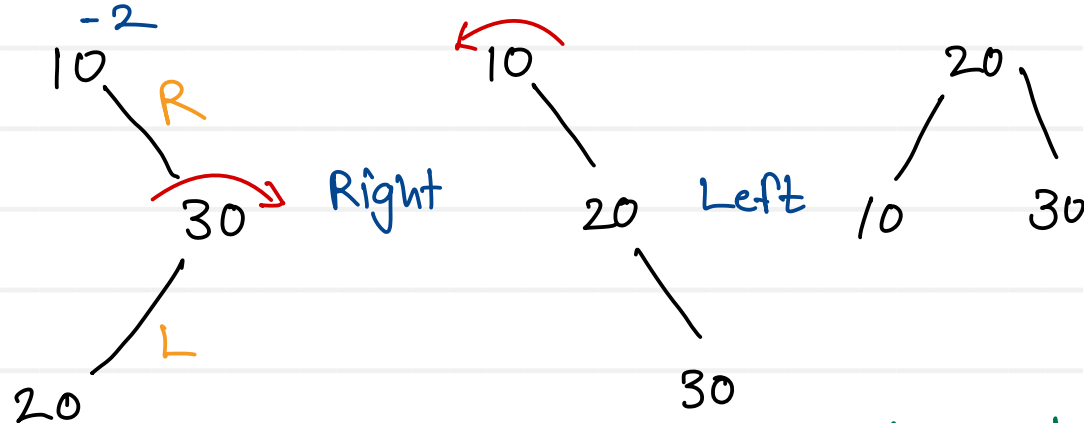
delete( ) {
    arr[size] = arr[size];
    size--;
    pi = 1;
    ci = pi * 2;
    while( ci <= size ) {
        if( (ci+1) <= size && arr[ci+1] > arr[ci] )
            ci = ci+1;
        if( arr[pi] > arr[ci] )
            break;
        int temp = arr[pi];
        arr[pi] = arr[ci];
        arr[ci] = temp;
        pi = ci;
        ci = pi * 2;
    }
}
    
```



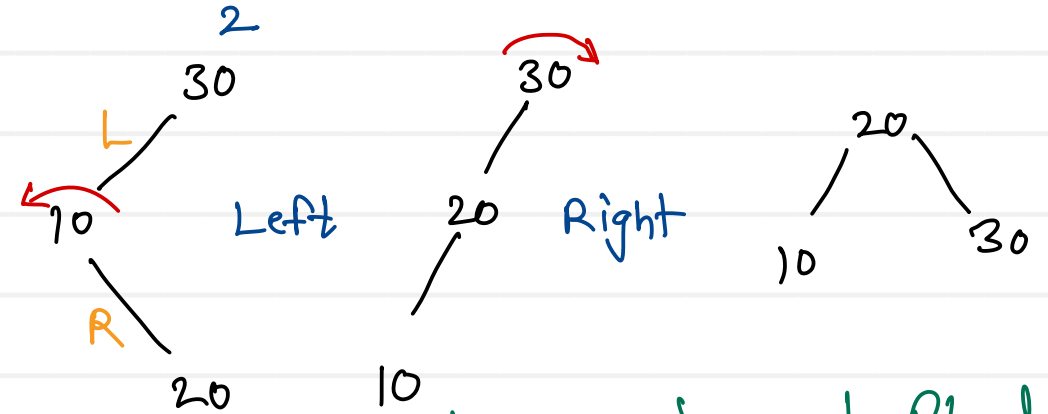
$bf < -1$ & $value > trav.right.data$



$bf > 1$ & $value < trav.left.data$



$bf < -1$ & $value < trav.right.data$



$bf > 1$ & $value > trav.left.data$

Graph : Terminologies

- **Graph** is a non linear data structure having set of vertices (nodes) and set of edges (arcs).

- $G = \{V, E\}$

Where V is a set of vertices and E is a set of edges

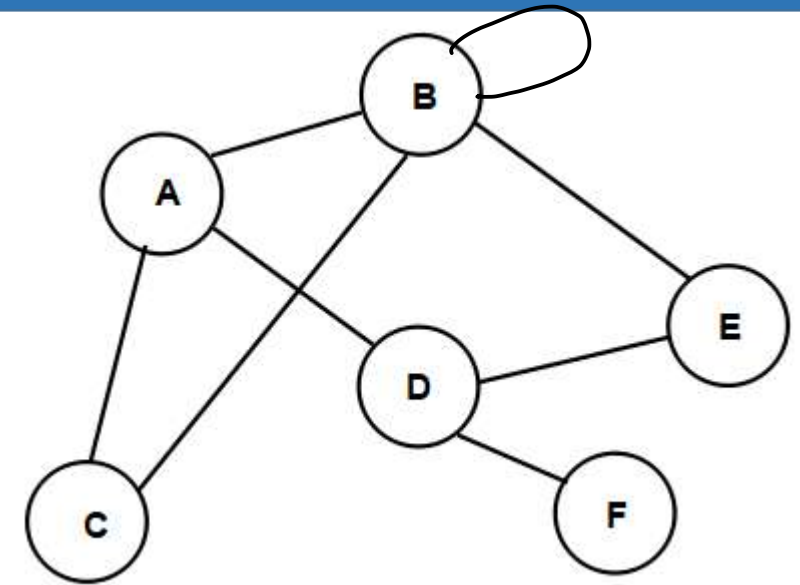
- **Vertex (node)** is an element in the graph

$$V = \{A, B, C, D, E, F\}$$

- **Edge (arc)** is a line connecting two vertices

$$E = \{(A,B), (A,C), (B,C), (B,E), (D, E), (D,F), (A,D)\}$$

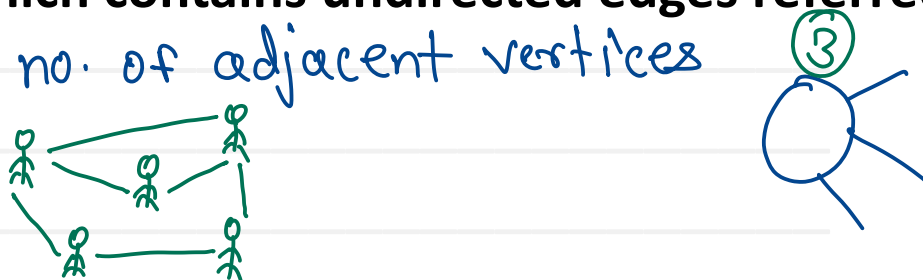
- Vertex A is set be **adjacent** to B, if and only if there is an edge from A to B.
- **Degree of vertex** :- Number of vertices adjacent to given vertex
- **Path** :- Set of edges connecting any two vertices is called as path between those two vertices.
 - Path between A to D = $\{(A, B), (B, E), (E, D)\}$
- **Cycle** :- Set of edges connecting to a node itself is called as cycle.
 - $\{(A, B), (B, E), (E, D), (D, A)\}$
- **Loop** :- An edge connecting a node to itself is called as loop. Loop is smallest cycle.



- **Undirected graph.**

- If we can represent any edge either (u,v) OR (v,u) then it is referred as **unordered pair of vertices** i.e. **undirected edge**.
- **graph which contains undirected edges referred as undirected graph.**

degree : no. of adjacent vertices



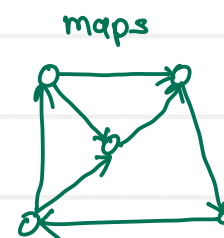
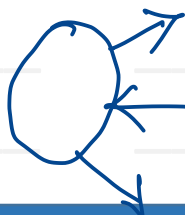
$$(u, v) == (v, u)$$

- **Directed Graph (Di-graph)**

- If we cannot represent any edge either (u,v) OR (v,u) then it is referred as an **ordered pair of vertices** i.e. **directed edge**.
- **graph which contains set of directed edges referred as directed graph (di-graph).**
- graph in which each edge has some direction

in degree : no. of in coming edges ①

out degree : no. of out going edges ②

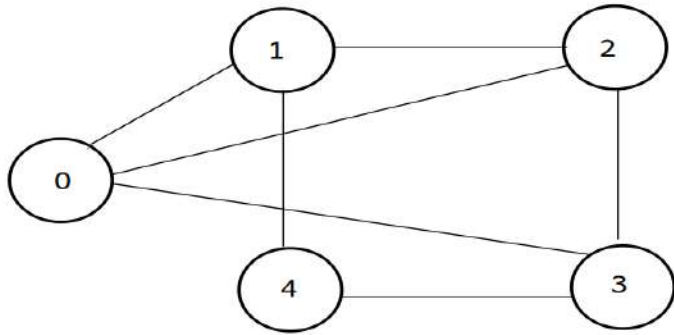


$$(u, v) \neq (v, u)$$

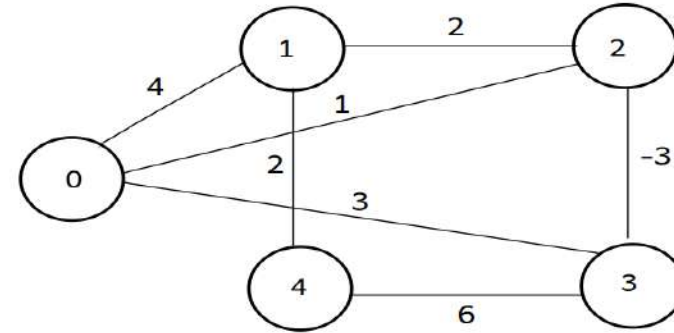
Graph : Types

- **Weighted Graph**

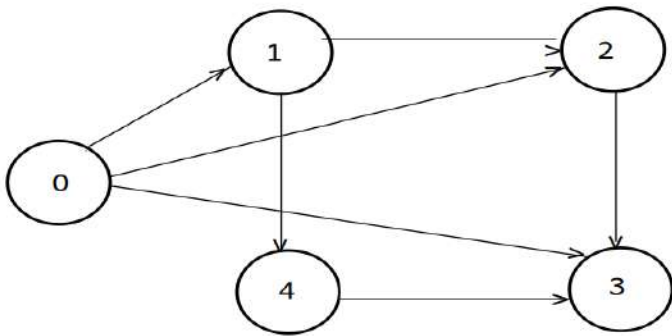
- A graph in which edge is associated with a number (ie weight)



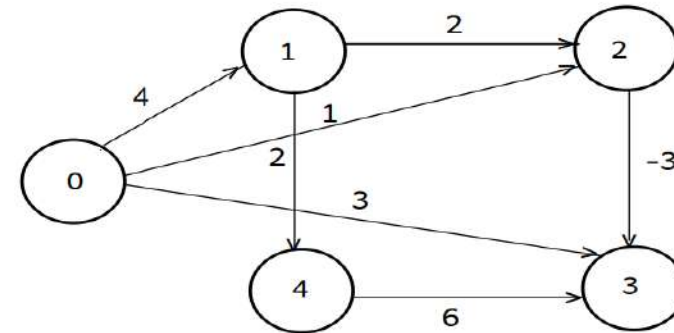
undirected unweighted graph



undirected weighted graph



directed unweighted graph



directed weighted graph

- **Simple Graph**

- Graph not having multiple edges between adjacent nodes and no loops.

- **Complete Graph**

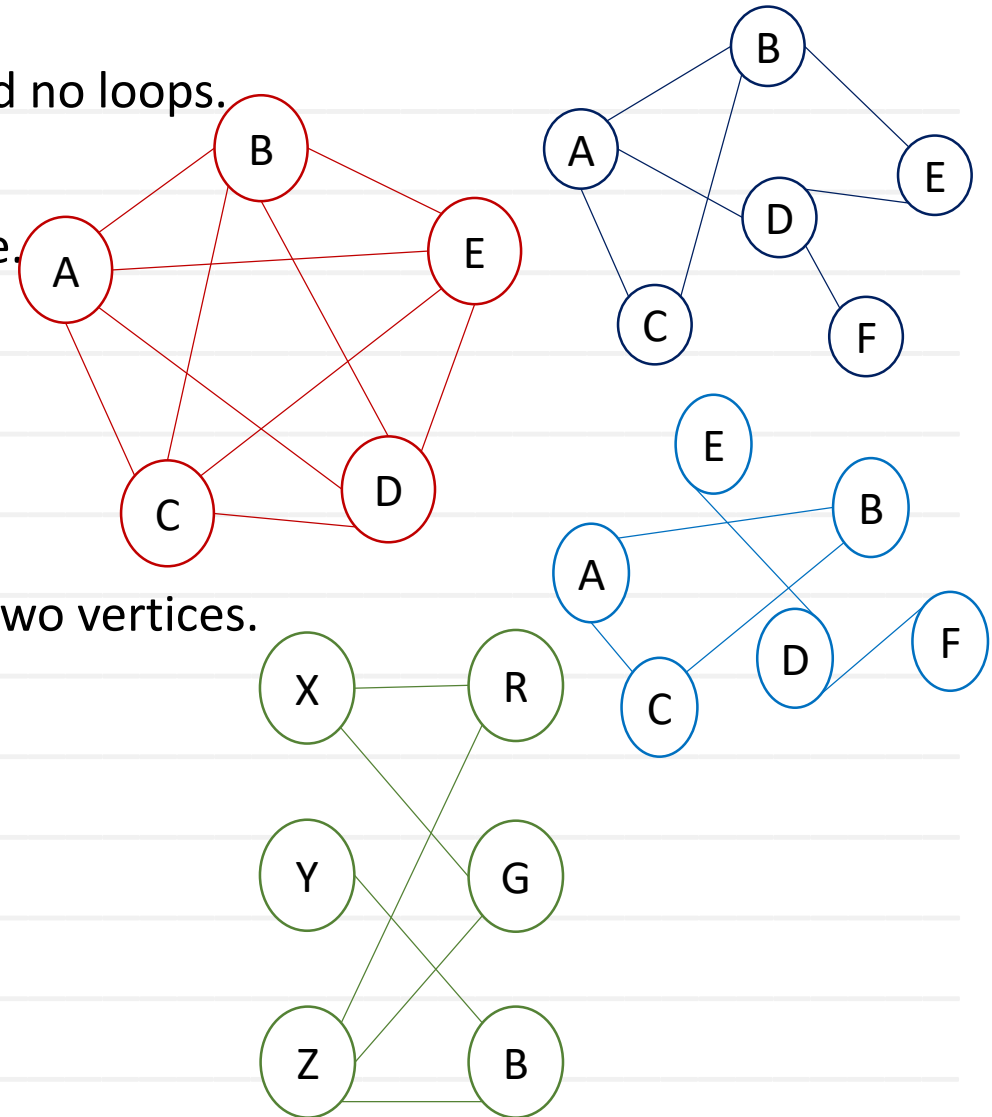
- Simple graph in which node is adjacent with every other node.
- Un-Directed graph: Number of Edges = $\frac{n(n-1)}{2}$
where, n – number of vertices
- Directed graph: Number of edges = $n(n-1)$

- **Connected Graph**

- Simple graph in which there is some path exist between any two vertices.
- Can traverse the entire graph starting from any vertex.

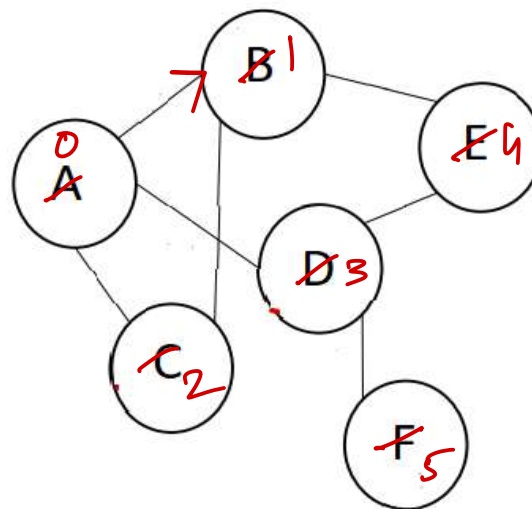
- **Bi-partite graph**

- Vertices can be divided in two disjoint sets.
- Vertices in first set are connected to vertices in second set.
- Vertices in a set are not directly connected to each other.

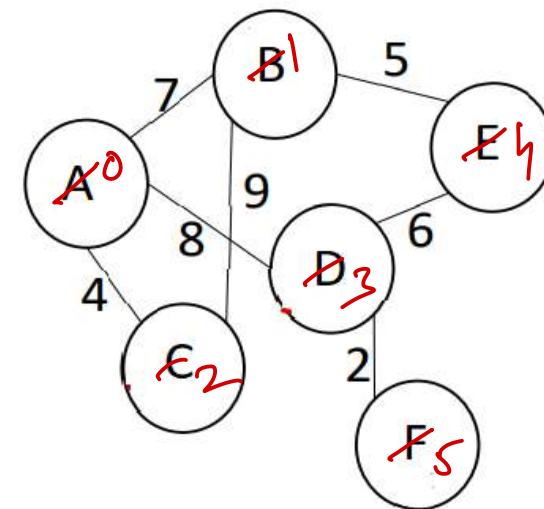


Graph Implementation – Adjacency Matrix

- If graph have V vertices, a V x V matrix can be formed to store edges of the graph.
- Each matrix element represent presence or absence of the edge between vertices.
- For non-weighted graph, 1 indicate edge and 0 indicate no edge.
- For weighted graph, weight value indicate the edge and infinity sign ∞ represent no edge.
- For un-directed graph, adjacency matrix is always symmetric across the diagonal.
- Space complexity of this implementation is $O(V^2)$.



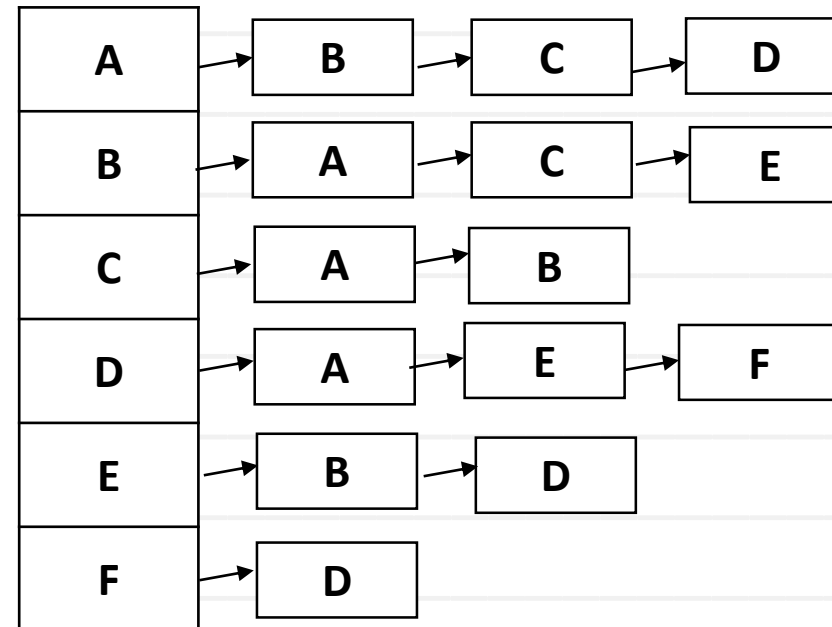
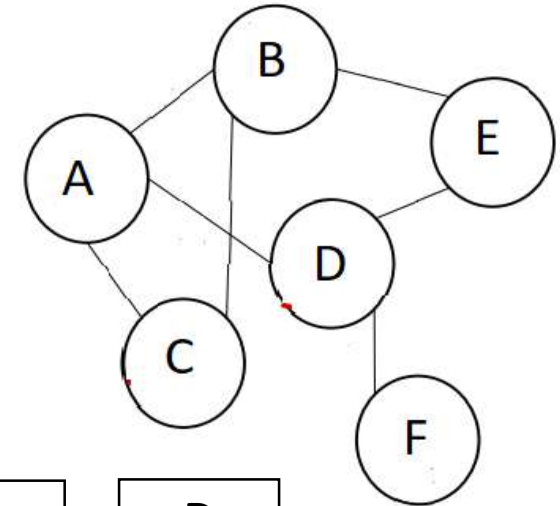
	A	B	C	D	E	F
A	0	1	1	1	0	0
B	1	0	1	0	1	0
C	1	1	0	0	0	0
D	1	0	0	0	1	1
E	0	1	0	1	0	0
F	0	0	0	1	0	0

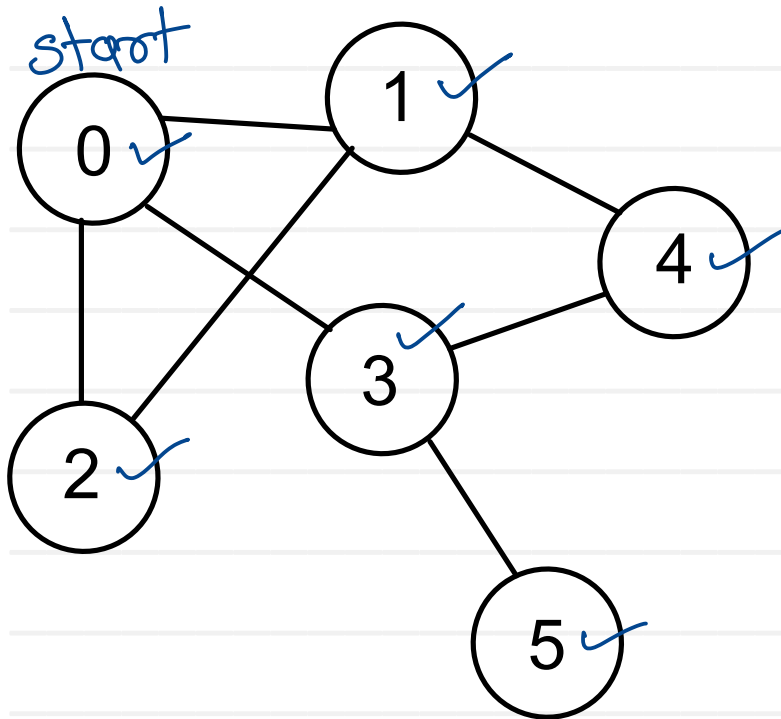


	A	B	C	D	E	F
A	∞	7	4	8	∞	∞
B	7	∞	9	∞	5	∞
C	4	9	∞	∞	∞	∞
D	8	∞	∞	∞	6	2
E	∞	5	∞	6	∞	∞
F	∞	∞	∞	2	∞	∞

Graph Implementation – Adjacency List

- Each vertex holds list of its adjacent vertices.
- For non-weighted graphs only, neighbor vertices are stored.
- For weighted graph, neighbor vertices and weights of connecting edges are stored.
- Space complexity of this implementation is $O(V+E)$.
- If graph is sparse graph (with fewer number of edges), this implementation is more efficient (as compared to adjacency matrix method).



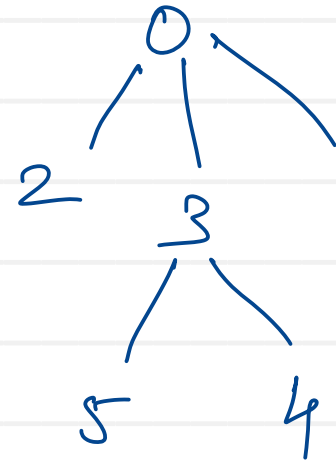


stack

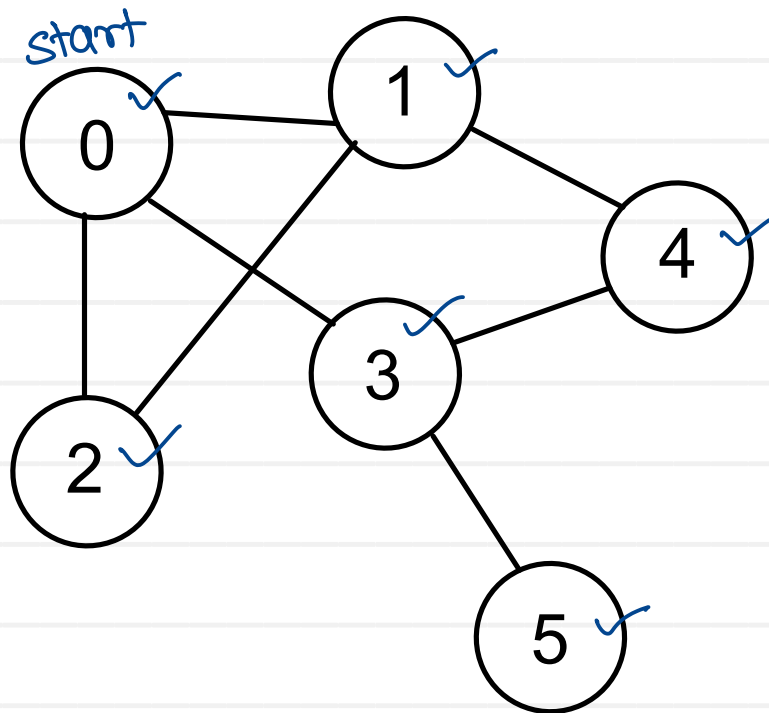
5
4
3
2
1
0

Traversal : 0, 3, 5, 4, 2, 1

1. Choose a vertex as start vertex.
2. Push start vertex on stack & mark it.
3. Pop vertex from stack.
4. Print the vertex.
5. Put all non-visited neighbours of the vertex on the stack and mark them.
6. Repeat 3-5 until stack is empty.



BFS Traversal

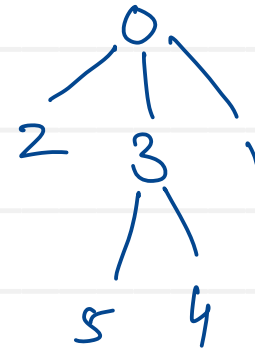


Queue

5
4
3
2
1
0

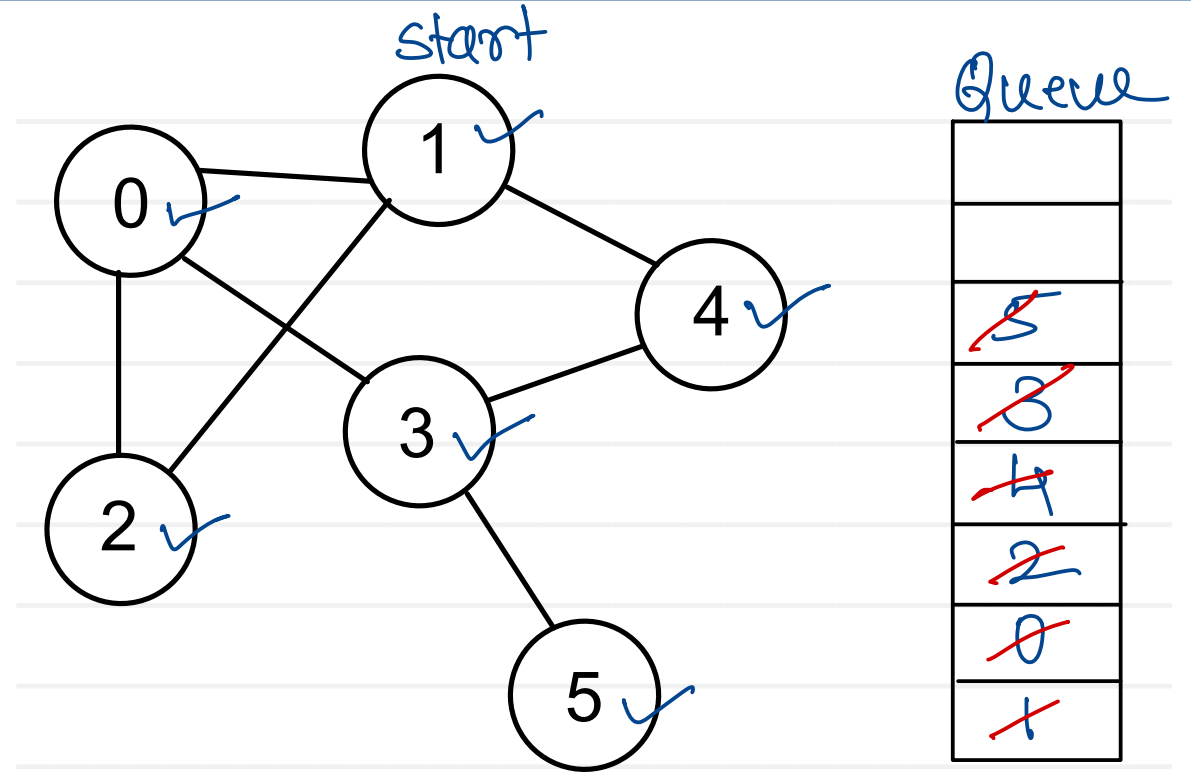
Traversal : 0, 1, 2, 3, 4, 5

1. Choose a vertex as start vertex.
2. Push start vertex on queue & mark it
3. Pop vertex from queue.
4. Print the vertex.
5. Put all non-visited neighbours of the vertex on the queue and mark them.
6. Repeat 3-5 until queue is empty.



Single Source Path Length

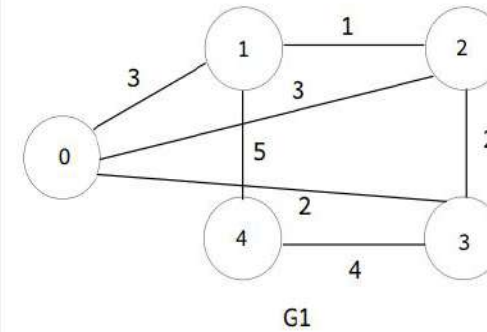
1. Create path length array to keep length of vertex from start vertex.
2. push start on queue & mark it. *length[start] = 0*
3. pop the vertex.
4. push all its non-marked neighbors on the queue, mark them.
5. For each such vertex calculate length as $\text{length}[\text{neighbor}] = \text{length}[\text{current}] + 1$
6. print current vertex to that neighbor vertex edge.
7. repeat steps 3-6 until queue is empty.
8. Print path length array.



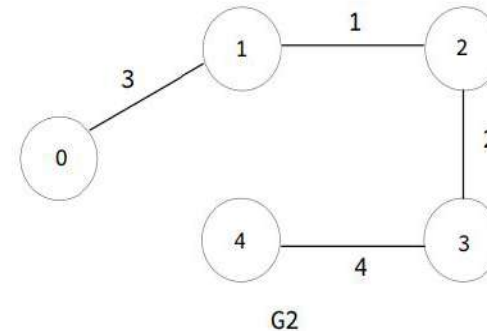
Path length tree : (1-0) (1-2) (1-4) (0-3) (3-5)

Spanning Tree

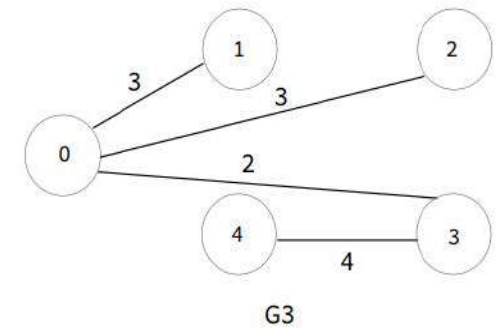
- Tree is a graph without cycles. Includes all V vertices and $V-1$ edges.
- Spanning tree is connected sub-graph of the given graph that contains all the vertices and sub-set of edges.
- Spanning tree can be created by removing few edges from the graph which are causing cycles to form.
- One graph can have multiple different spanning trees.
- In weighted graph, spanning tree can be made who has minimum weight (sum of weights of edges). Such spanning tree is called as Minimum Spanning Tree.
- Spanning tree can be made by various algorithms.
 - BFS Spanning tree
 - DFS Spanning tree
 - Prim's MST
 - Kruskal's MST



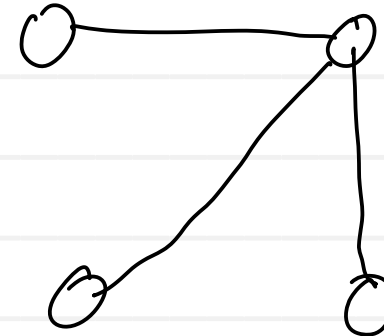
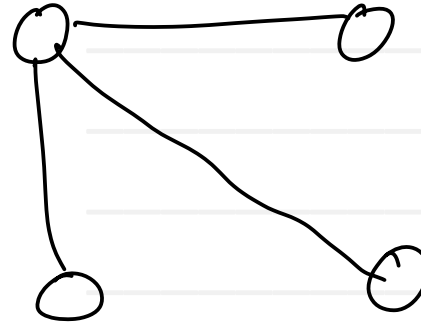
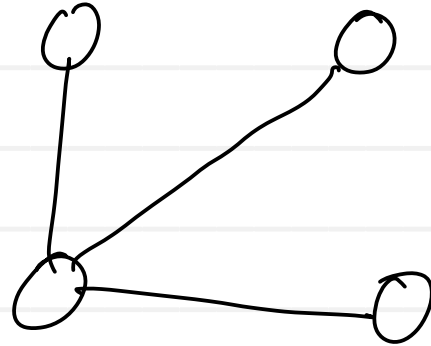
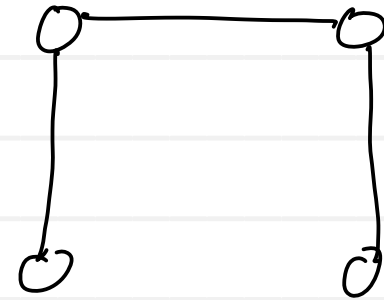
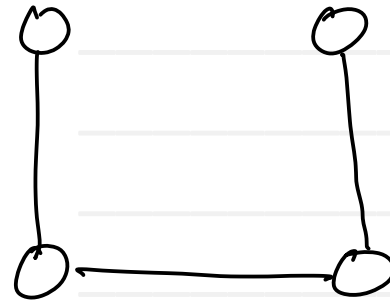
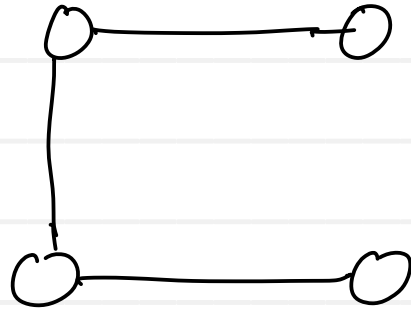
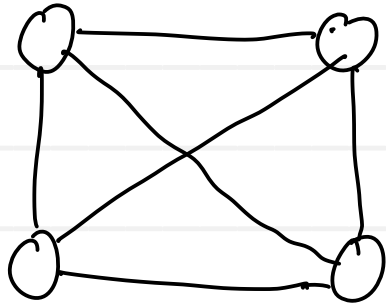
Weight of a graph G1 = 20



Weight of a graph G2 = 10

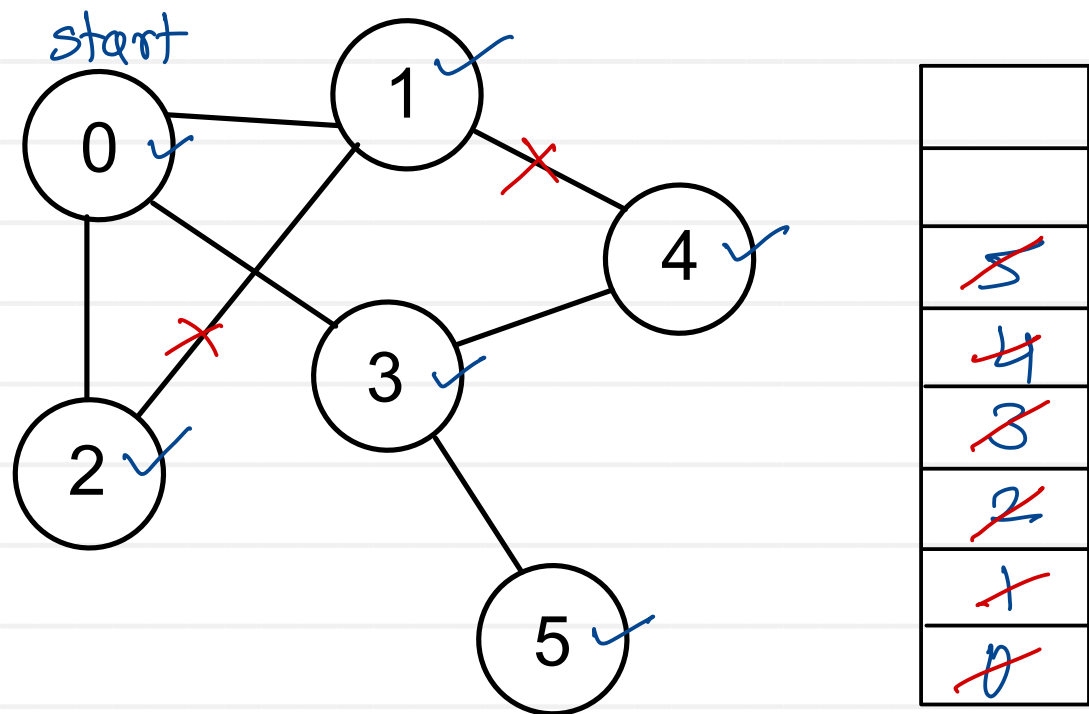


Weight of a graph G2 = 12



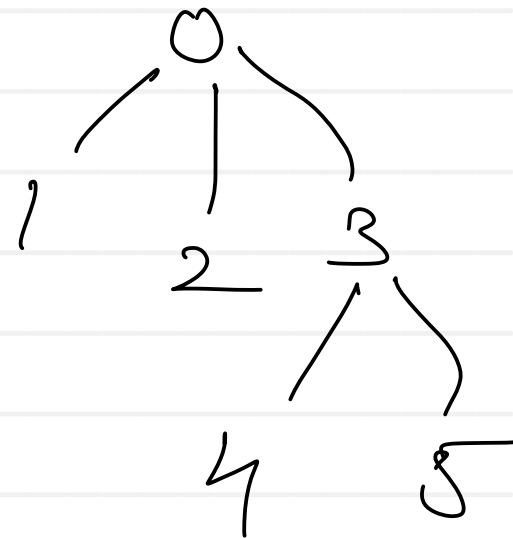
Spanning Tree

DFS Spanning Tree

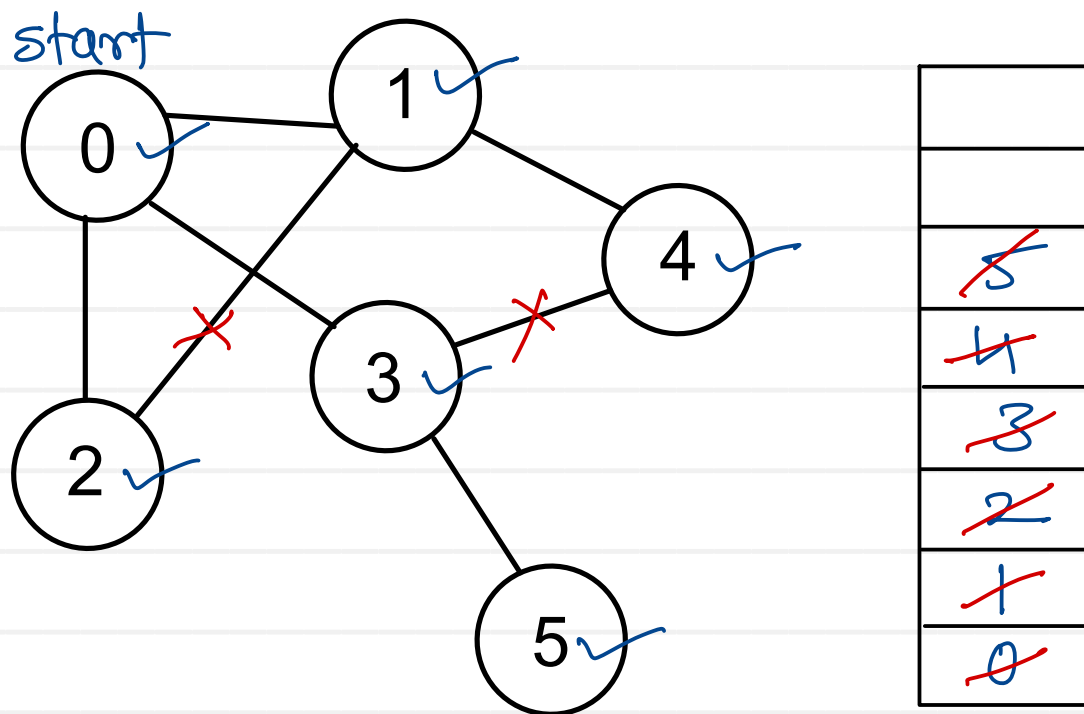


Spanning tree : (0-1), (0-2), (0-3), (3-4), (3-5)

1. push starting vertex on stack & mark it.
2. pop the vertex.
3. push all its non-marked neighbors on the stack, mark them and also print the vertex to neighboring vertex edges.
4. repeat steps 2-3 until stack is empty.

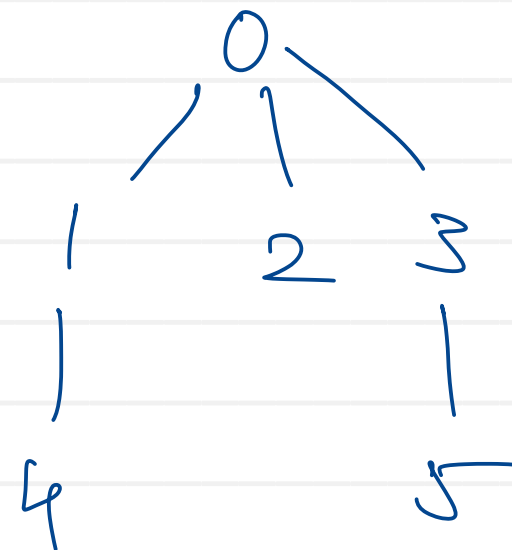


BFS Spanning Tree



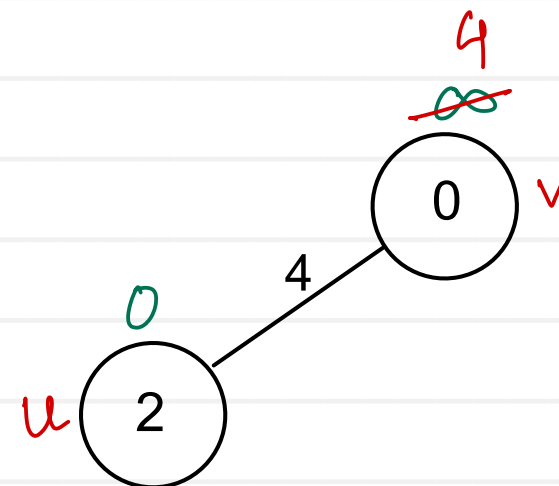
1. push starting vertex on queue & mark it.
2. pop the vertex.
3. push all its non-marked neighbors on the queue, mark them and also print the vertex to neighboring vertex edges.
4. repeat steps 2-3 until queue is empty.

Spanning tree : (0-1) (0-2) (0-3) (1-4) (3-5)



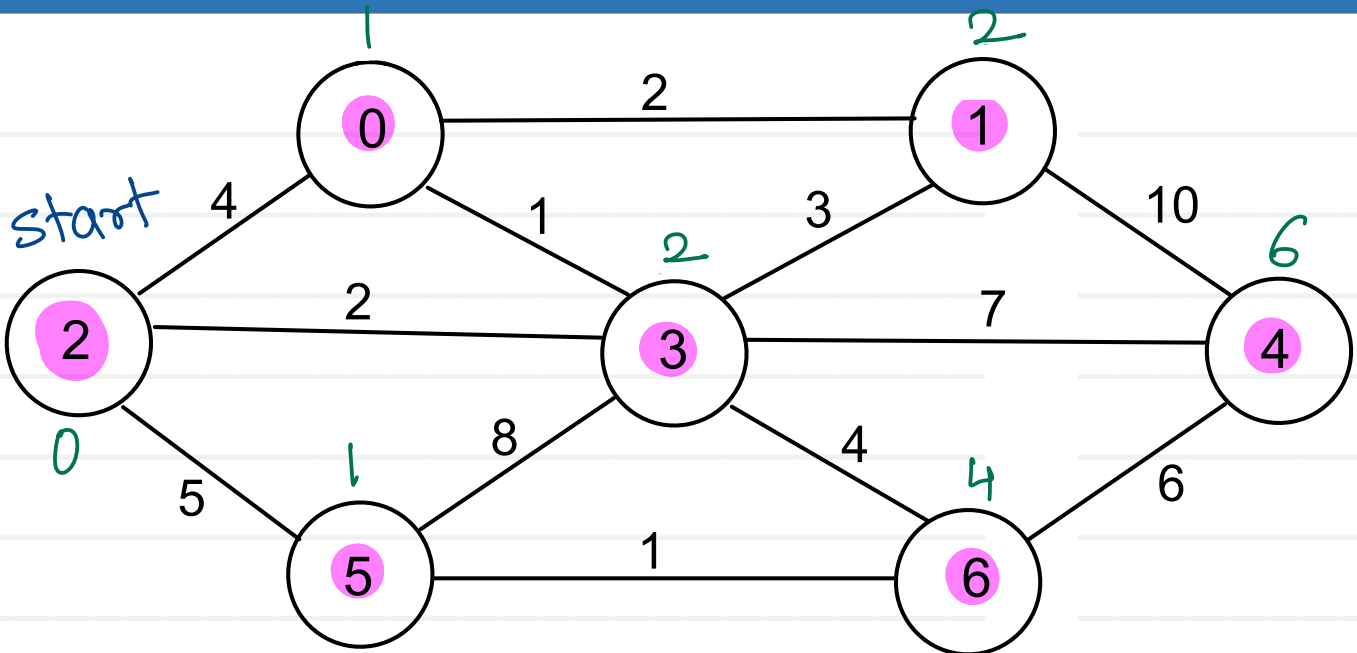
Prim's Algorithm

1. Create a set mst to keep track of vertices included in MST.
2. Also keep track of parent of each vertex. Initialize parent of each vertex -1.
3. Assign a key to all vertices in the input graph. Key for all vertices should be initialized to INF. The start vertex key should be 0.
4. While mst doesn't include all the vertices
 - i. Pick a vertex u which is not there in mst and has minimum key.
 - ii. Include vertex u to mst.
 - iii. Update key and parent of all adjacent vertices of u.
 - a. For each adjacent vertex v,
if weight of edge u-v is less than the current key of v,
then update the key as weight of u-v.
 - b. Record u as parent of v.



$$\text{if}(\text{adjmat}[u][v] < \text{key}[v])$$
$$\text{key}[v] = \text{adjmat}[u][v]$$

Prim's Algorithm



	K	P
0	1	3
1	2	0
2	0	-1
3	2	2
4	6	6
5	1	6
6	4	3

16

	K	P
0	4	2
1	∞	-1
2	0	-1
3	2	2
4	∞	-1
5	5	2
6	∞	-1

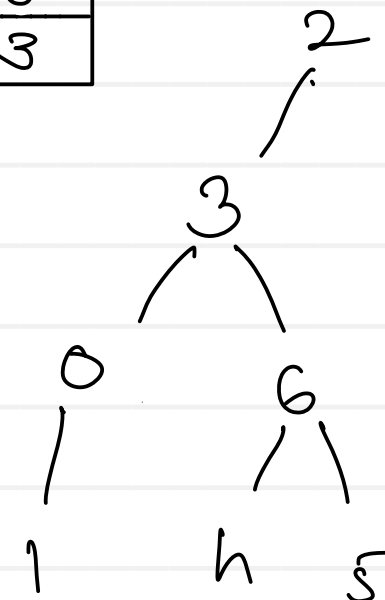
	K	P
0	1	3
1	3	3
2	0	-1
3	2	2
4	7	3
5	5	2
6	4	3

	K	P
0	1	3
1	2	0
2	0	-1
3	2	2
4	7	3
5	5	2
6	4	3

	K	P
0	1	3
1	2	0
2	0	-1
3	2	2
4	7	3
5	5	2
6	4	3

	K	P
0	1	3
1	2	0
2	0	-1
3	2	2
4	6	6
5	1	6
6	4	3

	K	P
0	1	3
1	2	0
2	0	-1
3	2	2
4	6	6
5	1	6
6	4	3





Sunbeam Institute of Information Technology

Pune and Karad

Module – Data Structures and Algorithms

Trainer - Devendra Dhande

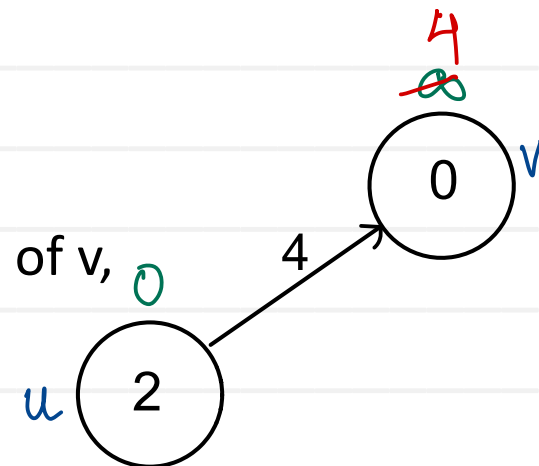
Email – devendra.dhande@sunbeaminfo.com

Dijkstra's Algorithm

1. Create a set spt to keep track of vertices included in shortest path tree.
2. Track distance of all vertices in the input graph. Distance for all vertices should be initialized to INF. The start vertex distance should be 0.
3. While spt doesn't include all the vertices
 - i. Pick a vertex u which is not there in spt and has minimum distance.
 - ii. Include vertex u to spt.
 - iii. Update distances of all adjacent vertices of u.

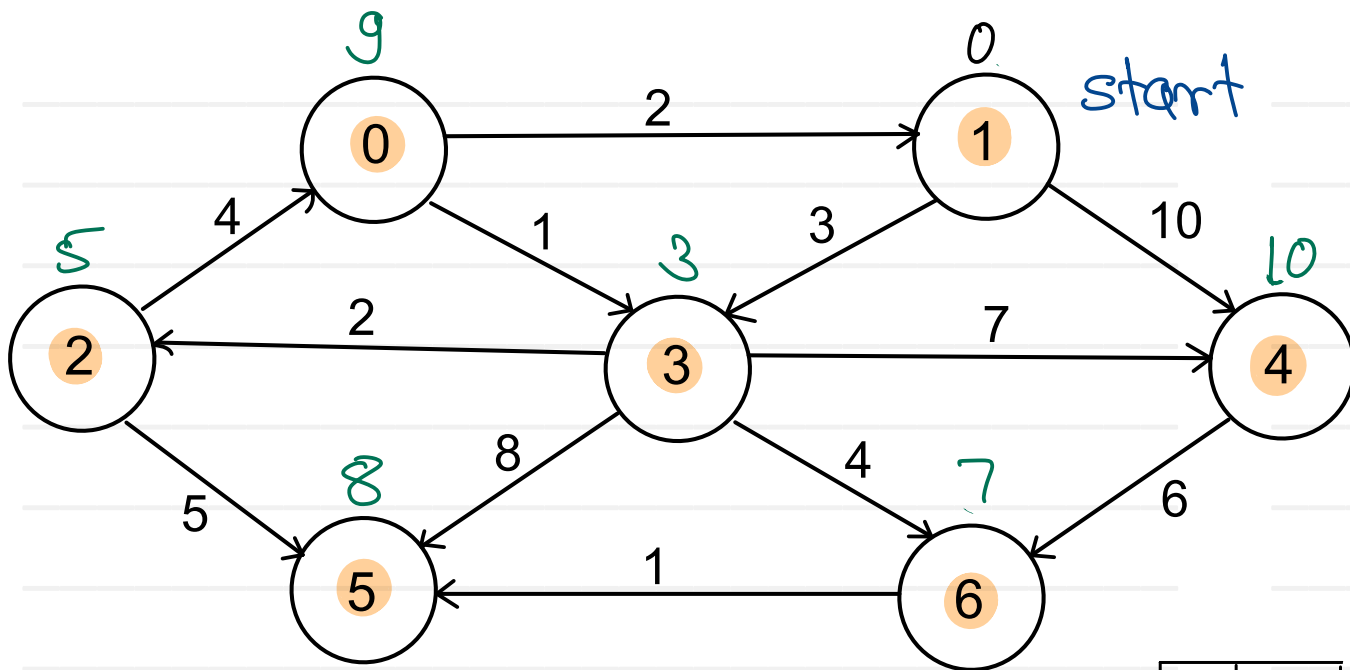
For each adjacent vertex v,

if distance of u + weight of edge u-v is less than the current distance of v,
then update its distance as distance of u + weight of edge u-v.



$$\text{if} (\text{dist}[u] + \text{adjmat}[u][v] < \text{dist}[v])$$
$$\text{dist}[v] = \text{dist}[u] + \text{adjmat}[u][v]$$

Dijkstra's Algorithm



	D
0	9
1	0
2	5
3	3
4	10
5	8
6	7

	D
0	∞
1	0
2	∞
3	3
4	10
5	∞
6	∞

	D
0	∞
1	0
2	5
3	3
4	10
5	11
6	7

	D
0	9
1	0
2	5
3	3
4	10
5	10
6	7

	D
0	9
1	0
2	5
3	3
4	10
5	8
6	7

	D
0	9
1	0
2	5
3	3
4	10
5	8
6	7

	D
0	9
1	0
2	5
3	3
4	10
5	8
6	7

Floyd Warshall Algorithm

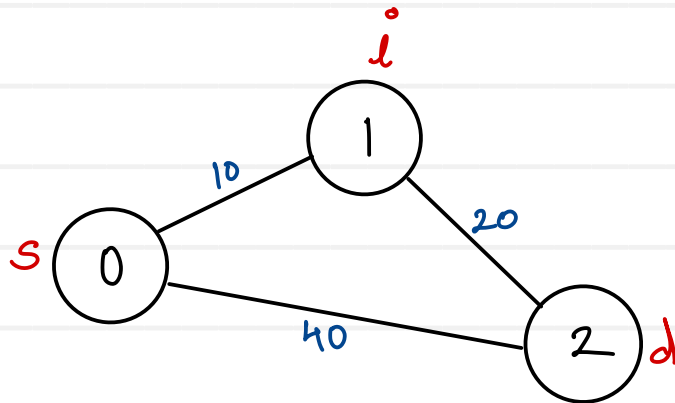
1. Create distance matrix to keep distance of every vertex from each vertex.

Initially assign it with weights of all edges among vertices
(i.e. adjacency matrix).

2. Consider each vertex (i) in between pair of any two vertices (u, v) and
find the optimal distance between s & d considering intermediate vertex

i.e. $\text{dist}(u,v) = \text{dist}(u,i) + \text{dist}(i,v)$,

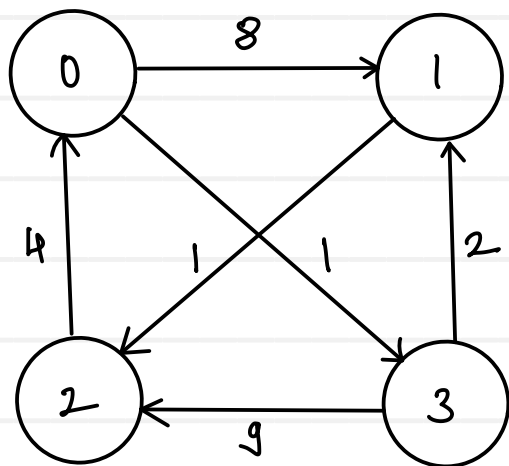
if $\text{dist}(u,i) + \text{dist}(i,v) < \text{dist}(u,v)$.



dist via i *direct dist*

$$\text{if}(\text{dist}[s][i] + \text{dist}[i][d] < \text{dist}[s][d])$$
$$\text{dist}[s][d] = \text{dist}[s][i] + \text{dist}[i][d];$$

Floyd Warshall Algorithm



$$d = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \end{matrix} & \begin{bmatrix} \infty & 8 & \infty & 1 \\ \infty & \infty & 1 & \infty \\ 4 & \infty & \infty & \infty \\ \infty & 2 & 9 & \infty \end{bmatrix} \end{matrix}$$

$$d = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \end{matrix} & \begin{bmatrix} 0 & 8 & \infty & 1 \\ \infty & 0 & 1 & \infty \\ 4 & \infty & 0 & \infty \\ \infty & 2 & 9 & 0 \end{bmatrix} \end{matrix}$$

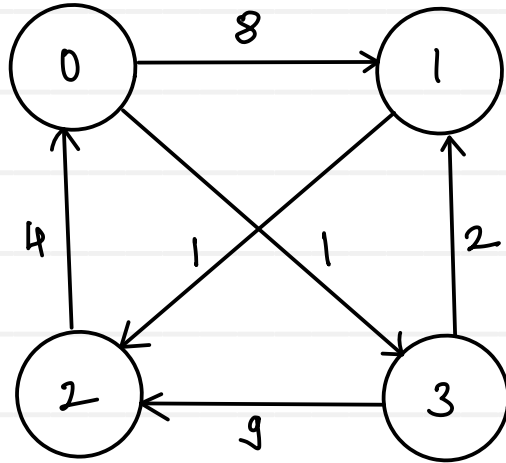
$$d_0 = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \end{matrix} & \begin{bmatrix} 0 & 8 & \infty & 1 \\ \infty & 0 & 1 & \infty \\ 4 & 12 & 0 & 5 \\ \infty & 2 & 9 & 0 \end{bmatrix} \end{matrix}$$

$$d_1 = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \end{matrix} & \begin{bmatrix} 0 & 8 & 9 & 1 \\ \infty & 0 & 1 & \infty \\ 4 & 12 & 0 & 5 \\ \infty & 2 & 3 & 0 \end{bmatrix} \end{matrix}$$

$$d_2 = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \end{matrix} & \begin{bmatrix} 0 & 8 & 9 & 1 \\ 5 & 0 & 1 & 6 \\ 4 & 12 & 0 & 5 \\ 7 & 2 & 3 & 0 \end{bmatrix} \end{matrix}$$

$$d_3 = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \end{matrix} & \begin{bmatrix} 0 & 3 & 4 & 1 \\ 5 & 0 & 1 & 6 \\ 4 & 7 & 0 & 5 \\ 7 & 2 & 3 & 0 \end{bmatrix} \end{matrix}$$

Floyd Warshall Algorithm

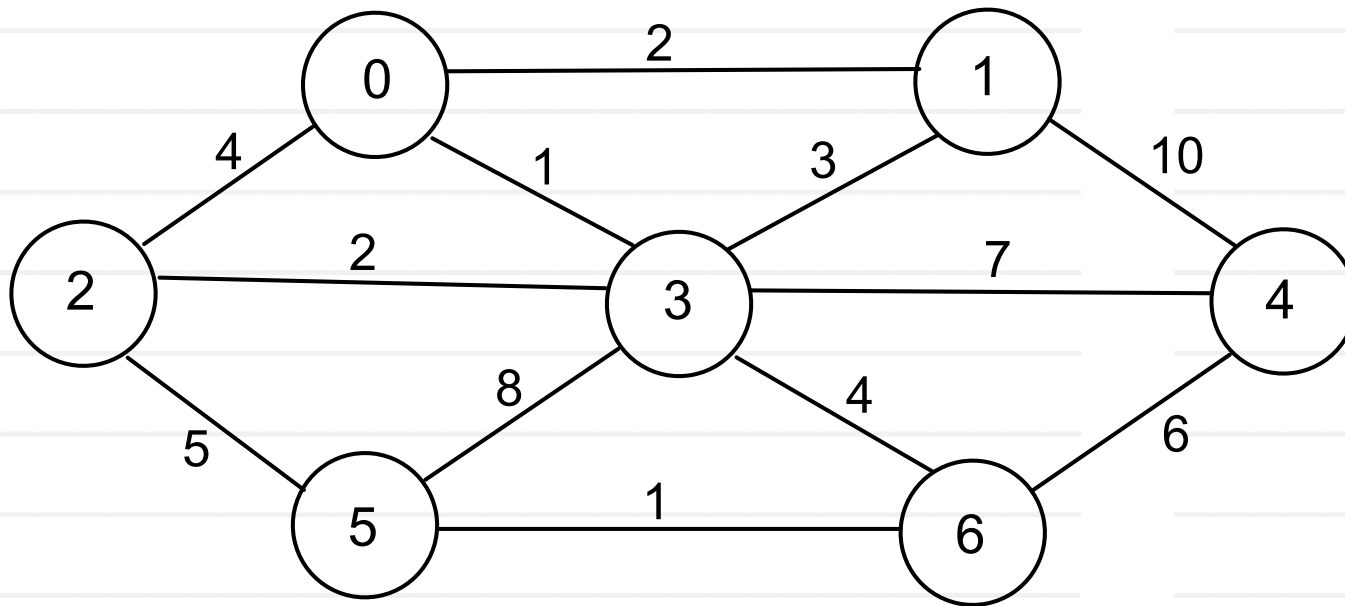


```

void FloydWarshallAlgorithm( ) {
    int dist[V][V] = new int[V][V];
    for(int s=0 ; s<V ; s++) {
        for(int d=0 ; d<V ; d++)
            dist[s][d] = adjmat[s][d];
        dist[s][s] = 0;
    }
    for(int i=0 ; i<V ; i++) {
        for(int s=0 ; s<V ; s++) {
            for(int d=0 ; d<V ; d++) {
                if(dist[s][i] + dist[i][d] < dist[s][d])
                    dist[s][d] = dist[s][i] + dist[i][d];
            }
        }
    }
}
  
```

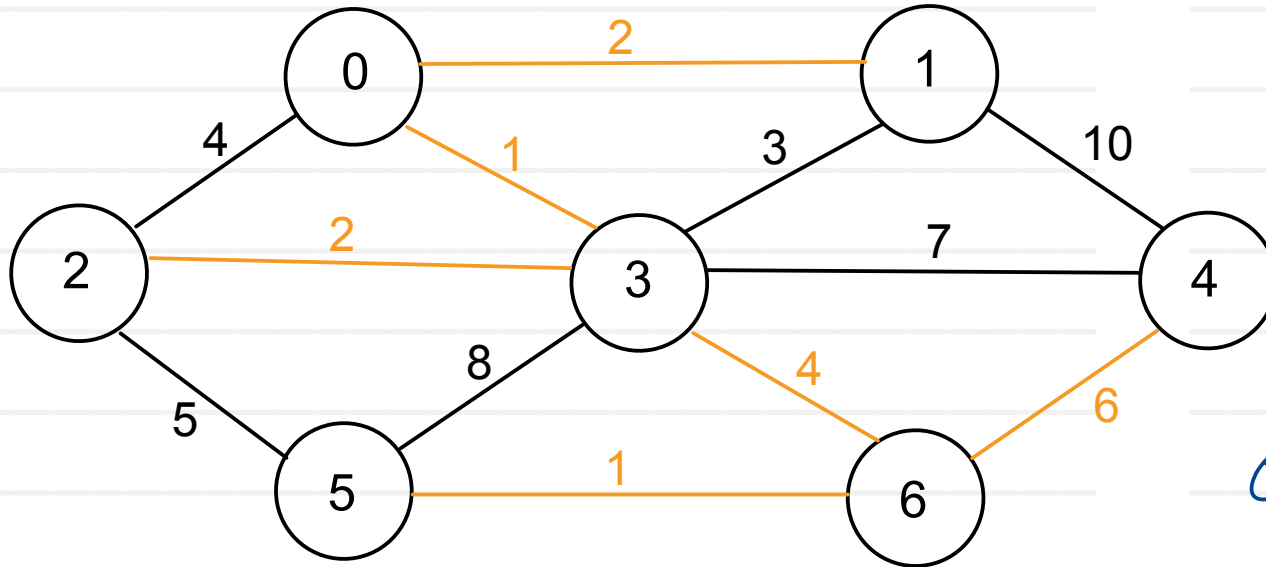
Union Find Algorithm

1. Consider all vertices as disjoint sets (parent = -1).
2. For each edge in the graph
 1. Find set(root) of first vertex.
 2. Find set(root) of second vertex.
 3. If both are in same set(same root), cycle is detected.
 4. Otherwise, merge(Union) both the sets i.e. add root of first set under second set



Union Find Algorithm

	0	1	2	3	4	5	6
Parents	3	2	5	1	5	-1	5



```

int find (int v, int parents[]) {
    while (parents[v] != -1)
        v = parents[v];
    return v;
}
    
```

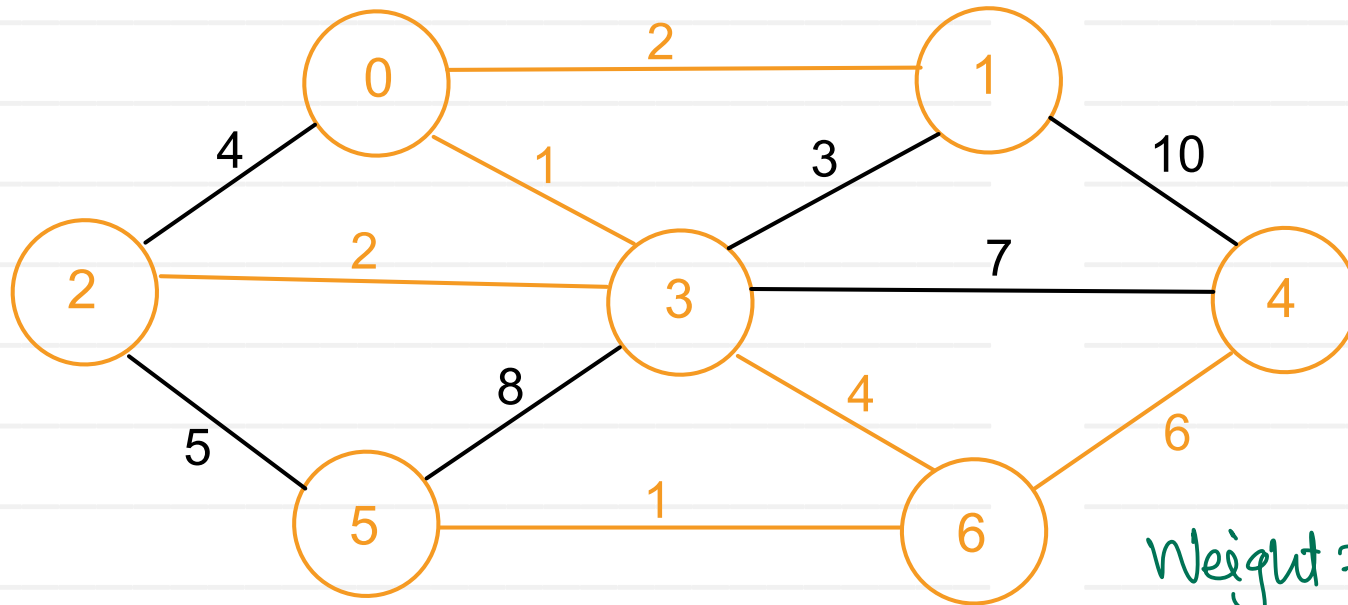
```

void union(int sr, int dr, int
           parents[]) {
    parents[sr] = dr;
}
    
```

sr	dr	s	d	
0	3	0	3	-1
6	5	6	5	-1
3	1	0	1	-2
1	2	3	2	-2
2	2	1	3	-3
2	2	2	0	-4
2	5	3	6	-4
5	5	2	5	-5
4	5	4	6	-6
3	4	3	4	-7
3	5	3	5	-8
1	4	1	4	-1

Kruskal's Algorithm

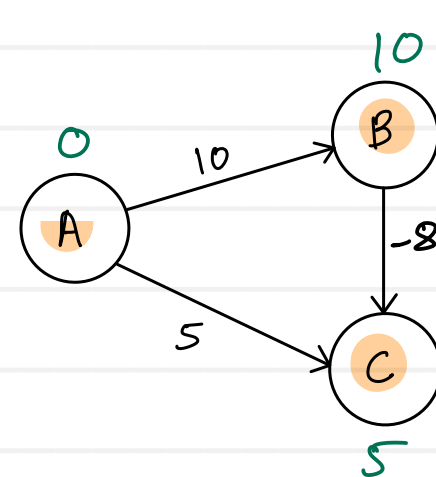
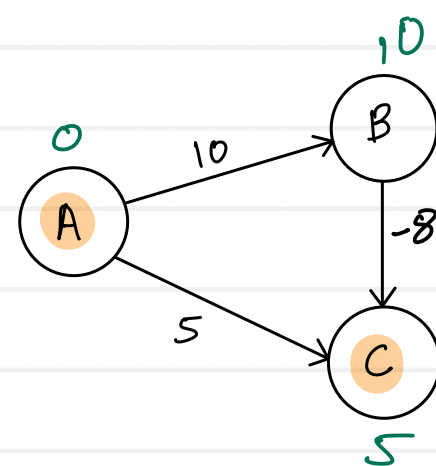
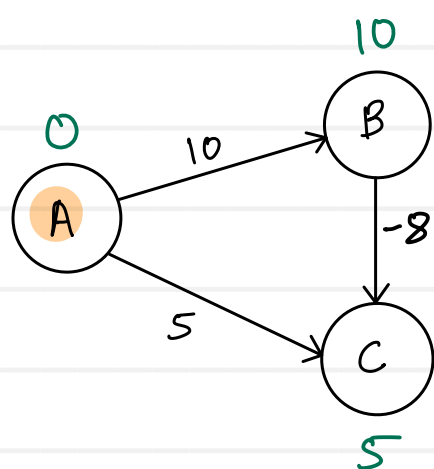
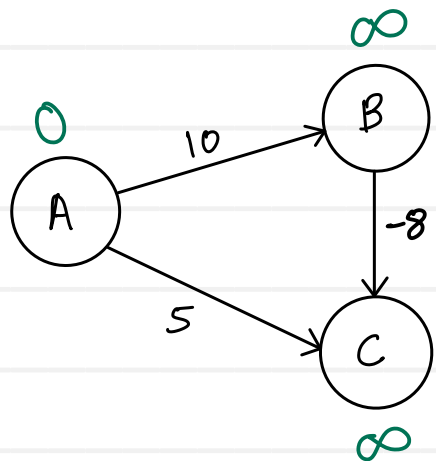
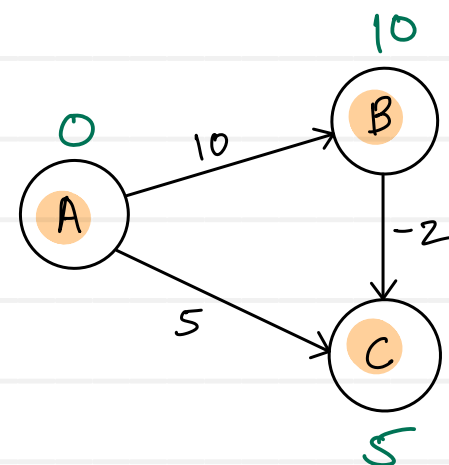
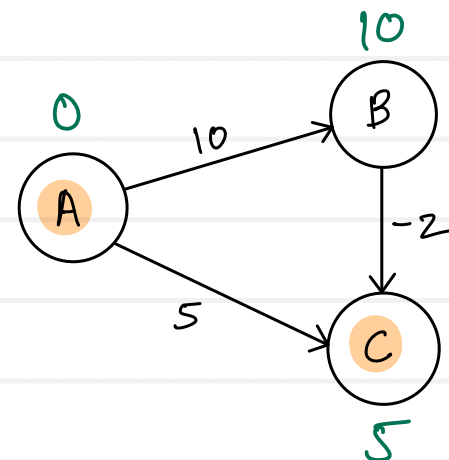
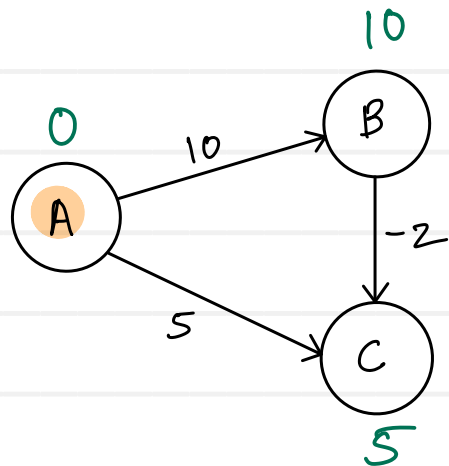
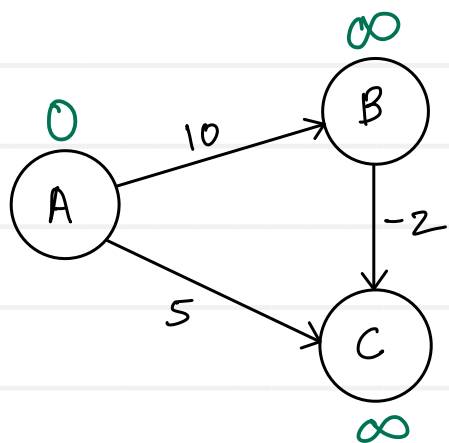
1. Sort all the edges in ascending order of their weight.
2. Pick the smallest edge.
Check if it forms a cycle with the spanning tree formed so far.
If cycle is not formed, include this edge.
Else, discard it.
3. Repeat step 2 until there are $(V-1)$ edges in the spanning tree.



Weight = 16

0	3	-	1	✓
6	5	-	1	✓
0	1	-	2	✓
3	2	-	2	✓
1	3	-	3	✗
2	0	-	4	✗
3	6	-	4	✓
2	5	-	5	✗
4	6	-	6	✓
3	4	-	7	
3	5	-	8	
1	4	-	10	

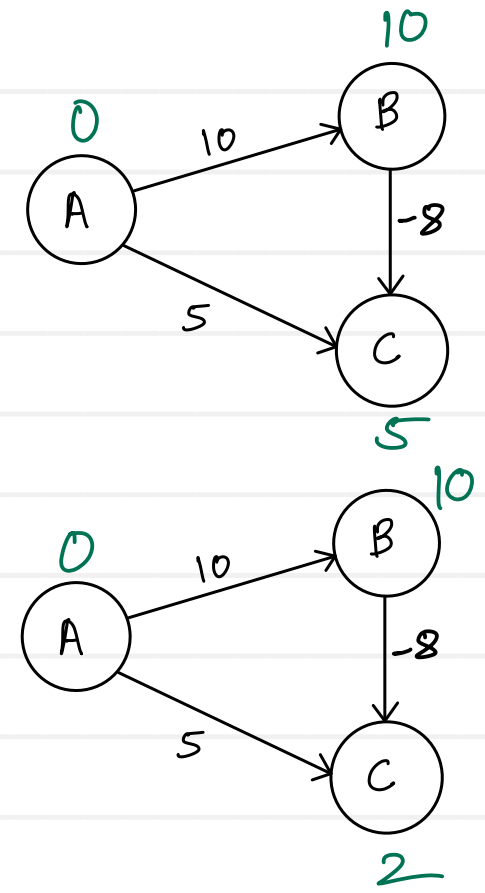
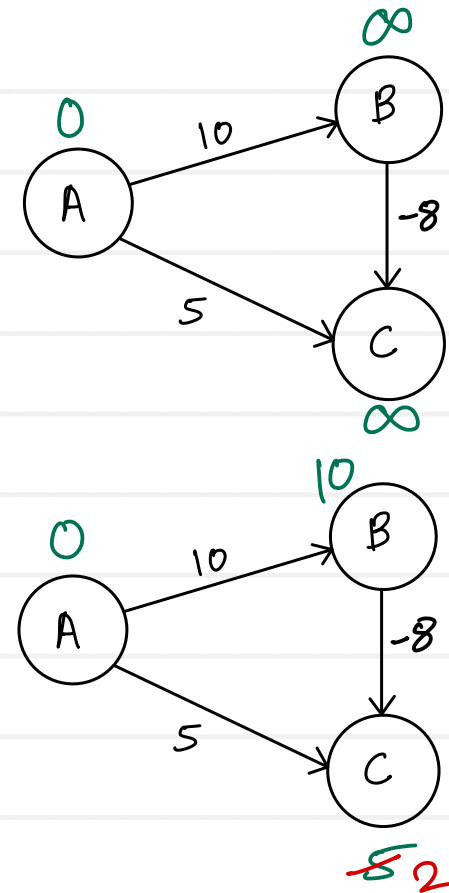
Dijkstra's Algorithm





Bellman Ford Algorithm

1. Initializes distances from the source to all vertices as infinite and distance to the source itself as 0.
2. Calculates shortest distance $V-1$ times:
For each edge $u-v$,
if $\text{dist}[v] > \text{dist}[u] + \text{weight of edge } u-v$,
then update $\text{dist}[v]$, so that
 $\text{dist}[v] = \text{dist}[u] + \text{weight of edge } u-v$.
3. Check if negative edge cycle in the graph:
For each edge $u-v$,
if $\text{dist}[v] > \text{dist}[u] + \text{weight of edge } (u,v)$,
then graph has -ve weight cycle.



Graph applications

- Graph represents flow of computation/tasks. It is used for resource planning and scheduling. MST algorithms are used for resource conservation. DAG are used for scheduling in Spark or Tez.
- In OS, process and resources are treated as vertices and their usage is treated as edges. This resource allocation algorithm is used to detect deadlock.
- In social networking sites, each person is a vertex and their connection is an edge. In Facebook person search or friend suggestion algorithms use graph concepts.
- In world wide web, web pages are like vertices; while links represents edges. This concept can be used at multiple places.
 - Making sitemap
 - Downloading website or resources
 - Developing web crawlers
 - Google page-rank algorithm
- Maps uses graphs for showing routes and finding shortest paths. Intersection of two (or more) roads is considered as vertex and the road connecting two vertices is considered to be an edge.

Merge sort

1. Divide array in two parts
2. Sort both partitions individually (by merge sort only)
3. Merge sorted partitions into temporary array
4. Overwrite temporary array into original array

No. of elements = n

No. of levels = $\log n$

Comps per level $\propto n$

Total comps = $n \log n$

Time $\propto$ comps

Time $\propto n \log n$

Best
Avg
Worst

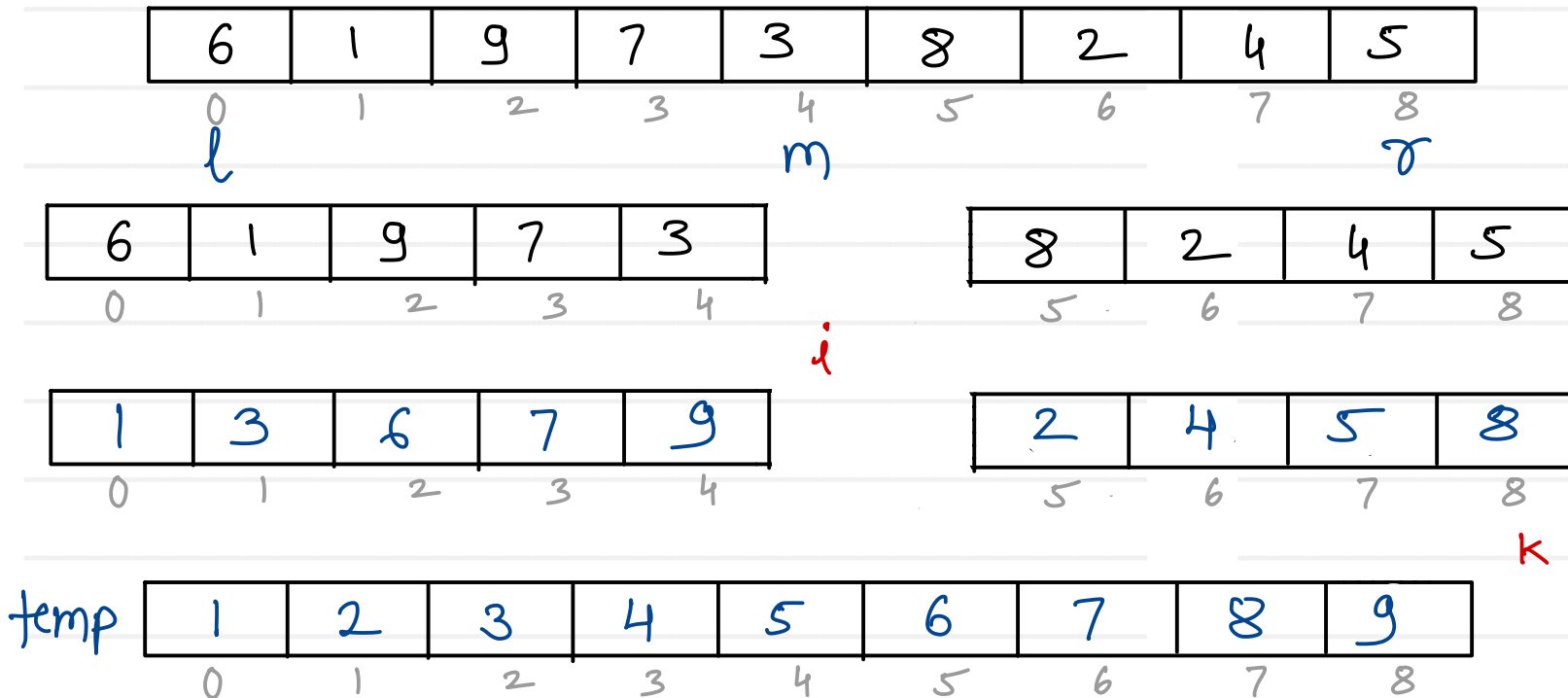
$$T(n) = O(n \log n)$$

To merge sorted partitions
we need temp array
temp $\rightarrow$ processing variable

AS $\propto$ size of (temp)

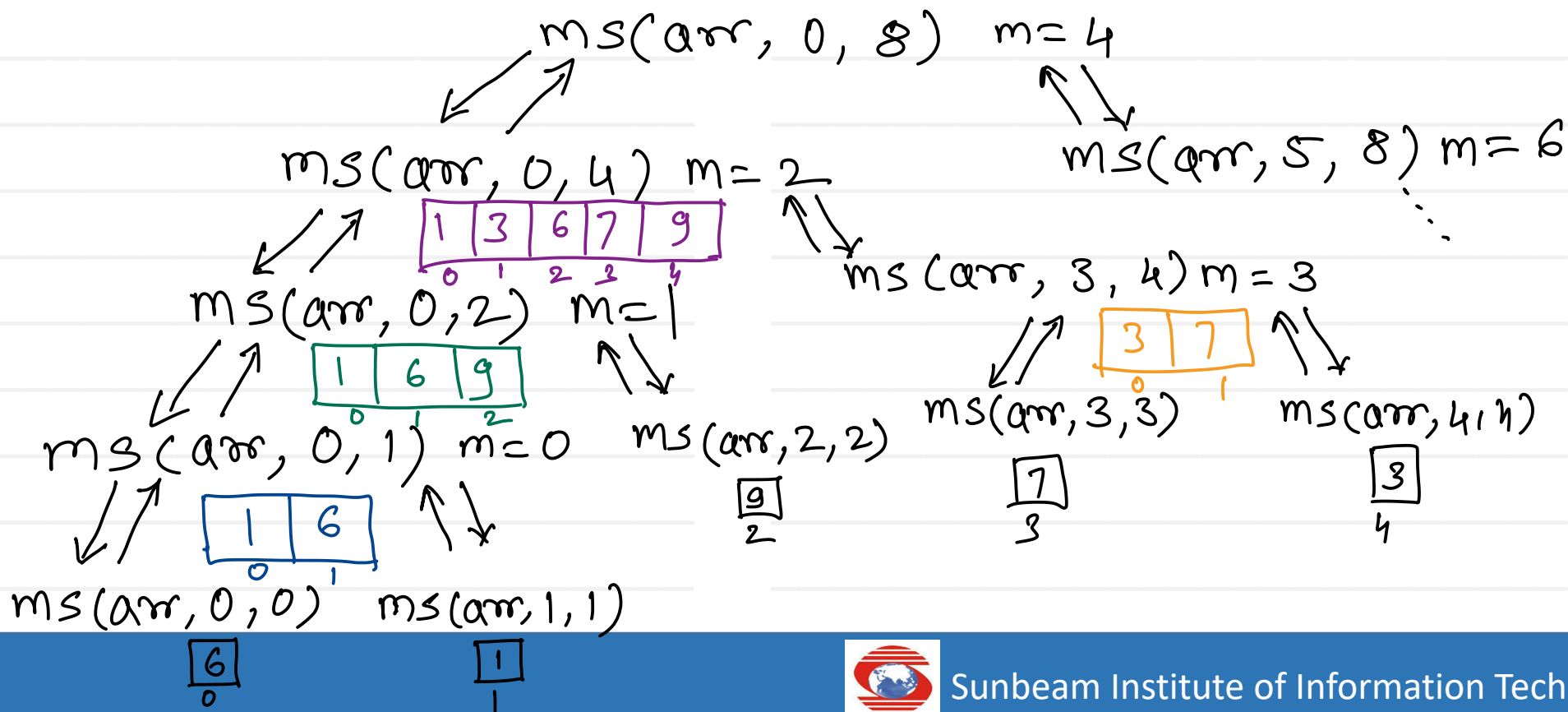
AS $\propto n$

$$AS(n) = O(n)$$



Merge sort

1	3	6	7	9				
1	3	6	7	9				
1	3	6	7	9				
6	1	8	7	3	8	2	4	5
0	1	2	3	4	5	6	7	8



Quick sort

1. Select pivot/axis/reference element from array
2. Arrange lesser elements on left side of pivot
3. Arrange greater elements on right side of pivot
4. Sort left and right side of pivot again (by quick sort)

→ selection of pivot :

- 1) extreme left or right
- 2) middle element
- 3) median ← random 3
random 5

no. of elements = n

no. of levels = $\log n$

comps per level = n

Total comps = $n \log n$

Time $\propto n \log n$

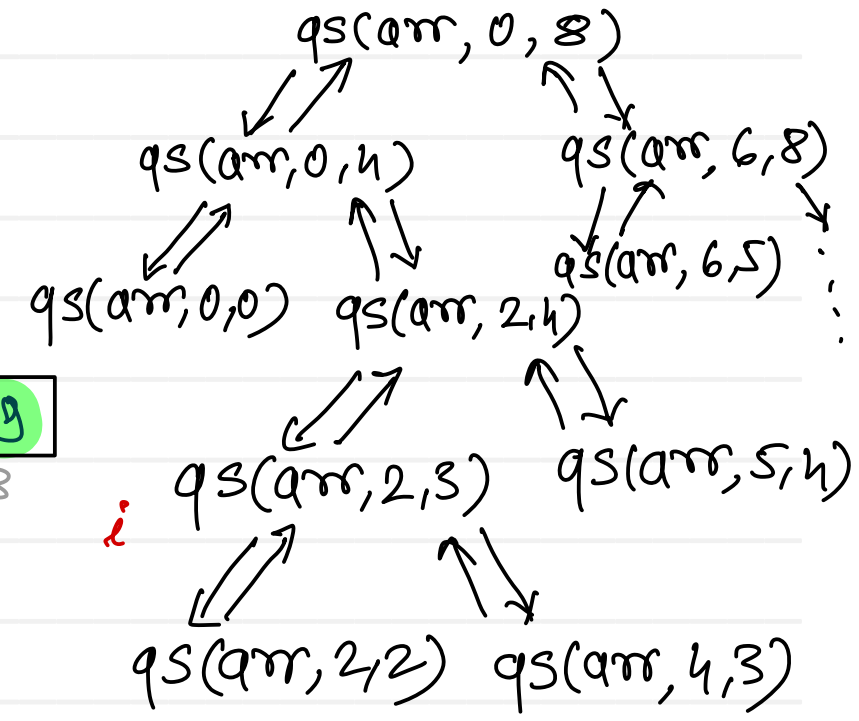
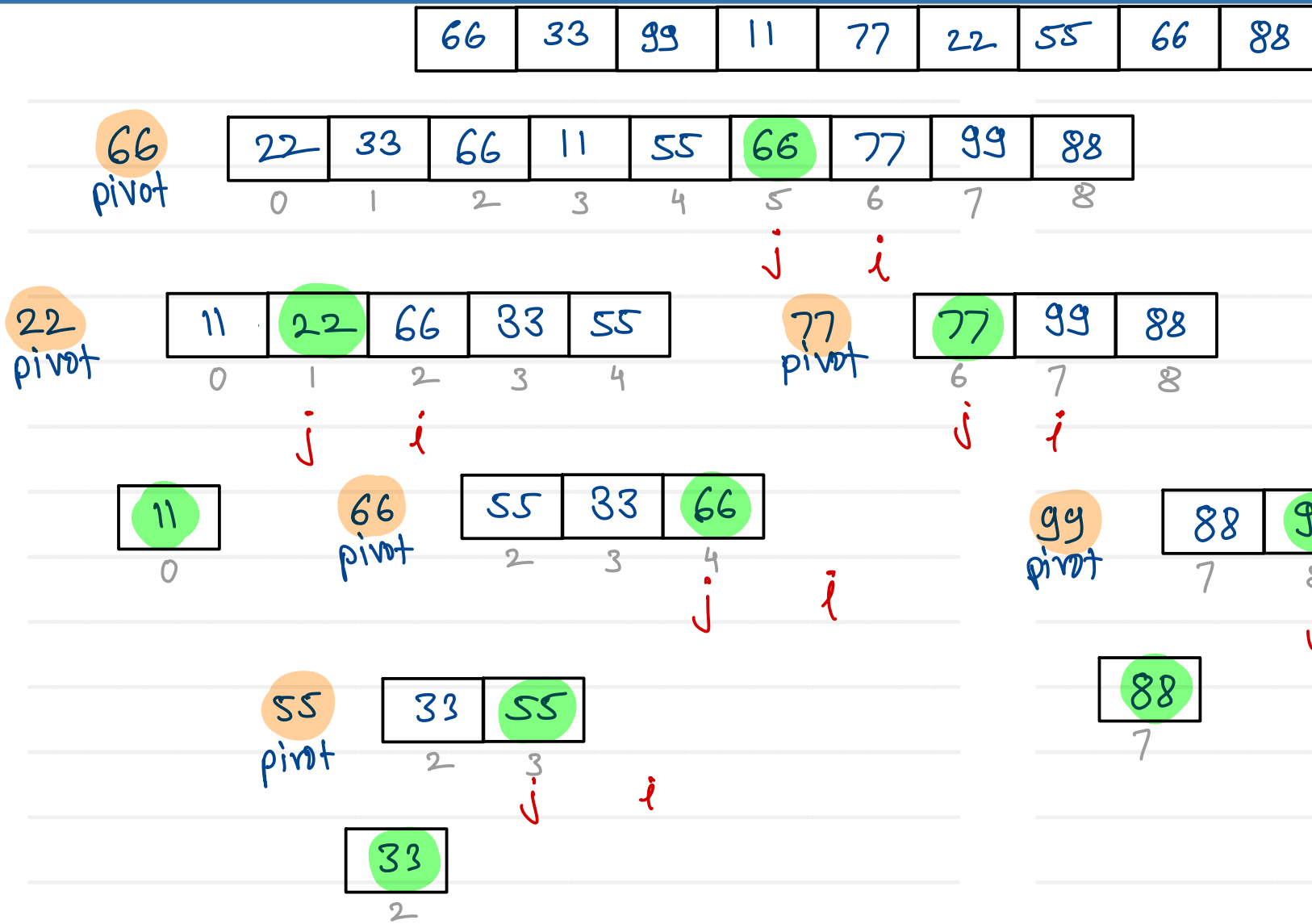
Best 2
Avg } $T(n) = O(n \log n)$

11	22	33	44	55	} no. of levels $\propto n$ per level comps = n comps = n^2
	22	33	44	55	
		33	44	55	
			44	55	
				55	

Worst } $T(n) = O(n^2)$

- Time complexity of quick sort is dependent on selection of pivot

Quick sort



Array : linear search — $O(n)$
binary search — $O(\log n)$

Linked List : search — $O(n)$

Binary tree : search — $O(n)$

BST : search — $O(\log n)$

Graph : search — $O(V^2)$

Hash Table : search — $O(1)$

- hashing is a technique in which data can be inserted, deleted and searched in constant average time $O(1)$
- Implementation of hashing is known as hash table
- Hash table is array of fixed size in which elements are stored in key - value pairs

Array - Hash table
Index - Slot

- In hash table only unique keys are stored
- Every key is mapped with one slot of the table and this is done with the help of mathematical function known as hash function

size=10

key value

8-V1

3-V2

10-V3

4-V4

6-V5

13-V6

collision

10, V3	0
	1
	2
3, V2	3
4, V4	4
	5
6, V5	6
	7
8, V1	8
	9

Hash Table

$$h(k) = k \% \text{size}$$

$$h(8) = 8 \% 10 = 8$$

$$h(3) = 3 \% 10 = 3$$

$$h(10) = 10 \% 10 = 0$$

$$h(4) = 4 \% 10 = 4$$

$$h(6) = 6 \% 10 = 6$$

$$h(13) = 13 \% 10 = 3$$

Collision:

Whenever multiple keys yield same slot

Collision handling techniques:

1. Closed addressing
 - i. Linear probing
 - ii. Quadratic probing
 - iii. Double hashing
2. Open addressing

Add: $O(1)$

1. slot = $h(k)$
2. arr[slot] = data

Search: $O(1)$

1. slot = $h(k)$
2. return arr[slot]

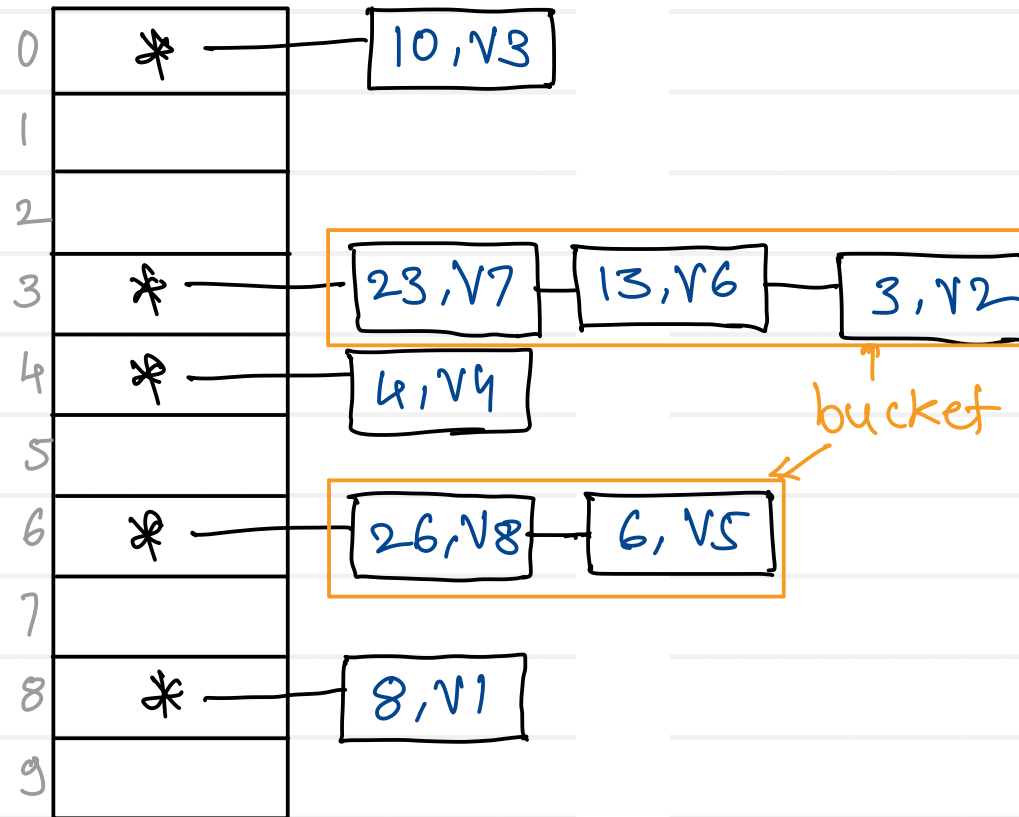
Delete: $O(1)$

1. slot = $h(k)$
2. arr[slot] = null

Closed Addressing / Chaining / Separate Chaining

Size = 10

8-V1
3-V2
10-V3
4-V4
6-V5
13-V6
23-V7
26-V8



Hash Table

$$h(k) = k \% \text{size}$$

$$h(13) = 13 \% 10 = 3$$

$$h(23) = 23 \% 10 = 3$$

$$h(26) = 26 \% 10 = 6$$

Advantage:

- multiple key-value pairs can be stored into hash table

Disadvantages:

- 1) key value pairs are stored outside the table
- 2) space requirement is more due linked list.
- 3) Worst case time complexity is $O(n)$ - when maximum keys will yield same slot.



Thank you!!!

Devendra Dhande

devendra.dhande@sunbeaminfo.com



Sunbeam Institute of Information Technology

Pune and Karad

Module – Data Structures and Algorithms

Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

Open addressing - Linear probing

size = 10

	10, V3	0
8-V1		1
3-V2		2
10-V3	3, V2	3
4-V4	4, V4	4
6-V5	13, V6	5
13-V6	6, V5	6
		7
	8, V1	8
		9

Hash Table

$$h(k) = k \% \text{ size}$$

$$h(k, i) = [h(k) + \underline{f(i)}] \% \text{ size}$$

$$f(i) = \underline{i}$$

where $i = 1, 2, 3, \dots$

$$h(13) = 13 \% 10 = 3 \text{ (C)}$$

$$h(13, 1) = [3 + 1] \% 10 = 4 \text{ (1st) (C)}$$

$$h(13, 2) = [3 + 2] \% 10 = 5 \text{ (2nd)}$$

Probing : finding next empty slot to store key value pair whenever collision will occur

Primary clustering : to find next empty, need to take long run of filled slots, "near" key position.

Open addressing - Quadratic probing

size = 10

	10, V3	0
8-V1		1
3-V2		2
10-V3	3, V2	3
4-V4	4, V4	4
6-V5		5
13-V6	6, V5	6
	13, V6	7
	8, V1	8
		9

Hash Table

$$h(k) = k \% \text{ size}$$

$$h(k, i) = [h(k) + f(i)] \% \text{ size}$$

$$f(i) = i^2$$

where $i = 1, 2, 3, \dots$

$$h(13) = 13 \% 10 = 3 \text{ (c)}$$

$$h(13, 1) = [3 + 1] \% 10 = 4 \text{ (1st) (c)}$$

$$h(13, 2) = [3 + 4] \% 10 = 7 \text{ (2nd)}$$

- in this technique, there is no guarantee of getting free slot for the key.

Open addressing - Quadratic probing

size = 10

	10, V3	0
8-V1		1
3-V2	23, V7	2
10-V3	3, V2	3
4-V4	4, V4	4
6-V5		5
13-V6	6, V5	6
23, V7	13, V6	7
	8, V1	8
		9

Hash Table

$$h(k) = k \% \text{ size}$$

$$h(k, i) = [h(k) + f(i)] \% \text{ size}$$

$$f(i) = i^2$$

where $i = 1, 2, 3, \dots$

$$h(23) = 23 \% 10 = 3 \text{ (C)}$$

$$h(23, 1) = [3 + 1] \% 10 = 4 \text{ (1st) (C)}$$

$$h(23, 2) = [3 + 4] \% 10 = 7 \text{ (2nd) (C)}$$

$$h(23, 3) = [3 + 9] \% 10 = 2 \text{ (3rd)}$$

Secondary clustering: to find next empty slot, need to take long run of filled slots "away" key position.

Open addressing - Double hashing

size = 11

8-V1

3-V2

10-V3

25-V4

Ⓢ

	0
	1
	2
3, V2	3
	4
	5
25, V4	6
	7
8, V1	8
	9
10, V3	10

Hash Table

- primary & secondary clustering is removed because for two distinct key, different path is followed to find free slot.

$$h_1(k) = k \% \text{ size}$$

$$h_2(k) = 7 - (k \% 7)$$

$$h(k, i) = [h_1(k) + i * h_2(k)] \% \text{ size}$$

$$h_1(25) = 25 \% 11 = 3 \quad \text{Ⓢ}$$

$$h_2(25) = 7 - (25 \% 7) = 3$$

$$h(25, 1) = [3 + 1 * 3] \% 11 = 6 \quad (1^{st})$$

$$h_1(36) = 3$$

$$h_2(36) = 6$$

$$h(36, 1) = [3 + 1 * 6] \% 11 = 9$$

Rehashing

$$\text{Load factor} = \frac{n}{N}$$

(λ)

n - number of elements (key-value) present in hash table

N - number of total slots in hash table

- Load factor ranges from 0 to 1.
- If $n < N$ Load factor < 1 - free slots are available
- If $n = N$ Load factor = 1 - free slots are not available

e.g. size = 10

$n = 6$, $N = 10$

$$\lambda = \frac{n}{N} = \frac{6}{10} = 0.6$$

$\lambda = 0.6$ means, hash table is 60 % filled.

- In rehashing, whenever hash table will be filled more than 60 or 70 % size of hash table is increased by twice
- Existing key value pairs are remapped according to new size

Algorithm Design Techniques

- 1) Divide & Conquer - merge & quick sort
- 2) Greedy - optimal solution

Prims, Dijkstras, Kruskals

- 3) Dynamic programming
Bellaman Ford, Floyd Warshal

Memoization

dp array $\rightarrow$ 1D or 2D array
+
recursion

(Top-down approach)

Tabulation

1D or 2D array
+
loops

(Bottom-up approach)

Problem solving technique : Greedy approach

- A greedy algorithm is any algorithm that follows the problem-solving heuristic of making the locally optimal choice at each stage with the intent of finding a global optimum.
- We can make choice that seems best at the moment and then solve the sub-problems that arise later.
- The choice made by a greedy algorithm may depend on choices made so far, but not on future choices or all the solutions to the sub-problem.
- It iteratively makes one greedy choice after another, reducing each given problem into a smaller one.
- A greedy algorithm never reconsiders its choices.
- A greedy strategy may not always produce an optimal solution.

e.g. Greedy algorithm decides minimum number of coins to give while making change.

coins available : 50, 20, 10, 5, 2, 1

Recursion

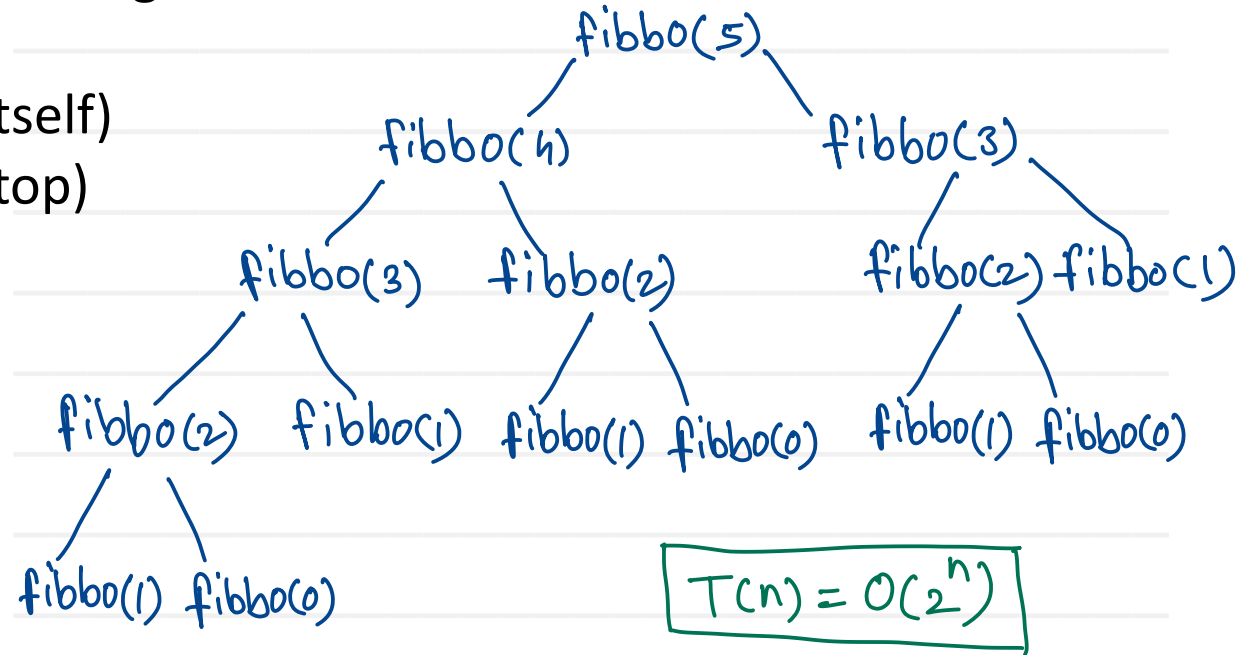
- Function calling itself is called as recursive function.
- For each function call stack frame is created on the stack.
- Thus it needs more space as well as more time for execution.
- However recursive functions are easy to program.
- Typical divide and conquer problems are solved using recursion.
- For recursive functions two things are must

- Recursive call (Explain process in terms of itself)
- Terminating or base condition (Where to stop)

e.g. Fibonacci Series

- Recursive formula
 $T_n = T_{n-1} + T_{n-2}$
- Terminating condition
 $T_1 = T_2 = 1$
- Overlapping sub-problem

```
int fibbo(int n) {  
    if(n==0 || n==1)  
        return n;  
    return fibbo(n-1)+fibbo(n-2);  
}
```



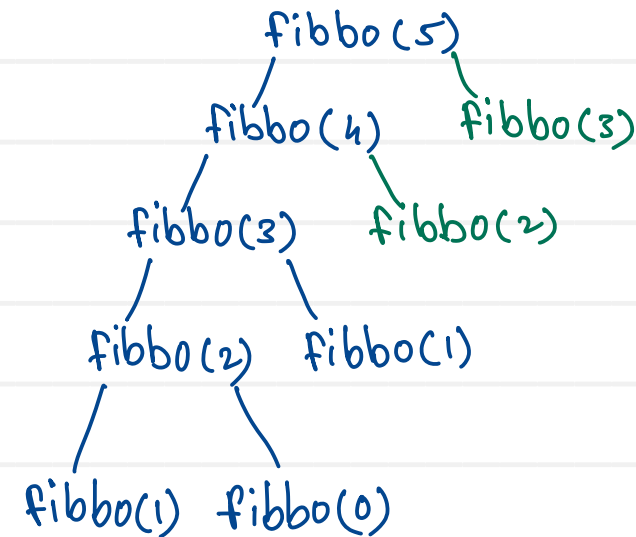
Memoization

- It's based on the Latin word memorandum, meaning "to be remembered".
- Memoization is a technique used in computing to speed up programs.
- This is accomplished by memorizing the calculation results of processed input such as the results of function calls.
- If the same input or a function call with the same parameters is used, the previously stored results can be used again and unnecessary calculation are avoided.
- Need to rewrite recursive algorithm. Using simple arrays or map/dictionary.

dp

0	1	1	2	3	5
✓	✓	✓	✓	✓	✓
0	1	2	3	4	5

```
int fibbo(int n) {  
    if (n == 0 || n == 1)  
        return n;  
    if (dp[n] != -1)  
        return dp[n];  
    dp[n] = fibbo(n-1) + fibbo(n-2);  
    return dp[n];  
}
```



Dynamic Programming

- Dynamic programming is another optimization over recursion.
- Typical DP problem give choices (to select from) and ask for optimal result (maximum or minimum).
- Technically it can be used for the problems having two properties
 - Overlapping sub-problems
 - Optimal sub-structure
- To solve problem, we need to solve its sub-problems multiple times.
- Optimal solution of problem can be obtained using optimal solutions of its sub-problems.
- Greedy algorithms pick optimal solution to local problem and never reconsider the choice done.
- DP algorithms solve the sub-problem in a iteration and improves upon it in subsequent iterations.

Dynamic programming

dp

0	1	1	2	3	5
0	1	2	3	4	5

```
int fibbo(int n) {  
    dp[0] = 0;  
    dp[1] = 1;  
    for (int i = 2; i <= n; i++)  
        dp[i] = dp[i-1] + dp[i-2];  
    return dp[n];  
}
```



Thank you!!!

Devendra Dhande

devendra.dhande@sunbeaminfo.com